UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica Applicata (DIEM)





Corso di Laurea Magistrale in Ingegneria Informatica

IoT and Embedded Systems

Scrambler Audio Real-Time basato su un'architettura SoC

Progettazione e implementazione hardware di un sistema di crittografia a flusso e accelerazione su piattaforma Xilinx Zynq-7000

Corso di SYSTEM ON CHIP

Prof. E. Napoli

Autore: Salvatore Coppa

s.coppa@studenti.unisa.it

Anno Accademico 2025/2026

Indice

1 Introduzione e Contesto **4**
- 1.1 Evoluzione delle comunicazioni sicure . . . 4
- 1.2 Dall'analogico al digitale: il ruolo del Ciphering . . . 5
  - 1.2.1 Effetti Spettrali dello Scrambling e Inviolabilità del Segnale . . . 5
- 1.3 Obiettivo del progetto: l'approccio SoC . . . 6
- 1.4 Struttura del lavoro . . . 6

2 Architettura del Sistema e Teoria Operativa **7**
- 2.1 Visione d'insieme del flusso dati . . . 7
- 2.2 Il principio dello Scrambling e il generatore LFSR . . . 8
  - 2.2.1 Linear Feedback Shift Register (LFSR) di Galois . . . 8
  - 2.2.2 Periodicità e limiti dell'implementazione a 16-bit . . . 10
- 2.3 Interfaccia fisica: Il Codec Audio SSM2603 . . . 11
  - 2.3.1 Interfacce Analogiche . . . 11
  - 2.3.2 Interfacce Digitali e Protocolli . . . 12
  - 2.3.3 Clocking e Segnali di Sistema . . . 12
- 2.4 Sintesi del Flusso Dati e Meccanismo di Scrambling . . . 13
  - 2.4.1 Il ruolo dell'Audio Controller e l'interfaccia PS-PL . . . 13
  - 2.4.2 Allineamento dei bit e Scrambling . . . 13

3 Modellazione e Simulazione di Sistema (MATLAB) **15**
- 3.1 Configurazione dell'Ambiente di Test . . . 15
- 3.2 Simulazione del Flusso Dati (Cifratura e Ripristino) . . . 15
- 3.3 Analisi di Vulnerabilità: L'Attacco Auto-Ritardo . . . 16
  - 3.3.1 Sperimentazione Preliminare e Analisi del Noise-Floor . . . 16
  - 3.3.2 Ricostruzione Temporale Esatta e Collisioni Spettrali . . . 17
  - 3.3.3 Condizioni di Successo e Applicabilità Reale dell'Attacco . . . 19

4 Progettazione dell'IP Core (Vitis HLS) **20**
- 4.1 HLS vs RTL Manuale: Analisi delle Scelte Progettuali . . . 20
- 4.2 Definizione dell'Interfaccia e dei Tipi di Dato . . . 21
- 4.3 Architettura del Core Crittografico . . . 22
- 4.4 Direttive di Sintesi (Pragmas AXI4-Lite) . . . 22
  - 4.4.1 Considerazioni sulle direttive di ottimizzazione . . . 23
- 4.5 Validazione Algoritmica (C Simulation) . . . 23
  - 4.5.1 Analisi dell'Output di Simulazione . . . 24
  - 4.5.2 Esportazione dei Dataset (File Dumping) . . . 24
- 4.6 Analisi Statistica e Validazione della Casualità . . . 25
  - 4.6.1 Condizionamento dei Dati: Rimozione della Componente Continua . . . 25
  - 4.6.2 Analisi del Polinomio Ottimale . . . 26
  - 4.6.3 Confronto Spettrale: Polinomio Ottimale vs Sub-ottimale . . . 27
- 4.7 Sintesi di Alto Livello (High-Level Synthesis) e Analisi dei Report . . . 28
  - 4.7.1 Stima delle Tempistiche (Timing Estimate) . . . 28
  - 4.7.2 Prestazioni: Latenza e Initiation Interval . . . 28
  - 4.7.3 Mappatura delle Interfacce Software-Hardware (SW-to-HW Mapping) . . . 29
  - 4.7.4 Occupazione delle Risorse (Resource Utilization) . . . 30
- 4.8 Validazione Hardware (C/RTL Co-simulation) . . . 30
  - 4.8.1 Verifica Funzionale . . . 31

4.8.2 Analisi delle Prestazioni Post-Sintesi . . . 31
4.9 Esportazione RTL e Generazione dell'IP Core . . . 32
4.9.1 Configurazione dell'Esportazione . . . 32
4.9.2 Finalizzazione del Flusso HLS . . . 32

5 Integrazione del Sistema Hardware (Vivado) 34
5.1 Configurazione dell'Ambiente e Importazione degli IP . . . 34
5.2 Configurazione del Processing System (Zynq-7000) . . . 34
5.3 Integrazione degli IP e Architettura del Block Design . . . 35
5.3.1 Doppia Istanza Crittografica (TX e RX) . . . 35
5.3.2 Interfacciamento Esterno: Audio Controller e AXI GPIO . . . 35
5.3.3 Interconnessione, Memoria e Interrupts . . . 36
5.4 Simulazione a Livello di Sistema (Zynq VIP) . . . 38
5.4.1 Sviluppo del Testbench e Fasi Operative . . . 38
5.4.2 Analisi delle Transazioni AXI e del Risultato di Simulazione . . . 39
5.4.3 Validazione Matematica dell'Evoluzione di Stato . . . 43
5.5 Vincoli Fisici (XDC) e Generazione del Bitstream . . . 43
5.6 Analisi dei Report di Implementazione (Post-Routing) . . . 44
5.6.1 Analisi Temporale (Timing Summary) . . . 44
5.6.2 Occupazione delle Risorse (Utilization) . . . 45
5.6.3 Stima dei Consumi (Power Estimation) . . . 47

6 Sviluppo Software e Validazione (Vitis) 49
6.1 Configurazione dell'Ambiente e Board Support Package (BSP) . . . 49
6.2 Architettura del Software e Inizializzazione . . . 50
6.2.1 Setup delle Periferiche e del Codec Audio . . . 51
6.2.2 Inizializzazione HLS e Interrupts . . . 51
6.3 Setup Interattivo (Interfaccia Utente via UART) . . . 51
6.4 Gestione del Flusso Audio e Routing Dinamico . . . 52
6.4.1 Sincronizzazione e Ripristino . . . 52
6.4.2 Analisi delle Modalità di Routing . . . 52
6.4.3 Re-Keying Dinamico in Tempo Reale . . . 55
6.5 Risultati Sperimentali . . . 55
6.6 Considerazioni sulle Metodologie di Debugging Hardware . . . 56
6.6.1 XSCT (Xilinx Software Command-Line Tool) . . . 56
6.6.2 ILA (Integrated Logic Analyzer) . . . 56

7 Conclusioni e Sviluppi Futuri 58
7.1 Efficienza Architetturale e Paradigma Co-Design . . . 58
7.1.1 Analisi dei Colli di Bottiglia e Velocizzazione . . . 59
7.2 Limiti di Sicurezza e Scalabilità (Upgrade a 32-bit) . . . 59
7.3 Sviluppi Futuri: Architetture Multi-Board e VoIP . . . 60
7.4 Conclusione . . . 60

A Modellazione e Simulazione di Sistema: Codice MATLAB 61

B Progettazione HLS: Codici Sorgente e Validazione 64
B.1 File: scrambler.h . . . 64
B.2 File: scrambler.c . . . 64
B.3 File: scrambler_test.c . . . 65
B.4 Script MATLAB: Analisi Statistica LFSR . . . 67

C Integrazione Hardware: Vincoli e Configurazioni (Vivado) **70**
C.1 Testbench System-Level (Zynq VIP) 70
C.2 File dei Vincoli Fisici (XDC) 72

D Sviluppo Software: Applicazione Bare-Metal (Vitis) **74**
D.1 Header File: zybo_audio.h 74
D.2 Main Program: zybo_testapp.c 75

Elenco delle figure **85**

CAPITOLO 1

Introduzione e Contesto

1.1 Evoluzione delle comunicazioni sicure

Il concetto di *scrambling* (o mascheramento) audio affonda le sue radici nella necessità storica di garantire la riservatezza nelle comunicazioni strategiche, specialmente in ambito militare e diplomatico. Già durante la Seconda Guerra Mondiale, la protezione della voce era considerata un elemento critico per il successo delle operazioni.

I primi sistemi, come il celebre **SIGSALY** utilizzato dagli Alleati, rappresentarono una pietra miliare: fu il primo sistema di trasmissione vocale criptata digitale. Tuttavia, tali apparati occupavano intere stanze e consumavano enormi quantità di energia.

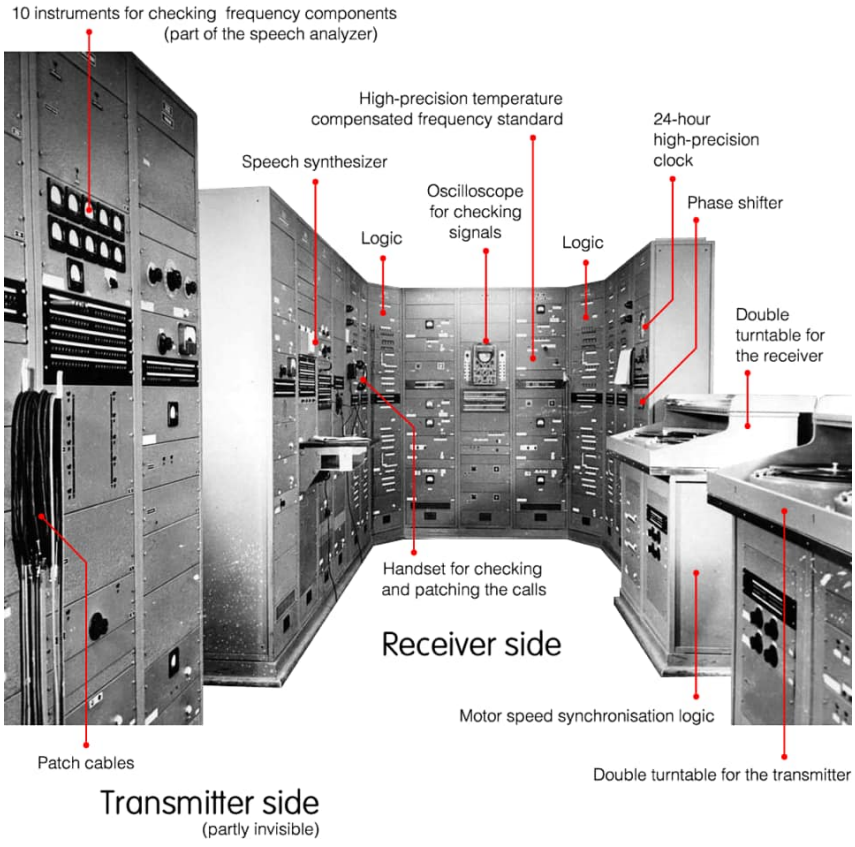


Figura 1.1: I terminali di trasmissione e ricezione del sistema SIGSALY (1943). Sviluppato dai Bell Telephone Laboratories, è stato utilizzato durante la Seconda Guerra Mondiale per le comunicazioni confidenziali al massimo livello statale tra Winston Churchill e Franklin D. Roosevelt. L'enorme ingombro fisico dell'apparato evidenzia il salto tecnologico rispetto all'implementazione su un singolo chip (SoC) moderno (Fonte: Crypto Museum).

Con l'avvento della Guerra Fredda e la successiva era digitale, la ricerca si è spostata verso la miniaturizzazione e l'efficienza algoritmica, portando alla nascita delle radio tattiche portatili.

1.2 Dall'analogico al digitale: il ruolo del Ciphering

Mentre i primi scrambler operavano nel dominio analogico tramite l'inversione di banda (Frequency Inversion), la vera rivoluzione è avvenuta con la digitalizzazione del segnale. In questo contesto, l'uso di registri a scorrimento a retroazione lineare (**LFSR - Linear Feedback Shift Register**) è diventato lo standard per la generazione di sequenze pseudo-casuali (*Stream Ciphers*).

Queste tecnologie sono fondamentali nelle radio militari per due ragioni principali:

- **COMSEC (Communications Security):** Impedire a un utente non autorizzato di comprendere il contenuto del messaggio.
- **LPI/LPD (Low Probability of Intercept/Detection):** Far apparire il segnale trasmesso simile al rumore bianco, rendendo difficile anche solo rilevare la presenza di una trasmissione radio.

1.2.1 Effetti Spettrali dello Scrambling e Inviolabilità del Segnale

Per comprendere la robustezza di questo approccio, è necessario analizzare l'effetto dell'operatore logico XOR nel dominio delle frequenze.

Il segnale vocale umano possiede una firma spettrale ben definita: la stragrande maggioranza dell'energia acustica è concentrata in una banda stretta, tipicamente compresa tra i 300 Hz e i 3400 Hz, caratterizzata da picchi evidenti in corrispondenza delle formanti vocali. Al contrario, un registro LFSR ben progettato genera una sequenza pseudo-casuale che, dal punto di vista dell'analisi dei segnali, approssima un **Rumore Bianco**. Il rumore bianco possiede una densità spettrale di potenza piatta: la sua energia è distribuita in modo uniforme su tutte le frequenze disponibili fino alla frequenza di Nyquist (nel nostro caso, 24 kHz).

Quando il microcontrollore esegue lo scrambling, combinando la voce in chiaro con il flusso LFSR tramite operatore XOR, il segnale vocale viene "spalmato" e annegato all'interno del rumore. Il segnale risultante intercettato da un potenziale attaccante erediterà le proprietà statistiche della chiave crittografica, presentando anch'esso uno spettro piatto e caotico. I picchi delle formanti vocali risultano completamente livellati e indistinguibili dal *noise floor*.

Perché i filtri audio tradizionali falliscono?

In presenza di disturbi acustici convenzionali (come un ronzio di fondo o un fischio a frequenza fissa), l'Ingegneria dei Segnali (DSP) permette di recuperare il messaggio originale applicando dei filtri lineari (Passa-Basso, Passa-Alto o Elimina-Banda) per sopprimere le frequenze indesiderate.

Contro uno *stream cipher* digitale, tuttavia, **qualsiasi tecnica di filtraggio risulta matematicamente inefficace**. Il motivo risiede nel principio di *sovrapposizione spettrale*: poiché il rumore bianco generato dall'LFSR copre l'intero spettro, esso occupa esattamente la stessa banda di frequenze del segnale vocale (300 - 3400 Hz). Se un attaccante tentasse di applicare un filtro Passa-Basso per eliminare il rumore ad alta frequenza, la porzione di rumore LFSR residente in banda vocale rimarrebbe intatta, mantenendo il segnale incomprensibile. Se, viceversa, tentasse di filtrare la banda vocale, eliminerebbe irrimediabilmente anche la voce stessa.

Dal momento che il rumore e il segnale sono sovrapposti sia nel dominio del tempo che in quello delle frequenze, l'intercettatore non ha alcuno strumento analitico per separarli. L'unica soluzione per estrarre l'informazione è possedere l'esatta replica del rumore originale per eseguire la decodifica (XOR inverso). E poiché l'LFSR genera una sequenza deterministica ma crittograficamente sicura, l'unico modo per ottenere tale replica è conoscere l'esatto polinomio generatore e il **Seed** iniziale segreto condiviso tra i due nodi di comunicazione.

Quest'ultimo parametro costituisce a tutti gli effetti la chiave simmetrica del sistema e necessita di essere scambiato preventivamente attraverso un canale sicuro per preservare la confiden-

zialità ed evitare attacchi di intercettazione o manipolazione. Negli apparati di telecomunicazione moderni, tale distribuzione avviene tipicamente mediante protocolli di crittografia asimmetrica a chiave pubblica, come il protocollo Diffie-Hellman o varianti basate su curve ellittiche (ECDH), che consentono l'accordo su un segreto condiviso operando anche su un mezzo intrinsecamente insicuro. In contesti strettamente tattico-militari o avionici, si preferisce invece ricorrere alla distribuzione fisica delle chiavi (*Pre-Shared Keys*): i *seed* vengono iniettati direttamente nella memoria non volatile del dispositivo prima della missione tramite dispositivi hardware dedicati (noti storicamente come *Data Transfer Device* o moduli *Key Fill*), immunizzando la fase di setup da qualsiasi tentativo di intercettazione via etere.

1.3 Obiettivo del progetto: l'approccio SoC

L'implementazione del sistema su un'architettura **System-on-Chip (SoC)**, come la piattaforma Xilinx Zybo Z7, si basa sull'applicazione del paradigma di *Hardware/Software Co-Design*. A differenza di un approccio puramente software, l'uso di un SoC permette di ottimizzare l'efficienza del sistema ripartendo il carico di lavoro tra i diversi domini dell'architettura.

In questo progetto, il processore (PS - Processing System) assume il ruolo di supervisore: gestisce l'inizializzazione delle periferiche, il routing del segnale e l'interfaccia utente. L'onere computazionale dell'algoritmo, ovvero la generazione del rumore pseudo-casuale ad alta frequenza e l'operazione di XOR bit-a-bit, viene invece demandato a un acceleratore dedicato all'interno della logica programmabile (PL - Programmable Logic).

Questa ripartizione strutturale è tipica dei moderni apparati di telecomunicazione, in quanto lo spostamento delle operazioni critiche in hardware è necessario per soddisfare i rigidi vincoli di latenza deterministica ed elevata velocità di elaborazione richiesti dalla crittografia audio in tempo reale.

1.4 Struttura del lavoro

Nel presente elaborato verrà illustrato l'intero flusso di progettazione seguendo un approccio top-down (come schematizzato in Figura 1.2). In primo luogo, verrà analizzata la modellazione matematica e la simulazione del sistema in ambiente **MATLAB** (Model-Based Design). Successivamente, si esploreranno le fasi di sintesi dell'IP Core crittografico tramite **Vitis HLS**, l'integrazione architetturale dell'hardware in **Vivado** tramite bus AXI4-Lite e, infine, lo sviluppo del software *bare-metal* in **Vitis** per la gestione dinamica dei parametri di cifratura e il controllo del flusso audio tramite il codec di bordo.

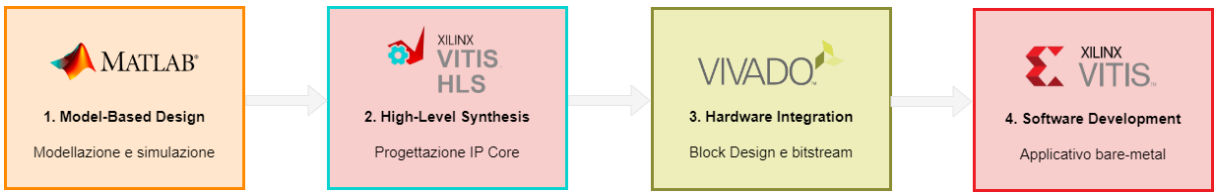


Figura 1.2: Rappresentazione schematica del flusso di progettazione *Hardware/Software Co-Design*: dalla modellazione ad alto livello fino all'implementazione *bare-metal*.

CAPITOLO 2

Architettura del Sistema e Teoria Operativa

2.1 Visione d'insieme del flusso dati

L'obiettivo del progetto è realizzare un sistema di elaborazione audio in tempo reale capace di mascherare e ripristinare un segnale vocale. Per ottenere latenze deterministiche, il sistema sfrutta un'architettura ibrida *Hardware/Software*.

Il flusso del segnale attraversa tre macro-blocchi principali:

1. **Acquisizione:** Il segnale analogico stereo proveniente da una sorgente esterna (ingresso Line In) viene digitalizzato (ADC) dal codec audio integrato sulla scheda.
2. **Elaborazione (SoC):** I campioni digitali arrivano al processore ARM (PS), il quale li instrada via bus verso un acceleratore hardware dedicato, situato nella logica programmabile (PL). Questo blocco esegue l'algoritmo di scrambling crittografico ad altissima velocità.
3. **Riproduzione:** Il campione elaborato (criptato o decriptato) viene restituito al processore e inviato nuovamente al codec, che lo converte in segnale analogico (DAC) per l'ascolto in cuffia.

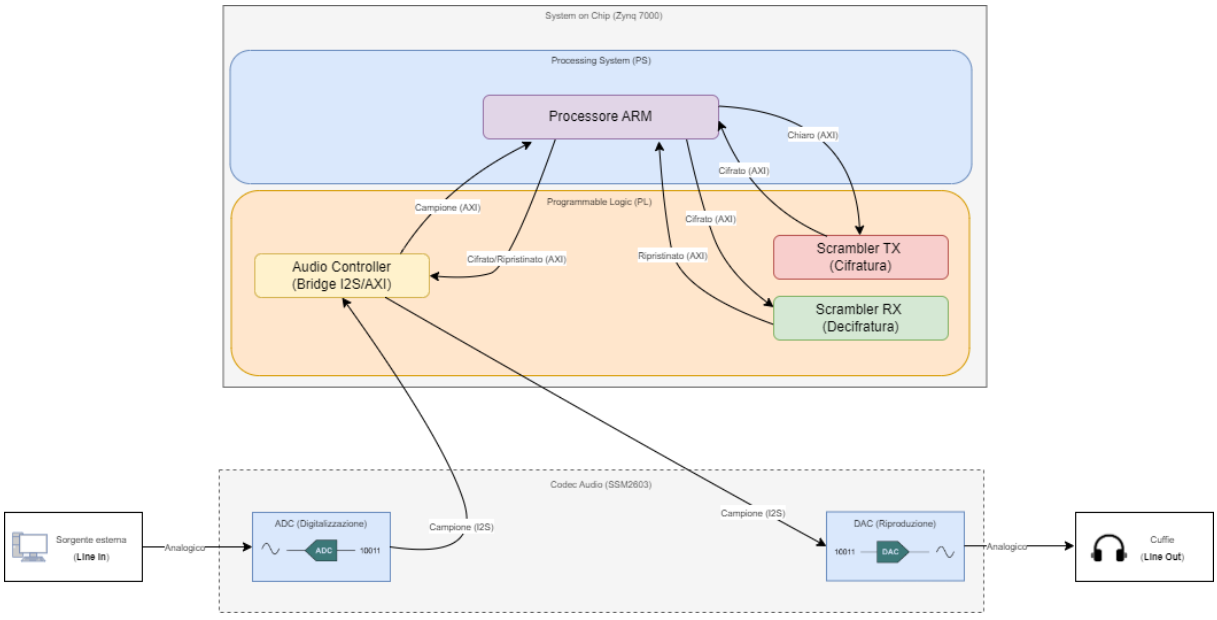


Figura 2.1: Diagramma a blocchi concettuale ad alto livello del sistema. Si evidenzia il partizionamento Hardware/Software: il Processing System (ARM) funge da nodo di instradamento, mentre la Programmable Logic ospita gli acceleratori crittografici e l'Audio Controller per interfacciarsi con il codec esterno.

2.2 Il principio dello Scrambling e il generatore LFSR

Il nucleo matematico dell'intero sistema di mascheramento si basa su una delle operazioni più note in crittografia: l'OR Esclusivo (XOR) bit a bit. Indicando con M il messaggio originale (la voce) e con K la chiave (il rumore), il segnale criptato C è dato da:

$$C = M \oplus K$$

Poiché l'operatore XOR è un'operazione involutoria (simmetrica), applicando nuovamente la stessa chiave al segnale criptato, si ottiene il messaggio originale in chiaro:

$$M = C \oplus K$$

Affinché il sistema sia sicuro ed efficace acusticamente, la chiave K non può essere statica, ma deve consistere in un flusso continuo di numeri apparentemente casuali (Rumore Bianco) generato alla stessa frequenza di campionamento dell'audio (48 kHz).

2.2.1 Linear Feedback Shift Register (LFSR) di Galois

Per generare questo flusso pseudo-casuale in hardware, lo standard ingegneristico prevede l'utilizzo di un **LFSR (Linear Feedback Shift Register)**. Come formalizzato nella letteratura informatica, un LFSR è un registro a scorrimento in cui il bit di ingresso è una funzione lineare del suo stato precedente.

Nel presente progetto è stata utilizzata una specifica architettura nota come **LFSR in configurazione di Galois**. A differenza dello schema classico (configurazione di Fibonacci), in cui i bit prelevati vengono concatenati e sommati esternamente per generare il nuovo bit in ingresso, l'architettura di Galois distribuisce l'operazione logica all'interno del registro stesso.

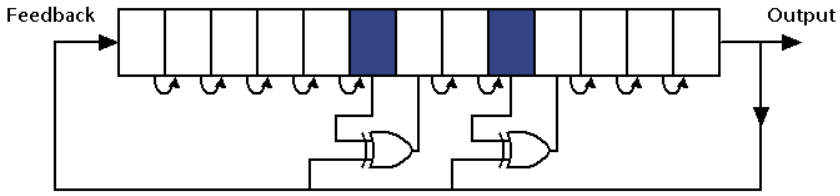


Figura 2.2: Schema a blocchi di un LFSR in configurazione di Galois. Il bit uscente (Output) funge da linea di retroazione (Feedback): viene inserito come nuovo bit iniziale e contemporaneamente sommato ai bit traslati in corrispondenza dei tap. (Fonte: Wikipedia)

In termini pratici, l'LFSR è composto da una catena di flip-flop. Ad ogni colpo di clock, i bit scorrono di una posizione verso destra. Il bit che viene espulso dal registro (il Least Significant Bit, o LSB) funge da uscita ma anche da linea di retroazione (*feedback*): esso viene reinserito all'inizio della catena diventando il nuovo bit più significativo (MSB) e, contemporaneamente, viene inviato agli stadi intermedi del registro.

Le posizioni intermedie in cui questa linea di retroazione va ad agire prendono il nome di **Tap**, la cui disposizione esatta è definita da una specifica equazione matematica chiamata *polinomio generatore*. Come illustrato nello schema di Galois (Figura 2.2):

- I bit che non sono tap vengono traslati al flip-flop successivo inalterati;
- Dei tap, invece, si fa lo XOR tra il bit uscente dallo stadio precedente e la linea di retroazione, e il risultato diventa il nuovo ingresso per il flip-flop successivo.

Questa configurazione è particolarmente vantaggiosa nella progettazione di circuiti digitali (come le FPGA). Poiché gli LFSR di Galois non richiedono una cascata di porte logiche per

combinare i tap e produrre il nuovo ingresso (come avviene invece nella topologia di Fibonacci), le operazioni di XOR avvengono tutte simultaneamente sui rispettivi stadi. Essendo le porte logiche distribuite in modo indipendente e operando in parallelo, il ritardo critico (*Critical Path*) del circuito si riduce al tempo di propagazione di una singola porta XOR, indipendentemente dal numero di tap. Ciò permette all'architettura di operare a frequenze di clock significativamente più elevate.

La disposizione esatta dei tap è il fattore determinante per la lunghezza della sequenza generata. Per comprendere questa dipendenza, occorre trasporre il circuito nel dominio dell'algebra astratta. Lo stato dell'LFSR a n bit può essere rappresentato come un polinomio $S(x)$ di grado massimo $n-1$, i cui coefficienti (s_i) appartengono al campo finito di Galois $GF(2)$, un sistema algebrico binario in cui l'operazione di addizione coincide esattamente con l'operatore logico XOR ($\oplus$):

$$S(x) = s_{n-1}x^{n-1} + s_{n-2}x^{n-2} + \cdots + s_1x + s_0$$

L'architettura hardware è governata da un *polinomio generatore* $P(x)$ di grado n, che definisce le posizioni della retroazione:

$$P(x) = x^n + c_{n-1}x^{n-1} + \cdots + c_1x + 1$$

dove i coefficienti $c_i \in \{0, 1\}$ indicano la presenza o l'assenza della porta XOR nello stadio i-esimo.

Ad ogni colpo di clock, l'avanzamento della macchina a stati equivale a calcolare il nuovo stato polinomiale $S_{next}(x)$ moltiplicando lo stato corrente per x (che corrisponde all'operazione meccanica di *shift*) e calcolandone il resto della divisione rispetto al polinomio generatore:

$$S_{next}(x) = (S(x) \cdot x) \pmod{P(x)}$$

Il significato fisico di questa operazione modulo si traduce direttamente nel comportamento del circuito. Moltiplicando $S(x)$ per x, i bit scorrono di una posizione verso destra. Nella notazione binaria standard, una moltiplicazione per x equivarrebbe a uno shift logico verso sinistra. Nell'architettura qui implementata, invece, i bit scorrono verso **destra**. Affinché lo shift a destra sia matematicamente equivalente alla moltiplicazione $S(x) \cdot x$, l'ordine dei coefficienti nel registro fisico è invertito (*bit-reversed*): il bit più significativo (MSB) memorizza il termine noto x^0, mentre il bit meno significativo (LSB) memorizza il termine di grado massimo x^{n-1}. Se il bit uscente dal registro è 0, non vi è alcun "overflow" oltre il grado $n-1$ e il nuovo stato è semplicemente il dato traslato. Se invece il bit uscente vale 1, la moltiplicazione genera un termine di grado x^n. Poiché nel campo vettoriale operiamo modulo $P(x)$, si pone per definizione $P(x) \equiv 0$. Questo implica che l'overflow x^n possa essere matematicamente sostituito dai restanti termini del polinomio:

$$x^n \equiv c_{n-1}x^{n-1} + \cdots + c_1x + 1$$

Sostituire questo termine in $GF(2)$ equivale a sommare (ovvero applicare lo XOR) i coefficienti non nulli c_i ai bit appena traslati. Questa operazione algebrica descrive esattamente il comportamento del *feedback* nell'LFSR di Galois, dove la maschera dei tap funge da divisore polinomiale.

Per garantire che l'LFSR esplori tutti i possibili stati non nulli prima di ripetersi (ottenendo una sequenza detta di *lunghezza massima*), è strettamente necessario che i tap configurino un polinomio matematicamente **primitivo**. Un polinomio si dice primitivo se è irriducibile (non fattorizzabile) e se è in grado di generare, tramite le sue potenze successive, tutti i $2^n - 1$ elementi non nulli del campo. Se i tap vengono disposti in posizioni arbitrarie (configurando un polinomio non primitivo), lo spazio totale degli stati si frammenta in sotto-cicli chiusi di lunghezza nettamente inferiore. In tal caso, il registro rimarrà intrappolato in un loop breve (come dimostrato empiricamente nell'analisi spettrale del Capitolo 3), degradando in modo irrimediabile l'entropia del rumore crittografico.

Nella trasposizione pratica dal dominio matematico a quello hardware (e nel relativo codice sorgente in C), il polinomio generatore viene codificato sotto forma di una **maschera binaria**. Ogni bit di questa maschera rappresenta uno stadio del registro:

- se il bit vale `1` (posizione di tap), in quella posizione viene istanziata fisicamente la porta XOR per implementare il coefficiente polinomiale;
- se il bit vale `0` (non-tap), il dato viene traslato al flip-flop successivo senza alterazioni (coefficiente nullo).

Nel presente progetto, l'LFSR a 16 bit impiega il valore esadecimale `0xB400` (in binario `1011 0100 0000 0000`): gli "1" logici indicano alla logica programmabile i punti precisi in cui posizionare le porte logiche per soddisfare il polinomio primitivo scelto.

Infine, un LFSR necessita di un valore iniziale non nullo per iniziare a operare, denominato **Seed**. Finché il trasmettitore e il ricevitore condividono lo stesso polinomio e vengono inizializzati con lo stesso Seed, produrranno in modo deterministico l'identica sequenza pseudo-casuale, permettendo la perfetta decodifica del segnale audio tramite la proprietà involutoria dello XOR.

2.2.2 Periodicità e limiti dell'implementazione a 16-bit

Una proprietà fondamentale degli LFSR è che, avendo un numero finito di stati, la sequenza generata è destinata a ripetersi. Un LFSR a n bit con un polinomio opportunamente scelto (polinomio di lunghezza massima) attraverserà tutti i possibili $2^n - 1$ stati non nulli prima di ricominciare il ciclo.

Nel presente proof-of-concept, si è scelto di implementare un LFSR a 16 bit. Dal punto di vista matematico, la lunghezza del periodo prima che il rumore si ripeta è di 65.535 campioni. Lavorando a una frequenza audio di 48.000 campioni al secondo, il periodo temporale T della crittografia risulta essere:

$$T = \frac{2^{16} - 1}{48000} \approx 1,36 \text{ secondi}$$

Questa specifica tecnica comporta un duplice effetto. Dal punto di vista acustico in tempo reale, 1,36 secondi di rumore caotico sono più che sufficienti a mascherare completamente le sillabe e i fonemi della voce umana, rendendola incomprensibile all'ascolto.

Tuttavia, dal punto di vista della sicurezza crittografica, un ciclo così breve espone il sistema all'attacco informatico del *Two-Time Pad* (o attacco dell'auto-ritardo) nel caso in cui le trasmissioni vengano registrate.

Se un attaccante intercetta il segnale criptato $C(t)$, sapendo che il rumore $K(t)$ ha un periodo esatto $T = 1,36$ secondi, può sferrare il cosiddetto **Attacco dell'Auto-Ritardo** (o vulnerabilità del *Two-Time Pad*). Operando in modo causale nel dominio del tempo, l'attaccante memorizza in un buffer la traccia intercettata ed esegue uno XOR tra il campione presente al tempo t e il campione passato ritardato esattamente di T. Matematicamente:

$$C(t) \oplus C(t - T) = [M(t) \oplus K(t)] \oplus [M(t - T) \oplus K(t - T)]$$

Per le proprietà di periodicità della chiave, l'operatore XOR annulla gli elementi identici ($K(t) \oplus K(t - T) = 0$). Il risultato netto è:

$$C(t) \oplus C(t - T) = M(t) \oplus M(t - T)$$

Il rumore è stato matematicamente distrutto senza conoscere né il Seed né il polinomio, lasciando solo la voce al tempo t sovrapposta alla voce al tempo $t - T$. Poiché il parlato naturale può contenere frequenti pause e silenzi, nei momenti in cui il segnale passato $M(t - T) = 0$, il frammento presente $M(t)$ emerge in modo chiaro e intellegibile (e viceversa).

Per scalare il sistema a un livello di sicurezza tattico reale, sarebbe sufficiente estendere il registro dell'algoritmo hardware a 32 bit. A parità di frequenza di campionamento audio ($f_s = 48.000$ Hz), il nuovo tempo di ripetizione T_{32} diverrebbe:

$$T_{32} = \frac{2^{32} - 1}{f_s} = \frac{4.294.967.295}{48000} \approx 89.478,48 \text{ secondi}$$

Questo estende la durata del periodo a circa **24,85 ore** continue, un tempo enormemente superiore alla durata di qualsiasi conversazione radio tattica. Non verificandosi mai una ripetizione del pattern all'interno della stessa trasmissione, l'attacco dell'auto-ritardo risulta matematicamente inapplicabile.

2.3 Interfaccia fisica: Il Codec Audio SSM2603

Il cuore della sezione audio della scheda Zybo Z7 è il codec **SSM2603** di Analog Devices. Si tratta di un componente a basso consumo progettato per applicazioni audio digitali, capace di operare con frequenze di campionamento comprese tra 8 kHz e 96 kHz. Nel presente progetto, il codec è stato configurato per lavorare a una frequenza fissa di **48 kHz**.

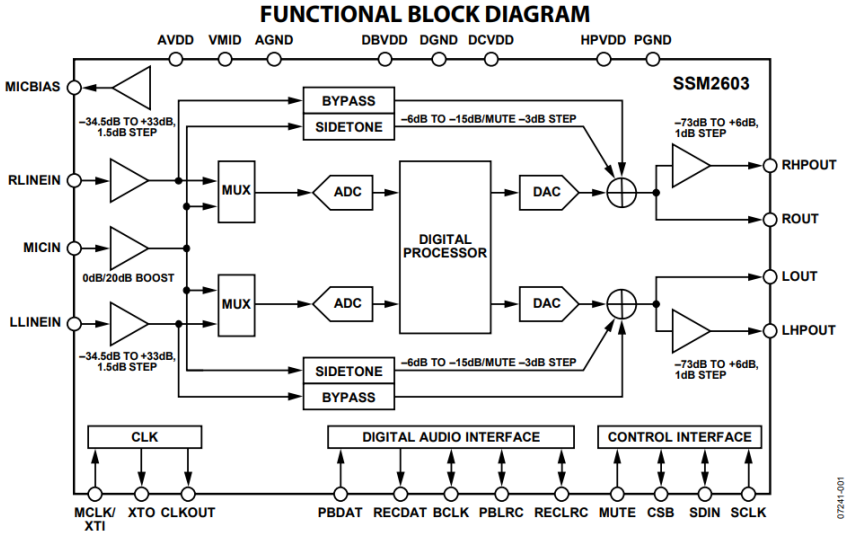


Figura 2.3: Diagramma a blocchi funzionale del codec SSM2603, che illustra il percorso del segnale dai convertitori ADC al processore digitale e ai DAC di uscita. (Fonte: Analog Devices SSM2603 Datasheet)

2.3.1 Interfacce Analogiche

La Zybo Z7 espone le capacità del codec attraverso tre jack audio standard da 3.5 mm, distinguibili dal codice colore e dalla funzione:

- **Line In (Azzurro - J7):** Ingresso stereo per segnali di linea.
- **Mic In (Rosa - J6):** Ingresso mono per microfono.
- **Headphone Out (Nero - J5):** Uscita stereo amplificata per cuffie.

Come è possibile osservare nella mappa dei componenti della scheda, questi connettori sono raggruppati lungo il margine destro del PCB.

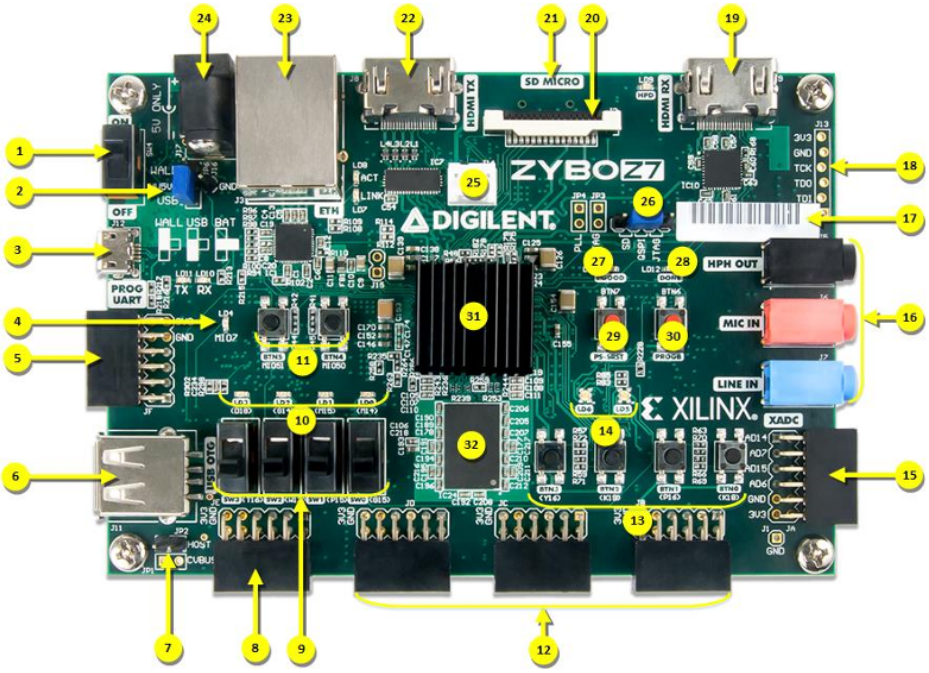


Figura 2.4: Panoramica della scheda Zybo Z7. Le porte analogiche collegate al codec audio (Line In, Mic In, Headphone Out) sono evidenziate dal callout numero 16. (Fonte: Digilent Zybo Z7 Reference Manual)

I segnali analogici in ingresso vengono convertiti in digitale dai convertitori Analogico-Digitali (ADC) a 24 bit interni all'SSM2603. Simmetricamente, i segnali digitali processati dal SoC vengono riportati nel dominio analogico da convertitori Digitale-Analogici (DAC) gemelli per la riproduzione in cuffia.

2.3.2 Interfacce Digitali e Protocolli

La comunicazione tra il SoC Zynq e il codec avviene attraverso due canali distinti:

1. **Interfaccia Dati (I2S):** Viene utilizzata per il trasferimento dei campioni audio in tempo reale. Il protocollo I2S (*Inter-IC Sound*) impiega i segnali `BCLK` (Bit Clock) per la sincronizzazione dei bit, `PBLRC/RECLRC` per distinguere i canali sinistro/destro e i pin `PBDAT/RECDAT` per i dati seriali. I campioni vengono trasmessi bit a bit, con il bit più significativo (MSB) per primo.

2. **Interfaccia di Controllo (I2C):** Poiché il codec è altamente personalizzabile, il processore deve poterne configurare i registri interni (es. volume, boost del microfono, attivazione dei path audio). Questo avviene tramite il bus I2C utilizzando i segnali `SCLK` e `SDIN` all'indirizzo `0x1A`.

2.3.3 Clocking e Segnali di Sistema

Per garantire il corretto funzionamento, il codec richiede un Master Clock (`MCLK`) stabile, fornito dal Processing System (PS) dello Zynq tramite il segnale `FCLK_CLK1`. Per una frequenza di campionamento di 48 kHz, l'MCLK è impostato esattamente a **12.288 MHz**.

Infine, il segnale **MUTE** (attestato sul pin P18 della FPGA) è un segnale *active-low* con resistenza di pull-down: deve essere pilotato a livello logico alto dal software per abilitare le uscite analogiche e permettere l'ascolto del segnale.

2.4 Sintesi del Flusso Dati e Meccanismo di Scrambling

Per comprendere l'operatività del sistema, è necessario analizzare il "ciclo di vita" di un singolo campione audio attraverso l'architettura SoC. Sebbene il codec SSM2603 e l'algoritmo LFSR siano i componenti principali, la comunicazione tra di essi è resa possibile da un'entità intermedia fondamentale: l'**Audio Controller IP**.

2.4.1 Il ruolo dell'Audio Controller e l'interfaccia PS-PL

L'Audio Controller è un blocco di logica programmabile (instanziato nella PL) che funge da *bridge* tra il mondo seriale del codec e il mondo a pacchetti del processore ARM.

- **Lato Codec (I2S):** Riceve i bit seriali dal pin `RECDAT` e li raggruppa in parole digitali.
- **Lato Processore (AXI):** Espone questi dati all'interno di registri mappati in memoria (*Memory-Mapped I/O*), permettendo al software di leggere il campione tramite il bus AXI4-Lite.

La scelta di adottare l'interfaccia **AXI4-Lite** per la comunicazione tra il processore (PS) e le periferiche audio è dettata da un preciso bilanciamento tra prestazioni e risorse. Operando su singoli campioni audio a una frequenza di 48 kHz, il *throughput* richiesto è estremamente contenuto (circa un campione ogni 20,8 μs). L'utilizzo di interfacce più complesse come **AXI4-Stream** (ottimizzata per trasferimenti massivi via DMA) o **AXI4-Full** (progettata per alte prestazioni e *burst*) avrebbe comportato un inutile spreco di risorse logiche sulla FPGA senza apportare benefici tangibili alla latenza del sistema.

2.4.2 Allineamento dei bit e Scrambling

Un dettaglio tecnico critico riguarda la precisione del dato. Il codec SSM2603 opera con una risoluzione di **24 bit**, ma l'algoritmo di scrambling implementato in HLS è ottimizzato per campioni a **16 bit**.

Il flusso operativo sul singolo campione avviene secondo i seguenti passaggi:

1. **Acquisizione:** Il processore legge il registro dell'Audio Controller ottenendo una parola a 32 bit contenente il campione a 24 bit.
2. **Adattamento (Bit-shift):** Il software esegue uno *shift* a destra di 8 posizioni (`» 8`) per estrarre i 16 bit più significativi (MSB), adattando il volume e il formato alla firma dell'IP Scrambler.
3. **Elaborazione Hardware:** Il valore a 16 bit viene inviato all'IP Scrambler. Qui, in un unico ciclo di clock, viene eseguito lo XOR con lo stato corrente dell'LFSR di Galois:

$$Sample_{out} = Sample_{in} \oplus LFSR_{state}$$

4. **Riallineamento:** Il risultato viene riletto dal processore e riportato a 24 bit tramite uno *shift* a sinistra (`« 8`) prima di essere inviato nuovamente all'Audio Controller per la riproduzione.

È doveroso sottolineare che l'operazione di troncamento (`» 8`) comporta una perdita irreversibile degli 8 bit meno significativi (LSB) del campione audio originale. Dal punto di vista della teoria dei segnali, ciò si traduce in una riduzione della gamma dinamica teorica da 144 dB (tipica dei 24 bit) a 96 dB (standard audio qualità CD a 16 bit).

Tuttavia, nell'ambito di un sistema di comunicazione vocale, questa perdita di informazione risulta percettivamente nulla. I livelli di rumore di fondo dei microfoni standard e dell'ambiente

circostante superano abbondantemente la risoluzione offerta da quegli ultimi 8 bit. In ottica di *Hardware/Software Co-Design*, questa scelta rappresenta un compromesso ingegneristico ottimale: sacrificando una porzione di dati analogicamente inudibile, si riduce significativamente l'ampiezza dei registri e la complessità combinatoria. Questo si traduce in un notevole risparmio di risorse logiche (LUT e Flip-Flop) all'interno della FPGA, massimizzando l'efficienza e abbattendo i tempi di esecuzione dell'IP crittografico.

CAPITOLO 3

Modellazione e Simulazione di Sistema (MATLAB)

Prima di procedere all'implementazione fisica su logica programmabile (FPGA) tramite i tool di sintesi che verranno presentati nei capitoli successivi, l'intero algoritmo crittografico è stato modellato e validato in ambiente MATLAB. Questo approccio, noto nell'ingegneria dei sistemi come *Model-Based Design* (MBD), consente di verificare la correttezza matematica del sistema al netto delle latenze fisiche dei circuiti. Oltre a confermare la perfetta reversibilità del cifrario in condizioni ideali, la simulazione ad alto livello fornisce i requisiti di progetto e l'ambiente idoneo per eseguire un collaudo di robustezza (*Penetration Testing*) volto a dimostrare analiticamente la vulnerabilità intrinseca dei registri a 16 bit prima della loro definitiva trasposizione in hardware.

3.1 Configurazione dell'Ambiente di Test

Per la simulazione è stato registrato un campione audio vocale reale $M(t)$, opportunamente strutturato per ricalcare le metriche di scansione temporale tipiche del parlato interrotto da pause.

▶ *Audio 0: Canale in chiaro (Sorgente)* - Segnale vocale nativo prima del processo di cifratura (*messaggio.wav*).

La registrazione (acquisita mono alla frequenza di campionamento $f_s = 48$ kHz) presenta il seguente sviluppo temporale nativo:

1. **Da 0.5s a 1.0s:** Pronuncia della parola *"Prova"*.
2. **Da 1.0s a 3.0s:** Intervallo di *silenzio* ambientale inter-sillabico (durata pari a 2.0 secondi).
3. **Da 3.0s in poi:** Pronuncia della sequenza numerica *"Uno, due, tre"*.

Il segnale originariamente espresso in virgola mobile (floating-point nell'intervallo $[-1.0, +1.0]$) è stato convertito nel dominio discreto a numeri interi con segno a 16-bit (`int16`, con dinamica $[-32768, +32767]$) al fine di emulare la dimensione dei registri e dei bus dati che verranno successivamente implementati all'interno della logica programmabile e descritti nel Capitolo 4.

3.2 Simulazione del Flusso Dati (Cifratura e Ripristino)

Lo script di modellazione (riportato integralmente in Appendice A) implementa la topologia matematica del generatore: un registro LFSR in configurazione di Galois governato dalla maschera polinomiale esadecimale `0xB400`, che costituirà la specifica per lo sviluppo del modulo in Vitis HLS. Ad ogni ciclo di campionamento, lo stato interno trasla generando una sequenza pseudo-casuale ad alta entropia, utilizzata come chiave di flusso $K(t)$ per l'operazione di OR Esclusivo bit-a-bit con il segnale in chiaro:

$$C(t) = M(t) \oplus K(t)$$

I risultati della simulazione del canale legittimo hanno confermato la validità del modello. Il segnale trasmesso e cifrato $C(t)$ si presenta spettralmente piatto, sovrapponibile a un rumore bianco totalmente incomprensibile all'ascolto.

Audio 1: Output di Cifratura (Tx) - File interamente mascherato da rumore bianco pseudo-casuale.

Simmetricamente, la riesecuzione dello XOR nel modello del ricevitore, inizializzato con il medesimo *Seed* (`0x1234`), ha computato un tasso di errore pari a zero campioni, garantendo la perfetta natura involutoria e speculare della crittografia a flusso prima di affrontarne l'ottimizzazione in hardware.

Audio 2: Output di Decifratura (Rx) - Ricostruzione bit-perfect operata dal modello del nodo legittimo.

3.3 Analisi di Vulnerabilità: L'Attacco Auto-Ritardo

Come analizzato nella teoria del *Two-Time Pad* (Sezione 2.2.2), un LFSR a 16 bit possiede uno spazio degli stati limitato a $2^{16} - 1 = 65.535$ combinazioni uniche prima del riavvolgimento ciclico della sequenza. Alla frequenza nativa del codec (48 kHz), il periodo temporale esatto di ripetizione della chiave coincide con:

$$T = \frac{65.535}{48.000} = 1,3653125 \text{ secondi}$$

Un attaccante non autorizzato che intercetti il flusso cifrato $C(t)$, pur ignorando la chiave d'accesso, può sfruttare questa periodicità memorizzando il flusso ed eseguendo una correlazione temporale causale. L'attacco si concretizza applicando un ritardo pari a T e calcolando lo XOR tra il campione presente e quello memorizzato un periodo prima:

$$\text{Recupero}(t) = C(t) \oplus C(t-T) = [M(t) \oplus K(t)] \oplus [M(t-T) \oplus K(t-T)]$$

Poiché la debolezza architetturale impone che $K(t) = K(t-T)$, la portante di rumore si elide matematicamente, lasciando l'intercettatore con il solo combinato delle tracce in chiaro:

$$\text{Recupero}(t) = M(t) \oplus M(t-T)$$

3.3.1 Sperimentazione Preliminare e Analisi del Noise-Floor

Eseguendo una prima prova di attacco direttamente sul file audio grezzo (*raw*), si è riscontrato un forte degrado acustico nel segnale recuperato, caratterizzato da una distorsione metallica ad energia elevata che inficiava l'intellegibilità dei fonemi.

Audio 3: Output dell'Attacco Auto-Ritardo - Traccia estratta dall'intercettatore affetta da noise-floor ambientale.

Dal punto di vista della teoria dei segnali, tale fenomeno è giustificato dal fatto che il silenzio registrato dal microfono non equivale mai a uno zero logico assoluto (`0x0000`), bensì a un *noise-floor* ambientale fluttuante su valori minimi (es. $-1, -3, +2$). In rappresentazione binaria a complemento a due, un valore anche minimamente negativo (come -1) satura l'intera parola a livello logico alto (`1111 1111 1111 1111`). Lo XOR tra la voce utile e una stringa di bit interamente asseriti ad 1 comporta l'inversione radicalmente distruttiva del segno e dell'ampiezza del campione vocale, destrutturando la geometria dell'onda sonora.

Per isolare matematicamente l'attacco ed emulare condizioni di grounding ideali, nello script è stato implementato un **Noise Gate Digitale**:

```
1 voce_int16(abs(voce_int16) < 300) = 0;
```

Listing 3.1: Filtro a soglia per l'azzeramento del noise-floor ambientale.

Questo filtro forza a zero logico assoluto ogni oscillazione inferiore a un'ampiezza arbitraria di 300LSB, ricreando una condizione di silenzio crittografico perfetto e consentendo di esaminare analiticamente la sovrapposizione dei campioni.

3.3.2 Ricostruzione Temporale Esatta e Collisioni Spettrali

Applicando il *Noise Gate*, l'attacco ha manifestato la compromissione del cifrario a 16 bit. L'ascolto della traccia estratta rivela la sequenza acustica risultante: *"Prova... uno, due, uno, du..."*.

Audio 4: Output dell'Attacco Auto-Ritardo - Traccia estratta dall'intercettatore in cui si apprezza la ricomparsa di *"Prova"* e l'interruzione su *"uno, du..."*.

Questo andamento è rigorosamente dimostrabile calcolando gli spostamenti temporali operati dall'equazione causale $M(t) \oplus M(t - 1,365s)$ lungo la timeline del file audio:

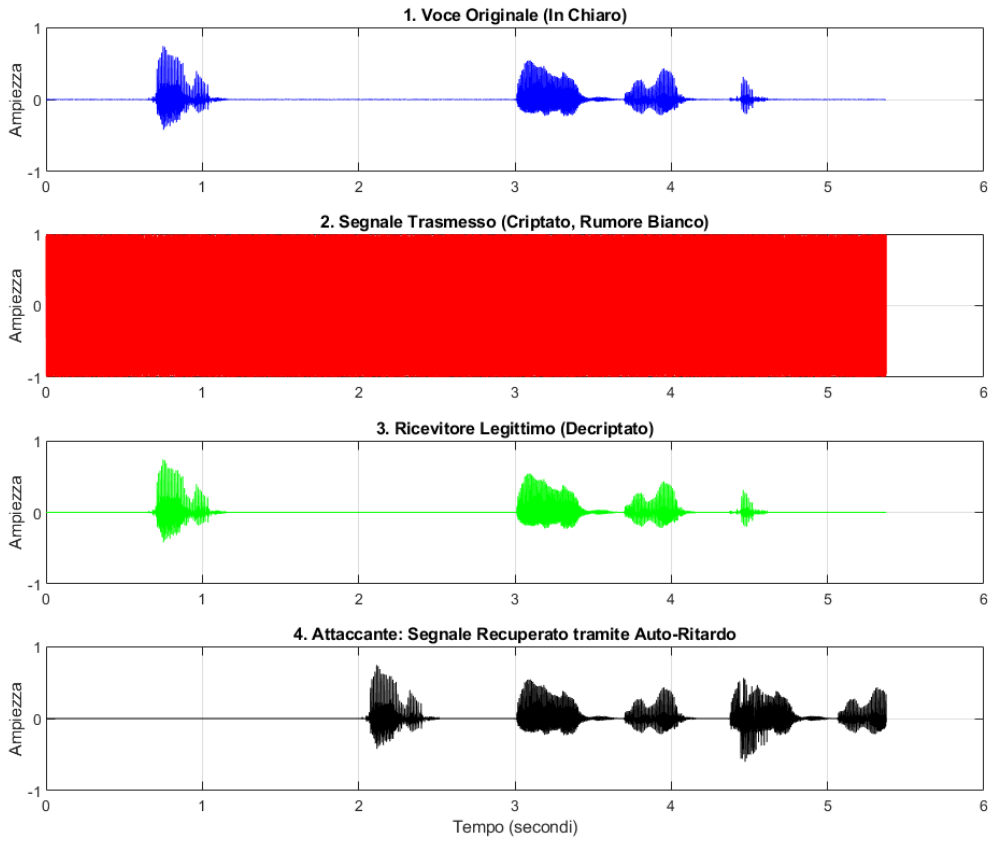


Figura 3.1: Tracciati d'onda della simulazione Model-Based Design (MATLAB). Il grafico inferiore (4) mostra il segnale recuperato tramite Auto-Ritardo.

1. **Intervallo** $0.0s \leq t < 1.365s$**:** Il buffer di memoria passato è vuoto. L'attacco non può computare lo XOR e l'output in questa finestra rimane confinato a rumore non decodificato.

2. **Intervallo** $1.365s \leq t < 1.865s$**:** Il tempo presente risiede nella traccia di silenzio ($M(t) = 0$). Il tempo passato ($t - 1.365s$) punta alla finestra $[0.0s, 0.5s]$, anch'essa vuota prima dell'inizio del parlato. L'equazione computa $0 \oplus 0 = 0$, generando silenzio.

3. **Intervallo** $1.865s \leq t < 2.365s$ **(Recupero ritardato di "Prova"):** Il tempo presente si trova ancora nella finestra di silenzio ($M(t) = 0$). Sottraendo il ritardo T, il tempo passato ($t - 1.365s$) intercetta invece la finestra $[0.5s, 1.0s]$, ovvero l'esatto istante in cui in origine era stata pronunciata la parola *"Prova"*. La formula si risolve in:

$$\text{Recupero}(t) = 0 \oplus \text{Voce("Prova")} = \text{Voce("Prova")}$$

La parola originaria emerge in modo *bit-perfect*, facendo ascoltare all'intercettatore un'eco fedele del primo fonema, traslato visivamente nel grafico esattamente sopra la zona piana di silenzio.

4. **Intervallo** $3.0s \leq t < 4.365s$ **(Recupero in tempo reale di "Uno, due"):** Il tempo presente intercetta l'inizio della sequenza numerica (*"Uno, due"*). Il tempo passato ($t - 1.365s$) punta alla seconda metà del silenzio ambientale ($M(t - T) = 0$). L'equazione determina:

$$\text{Recupero}(t) = \text{Voce("Uno, due")} \oplus 0 = \text{Voce("Uno, due")}$$

In questa finestra l'utente ascolta l'audio estratto in perfetta sincronia col presente, poiché la chiave passata (il silenzio) non introduce alterazioni. Nel tracciato grafico, la curva nera dell'attacco converge matematicamente con quella blu originale, occultandosi perfettamente al di sotto di essa.

5. **Intervallo** $t \geq 4.365s$ **(Collisione Acustica):** Il tempo presente intercetta la fine dell'audio in chiaro (la parola *"tre"*), ma il tempo passato supera la barriera dei 3.0 secondi, veicolando l'inizio della sequenza ritardata (le parole *"uno, due"*). L'equazione genera la sovrapposizione logica:

$$\text{Recupero}(t) = \text{Voce("tre")} \oplus \text{Voce("uno, due")}_{\text{ritardato}}$$

L'interazione bit-a-bit tra le due onde in banda vocale genera un mascheramento distruttivo. Dal punto di vista psicoacustico, l'energia della parola *"tre"* (presente) viene totalmente sovrastata e distorta dai fonemi ritardati *"uno, due"*, portando l'intercettatore a percepire l'ingannevole sequenza finale: *"...Prova... uno, due... uno, due"* (fenomeno apprezzabile nel dettaglio in Figura 3.3).

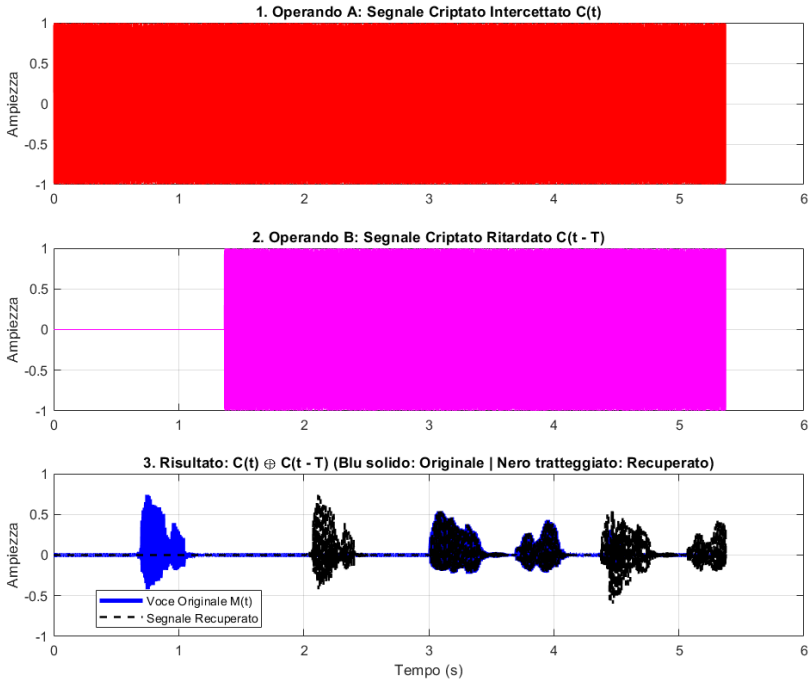


Figura 3.2: Dettaglio analitico delle componenti in XOR. La sovrapposizione tra l'output dell'attacco (nero) e la traccia nativa (blu trasparente) evidenzia visivamente le zone di perfetta convergenza e le aree di collisione distruttiva.

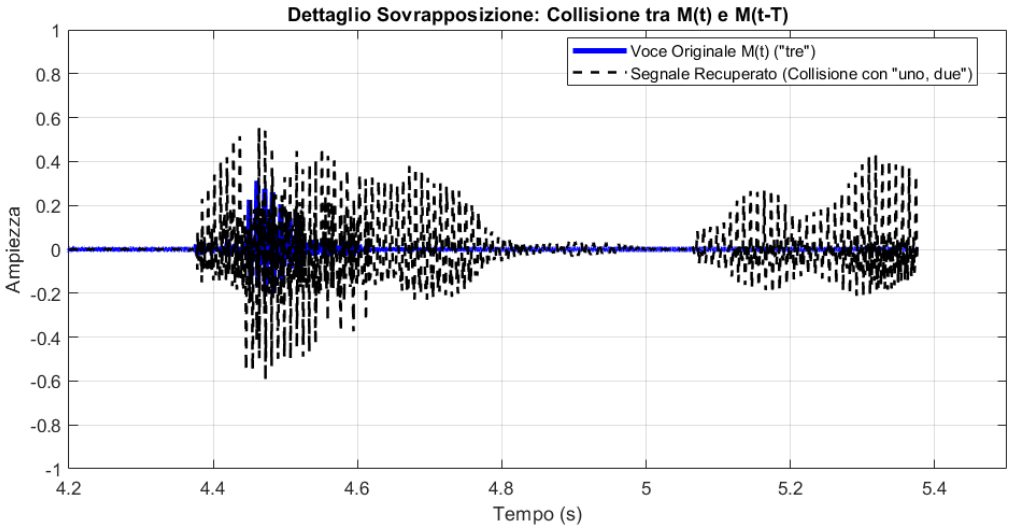


Figura 3.3: Dettaglio ingrandito della collisione acustica nell'intervallo temporale finale. L'asse delle ascisse è limitato alla finestra $t \in [4.2s, 5.5s]$. È visibile come il segnale recuperato (tratteggio nero, corrispondente alle parole "uno, due" ritardate) sovrasti e distrugga l'ampiezza della traccia nativa (linea blu continua, corrispondente alla parola "tre"), rendendone impossibile l'estrazione in chiaro.

3.3.3 Condizioni di Successo e Applicabilità Reale dell'Attacco

L'analisi temporale appena condotta evidenzia una proprietà fondamentale dell'attacco *Two-Time Pad* applicato ai segnali audio: l'estrazione in chiaro del messaggio ha successo **esclusivamente quando una delle due finestre temporali sovrapposte corrisponde a una pausa di silenzio** ($M(t) = 0$ oppure $M(t - T) = 0$). Al contrario, quando l'equazione costringe la sovrapposizione di due porzioni attive di fonemi, le ampiezze collidono distruggendo reciprocamente l'intellegibilità dell'informazione.

È fondamentale precisare che il campione audio utilizzato per questo collaudo è stato **costruito ad hoc con finalità puramente dimostrative**. In uno scenario reale, specialmente nelle comunicazioni radio tattiche (dove si predilige un eloquio rapido e continuo tramite protocolli *Push-to-Talk*), è altamente improbabile che un operatore inserisca pause di assoluto silenzio lunghe ben due secondi.

Tuttavia, il *Penetration Test* ha assolto al suo scopo di dimostrare la vulnerabilità strutturale del sistema a 16 bit: benché nel parlato naturale le pause siano notevolmente più brevi (respiri, spazi inter-sillabici, esitazioni), la periodicità della chiave lascerebbe comunque "scoperti" frammenti di parole e fonemi isolati. In ambito di sicurezza delle telecomunicazioni (COMSEC), la possibilità di intercettare anche una singola frazione del messaggio originale costituisce una *falla critica*, motivando categoricamente la necessità di scalare la sintesi hardware ai 32 o 64 bit (come verrà discusso nel Capitolo 7) per garantire la totale aperiodicità della chiave all'interno della medesima sessione di trasmissione.

CAPITOLO 4

Progettazione dell'IP Core (Vitis HLS)

L'obiettivo di questa fase è la traduzione del modello matematico dell'LFSR di Galois in una descrizione *Register Transfer Level* (RTL) sintetizzabile su FPGA. La progettazione è stata condotta avvalendosi dell'approccio **High-Level Synthesis (HLS)** tramite l'ambiente di sviluppo **Xilinx Vitis HLS**.

4.1 HLS vs RTL Manuale: Analisi delle Scelte Progettuali

La scelta di adottare la sintesi di alto livello (HLS) anziché procedere con la tradizionale descrizione manuale in linguaggio HDL (Verilog o VHDL) rappresenta una decisione architetturale consapevole, strettamente legata all'evoluzione storica del paradigma di *Hardware/Software Co-Design*.

Da decenni l'ingegneria informatica insegue il traguardo del partizionamento automatico: l'idea, a lungo considerata un "mito" accademico, di poter scrivere un intero sistema in un linguaggio ad alto livello e demandare a un compilatore la scelta di cosa eseguire su CPU e cosa sintetizzare in silicio. Sebbene i tentativi passati (come il SystemC) non abbiano raggiunto una diffusione industriale universale, la moderna **High-Level Synthesis (HLS)** concretizza in gran parte questa visione, permettendo di ottenere acceleratori hardware dedicati partendo da sorgenti C/C++.

In un confronto diretto con la scrittura HDL manuale, l'approccio HLS adottato per questo progetto offre vantaggi determinanti:

1. **Astrazione nativa e Risoluzione dei Conflitti di Direzione (AXI4):** Come si vedrà nella Sezione 4.4, l'HLS permette di dotare l'acceleratore di un'interfaccia standard AXI4-Lite nativa dichiarando un semplice `#pragma`. È doveroso precisare che, anche nella progettazione RTL manuale pura, l'ambiente Vivado (tramite il wizard *Create and Package New IP*) è in grado di avvolgere in automatico il codice Verilog custom all'interno di un'infrastruttura di comunicazione AXI, sollevando il progettista dall'onere di scrivere da zero le macchine a stati per l'*handshaking* del protocollo.

   Tuttavia, l'approccio HLS offre un livello di astrazione nettamente superiore per quanto riguarda il *binding* e, soprattutto, la **direzionalità** dei registri. Nel wizard RTL, i registri generati (es. `slv_reg0`, `slv_reg1`) nascono nativamente per essere scritti dal *Master* (il processore ARM). Se l'acceleratore custom deve esporre un risultato calcolato in modo continuo dall'hardware (come il nostro output `y` crittografato), il progettista è costretto a forzare una "redirezione" nel codice sorgente: quando il processore ARM richiede di leggere l'indirizzo associato al registro, il codice Verilog deve essere alterato manualmente affinché non restituisca il contenuto del registro generato dall'interfaccia, ma il valore della variabile custom calcolata dall'hardware. Questa disconnessione ha come costo architetturale quello di sprecare i 32 bit di Flip-Flop allocati nel silicio per il vero registro (che rimangono fisicamente inutilizzati) e invalida inevitabilmente le routine di *Self-Test* dei driver generati da Vitis (le quali tentano di scrivere un valore fittizio e rileggerlo identico, scontrandosi con il fatto che ora in lettura viene forzato il risultato dell'hardware).

   Con Vitis HLS questo conflitto architetturale è inesistente. Il compilatore inferisce automaticamente la semantica delle variabili C: identificando `y` come un parametro di uscita

passato per riferimento, sintetizza nativamente un registro AXI *Read-Only*. In questo modo, ottimizza l'uso del silicio (non istanziando logica di scrittura superflua) e genera driver software intrinsecamente sicuri e coerenti, le cui funzioni di Self-Test non produrranno mai falsi fallimenti.

2. **Verifica e Validazione (C-Simulation):** La validazione funzionale di un IP Core descritto in C (tramite l'esecuzione nativa su PC del *testbench*) risulta ordini di grandezza più rapida rispetto alla complessa simulazione temporale di un modulo Verilog in ambienti come ModelSim o Vivado XSIM.

3. **Time-to-Market:** Lo sviluppo di un acceleratore tramite descrizione RTL manuale richiede tipicamente mesi di progettazione e debugging per garantire prestazioni ottimali. L'HLS, al contrario, abbatte drasticamente i tempi di sviluppo (nell'ordine di giorni), offrendo un enorme vantaggio in fase di prototipazione.

Tuttavia, l'utilizzo dell'HLS non prescinde dalle logiche dell'elettronica digitale e impone rigidi compromessi. In primo luogo, la **mentalità del progettista** deve rimanere ancorata all'hardware soggiacente: scrivere codice C per un processore General Purpose (dotato di sistema operativo e decine di Gigabyte di RAM) è concettualmente diverso dallo scrivere C per la sintesi su FPGA. Come evidenziato dai limiti intrinseci del linguaggio in questo contesto, nella logica programmabile non esiste un file system (precludendo l'uso di funzioni come `fprintf` nell'IP sintetizzato) e, soprattutto, non è tollerata l'**allocazione dinamica della memoria** (`malloc`). Le risorse logiche devono essere note e istanziate staticamente a priori.

In secondo luogo, si accetta un compromesso prestazionale spaziale. Un ingegnere RTL esperto, scrivendo codice Verilog manuale, manterrà un controllo deterministico sul singolo ciclo di clock, estraendo le massime prestazioni assolute con un'impronta di area minima. Il codice RTL generato dall'HLS risulta inevitabilmente più verboso e meno ottimizzato a livello di gate logici. Nel caso di studio del nostro IP Scrambler, la semplicità matematica dell'algoritmo LFSR mitiga quasi del tutto lo svantaggio spaziale dell'HLS, permettendo di capitalizzare i vantaggi in termini di produttività software senza subire penalizzazioni tangibili sull'occupazione delle risorse hardware della scheda Zybo Z7.

4.2 Definizione dell'Interfaccia e dei Tipi di Dato

Il primo passo nella progettazione HLS consiste nel definire l'interfaccia del modulo e la precisione dei dati. Come visibile nel file *header* (`scrambler.h`, riportato nell'Appendice B), sono stati definiti due tipi di dato fondamentali:

- `data_t`: mappato su uno `short` (16 bit con segno). Rappresenta il singolo campione audio letto dal codec. La scelta dei 16 bit è ottimale per il bilanciamento tra qualità audio e risorse hardware, come discusso nel capitolo precedente.

- `seed_t`: mappato su un `unsigned short` (16 bit senza segno). Costituisce lo stato interno del registro LFSR e la chiave crittografica.

Il prototipo della funzione *top-level* è il seguente:

```
1 void scrambler(data_t *y, data_t x, seed_t seed, bool load_seed);
```

L'argomento `x` (input) è passato per valore, venendo mappato dal sintetizzatore come una semplice porta di lettura. L'argomento `y` (output) è passato per riferimento tramite puntatore: in hardware, questo si traduce nella generazione di un registro di uscita in cui il risultato viene validato a fine elaborazione.

4.3 Architettura del Core Crittografico

L'implementazione del generatore pseudo-casuale e dell'operazione di scrambling risiede nel file `scrambler.c` (Appendice B).

Un elemento architetturale di fondamentale importanza è la dichiarazione della variabile di stato dell'LFSR:

```
static seed_t lfsr_state = 0xACE1;
```

L'utilizzo della keyword `static` impone al sintetizzatore di allocare un registro fisico nella logica sequenziale (Flip-Flop) capace di mantenere la memoria del proprio valore tra chiamate successive della funzione. Senza questa direttiva, lo stato verrebbe distrutto a ogni campione audio elaborato, invalidando la generazione della sequenza.

La logica implementa inoltre un controllo di sicurezza per prevenire il *lock-up state*: poiché un LFSR inizializzato a zero rimane bloccato permanentemente a zero, se il processore tenta di forzare un `seed` nullo tramite il segnale `load_seed`, l'hardware sovrascrive autonomamente l'ingresso con un valore di *default* di sicurezza.

Il core dell'algoritmo (calcolo del nuovo stato ed esecuzione dello XOR) non fa uso di cicli iterativi, ma si affida a operazioni *bitwise* elementari, mappandosi direttamente in porte logiche combinatorie:

```
unsigned short lsb = lfsr_state & 1;
lfsr_state >>= 1;
if (lsb) {
    lfsr_state ^= 0xB400;
}
*y = x ^ (data_t)lfsr_state;
```

4.4 Direttive di Sintesi (Pragmas AXI4-Lite)

Affinché il processore Zynq (PS) possa comunicare con l'IP generato, le porte della funzione C devono essere dotate di protocolli di comunicazione standard. Tramite le direttive `#pragma HLS INTERFACE`, si è scelto di adottare il protocollo **AXI4-Lite** per tutti gli ingressi e le uscite.

Le direttive implementate nel sorgente per mappare i segnali sull'interfaccia bus sono le seguenti:

```
// Direttive per l'interfacciamento AXI4-Lite (bundle unico scrambler_io)
#pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=return
#pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=x
#pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=y
#pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=seed
#pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=load_seed
```

L'utilizzo del parametro `bundle=scrambler_io` istruisce il tool di sintesi a raggruppare tutte le porte fisiche dell'IP sotto un'unica interfaccia *slave* AXI4-Lite. Dal punto di vista dell'architettura di sistema, il blocco apparirà sul bus di comunicazione come un singolo *endpoint* dotato di un unico indirizzo base di memoria. I vari segnali (`x`, `y`, `seed`) saranno accessibili dal processore ARM semplicemente sommando un offset specifico a questo indirizzo base. Senza questo raggruppamento, Vitis HLS genererebbe interfacce bus indipendenti per ogni variabile, complicando notevolmente il cablaggio nel Block Design di Vivado e sprecando risorse logiche.

L'inclusione della porta `return` nel medesimo bundle è altrettanto fondamentale, in quanto impone la generazione dell'interfaccia di controllo hardware **AP_CTRL** e la sua mappatura

all'interno del registro AXI (all'offset `0x00`). Questa interfaccia espone lo stato operativo dell'algoritmo tramite i segnali di *handshaking* standard del protocollo HLS (*Block-Level Interface Protocols*):

- **`ap_start`:** È un bit di comando. Il processore ARM vi scrive un '1' logico per impartire al circuito l'ordine fisico di iniziare l'elaborazione del campione.
- **`ap_done`:** È un bit di stato. L'hardware lo alza a '1' autonomamente per segnalare al processore di aver terminato i calcoli e che il dato nel registro di uscita (`y`) è stabile e pronto per essere letto.
- **`ap_idle` e `ap_ready`:** Indicano rispettivamente che il blocco hardware è a riposo (non sta calcolando nulla) o che è pronto ad accettare nuovi dati in ingresso alla transizione di clock successiva.

Mappando questi segnali sul bus, essi diventano accessibili dal processore. Ciò permette al software C in Vitis di orchestrare l'acceleratore: ad esempio, la funzione C che verrà generata da Vitis `XScrambler_Start()` non fa altro che scrivere '1' sul bit *ap_ start*, mentre il bit *ap_ done* viene cablato fisicamente per scatenare un *interrupt* verso l'ARM non appena il calcolo è concluso.

4.4.1 Considerazioni sulle direttive di ottimizzazione

In implementazioni DSP complesse, come nel caso di filtri FIR che operano su array tramite cicli `for`, l'utilizzo di direttive di ottimizzazione quali `PIPELINE` o `UNROLL` risulta strettamente necessario per abbassare l'Initiation Interval (II) e rispettare i vincoli temporali del campionamento audio.

Nel presente progetto, si è optato per non includere tali direttive. La motivazione è architetturale: l'algoritmo LFSR implementato non presenta cicli (*loops*), ma esegue l'elaborazione del singolo campione tramite un percorso *feed-forward* estremamente lineare e povero di operazioni logiche complesse. Come verrà dimostrato nell'analisi dei report di sintesi, la latenza naturale del blocco risulta già abbondantemente inferiore al periodo di campionamento dell'audio, rendendo superflua qualsiasi forzatura sul pipelining.

4.5 Validazione Algoritmica (C Simulation)

Prima di procedere alla sintesi RTL, il comportamento del codice è stato validato tramite una simulazione software in C. Il *testbench* (`scrambler_test.c`, Appendice B) è stato progettato per verificare tre requisiti critici del sistema:

1. **Verifica del Periodo Matematico:** Una routine autonoma all'interno del testbench isola la logica dell'LFSR e conta il numero di iterazioni necessarie affinché lo stato torni al valore di partenza (*seed*). Il test ha confermato matematicamente che il polinomio `0xB400` genera esattamente 65.535 combinazioni uniche (pari a $2^{16} - 1$), confermandone la primitività.
2. **Simulazione di Trasmissione e Ricezione:** Il main del testbench simula l'interazione tra due dispositivi (Tx e Rx). Viene fornito un array di campioni audio in chiaro; l'algoritmo viene inizializzato per simulare la criptazione (Tx), e i dati risultanti vengono nuovamente passati alla funzione reinizializzata con lo stesso *seed* per simulare la decrittazione (Rx). Un controllo finale assicura che non vi sia alcun degrado dell'informazione.
3. **Generazione Dataset per Analisi Statistica:** Sono state implementate due routine specifiche per esportare su file di testo l'intera sequenza di campioni generata dall'algoritmo. Questa esportazione è vitale per poter valutare la bontà crittografica del rumore in un ambiente di analisi dati avanzato (MATLAB).

4.5.1 Analisi dell'Output di Simulazione

L'esecuzione della *C Simulation* in Vitis HLS ha prodotto risultati che validano la correttezza logica del sistema sotto ogni aspetto testato. Di seguito è riportato l'estratto completo del log di simulazione:

```
--- SIMULAZIONE SCRAMBLER ---
Inizializzazione TX con Seed...
Inizializzazione RX con stesso Seed...
In:     0 | Criptato:   1165 | Decriptato:     0
In:   100 | Criptato: -18910 | Decriptato:   100
In:  -200 | Criptato: -23525 | Decriptato:  -200
In:   300 | Criptato: -26435 | Decriptato:   300
In:  -400 | Criptato:   1720 | Decriptato:  -400
In:   500 | Criptato:  32144 | Decriptato:   500
In:  -600 | Criptato: -15462 | Decriptato:  -600
In:   700 | Criptato:   7589 | Decriptato:   700
In:  -800 | Criptato:  18284 | Decriptato:  -800
In:   900 | Criptato:  24130 | Decriptato:   900

SUCCESSO! Il segnale decriptato corrisponde all'originale.

--- VERIFICA PERIODO MASSIMO LFSR ---
L'LFSR si e' ripetuto dopo 65535 colpi di clock.
SUCCESSO! L'LFSR ha lunghezza massima (2^16 - 1).
File 'lfsr_samples.txt' generato con successo per MATLAB!
File 'lfsr_samples_subottimale.txt' generato con successo!

TEST COMPLETO SUPERATO CON SUCCESSO!
```

Listing 4.1: Estratto del log della simulazione C in Vitis HLS.

L'analisi di questo output conferma il raggiungimento di tutti gli obiettivi prefissati:

- **Efficacia e Reversibilità (Tx/Rx):** Come si evince dalla prima parte del log, l'operazione di scrambling maschera completamente l'ampiezza del segnale. Ad esempio, una rampa crescente in ingresso $(100, 300, 500)$ viene mappata in valori apparentemente caotici $(-18910, -26435, 32144)$, simulando l'introduzione di rumore bianco. La decrittazione ripristina i valori esatti senza perdita di informazione, confermando la perfetta reversibilità dell'operatore XOR.

- **Validazione del Polinomio Primitivo:** La routine di verifica ha isolato la logica del generatore confermando empiricamente la teoria: il registro LFSR ha impiegato esattamente 65.535 colpi di clock prima di ritornare allo stato iniziale. Questo certifica che la maschera esadecimale `0xB400` implementa un polinomio *primitivo*, garantendo la massima lunghezza possibile per un'architettura a 16 bit.

- **Esportazione Dataset:** Infine, il sistema segnala la corretta generazione e chiusura dei file di testo sul *file system* locale, confermando che i dati sono pronti per essere importati nel tool di analisi (MATLAB).

L'assenza di errori in questa fase garantisce che l'IP sia logicamente solido e pronto per il *C/RTL Co-simulation* e la successiva sintesi hardware.

4.5.2 Esportazione dei Dataset (File Dumping)

Per dimostrare la robustezza del mascheramento sonoro, non è sufficiente verificare il funzionamento a livello di bit, ma occorre analizzare le proprietà statistiche e spettrali del rumore

generato. A tal fine, il testbench si avvale della libreria standard C (`stdio.h`) per effettuare il *dumping* dei dati su *file system*.

Sono state sviluppate due funzioni dedicate, i cui listati completi sono presenti in Appendice B. La prima, `genera_dataset_matlab_opt`, esegue la logica LFSR per l'intero periodo massimo (65.535 cicli) utilizzando il polinomio primitivo ottimale (`0xB400`) e salva ogni stato in un file di testo (`lfsr_samples.txt`):

```
// Generazione dell'intero periodo massimo
for(long i = 0; i < 65535; i++) {
    fprintf(fp, "%d\n", current); // Scrittura del campione su file

    // Emulazione logica hardware
    unsigned short lsb = current & 1;
    current >>= 1;
    if (lsb) {
        current ^= 0xB400; // Polinomio primitivo (Tap 16, 14, 13, 11)
    }
}
```

Listing 4.2: Dettaglio della scrittura sequenziale dei campioni su file nel testbench.

La seconda funzione, `genera_dataset_matlab_subopt`, compie un'operazione speculare ma altera deliberatamente il polinomio generatore, impiegando una configurazione matematicamente *non primitiva* (ad esempio `0x0011`) e generando così un secondo file di test (`lfsr_samples_subottimale.txt`).

Questa esportazione consente di disaccoppiare la generazione dei dati dalla successiva validazione matematica. I due file prodotti vengono importati in ambiente MATLAB per dimostrare oggettivamente come la scelta del polinomio generatore determini l'effettiva sicurezza del sistema.

4.6 Analisi Statistica e Validazione della Casualità

Per verificare che il rumore generato sia efficace per mascherare l'audio, i dati sono stati analizzati in ambiente MATLAB (i relativi script sono riportati integralmente in Appendice B) tramite la funzione di **Autocorrelazione discreta** $R_{xx}(m)$:

$$R_{xx}(m) = \sum_{n} x[n] \cdot x[n-m]$$

L'autocorrelazione indica quanto un segnale "assomiglia a sé stesso" quando viene spostato nel tempo di un certo numero di campioni (m).

- Se il segnale è davvero casuale (simile al rumore bianco), l'autocorrelazione deve avere un valore altissimo solo quando lo spostamento è zero ($m = 0$).
- Se invece l'autocorrelazione presenta picchi elevati in altri punti, significa che il rumore contiene dei **pattern ripetitivi**.

In ambito crittografico, un rumore con pattern ripetitivi è pericoloso perché un attaccante potrebbe prevedere l'andamento della "chiave" e ricostruire il segnale originale.

4.6.1 Condizionamento dei Dati: Rimozione della Componente Continua

Prima di procedere al calcolo dell'autocorrelazione, si è reso necessario un filtraggio matematico preliminare sui dati grezzi. L'LFSR a 16 bit genera valori strettamente positivi nell'intervallo $[1, 65535]$. Dal punto di vista dell'analisi dei segnali, questa distribuzione asimmetrica rispetto allo zero implica la presenza di una forte componente continua (DC offset), il cui valore equivale alla media della distribuzione stessa ($\mu_x \approx 32767$).

Qualora l'autocorrelazione venisse calcolata direttamente su questo segnale, i prodotti incrociati della sommatoria risulterebbero costantemente positivi per qualsiasi ritardo m. Questo genererebbe un inviluppo triangolare di ampiezza spropositata che andrebbe a mascherare completamente le fluttuazioni ad alta frequenza (la componente alternata, o AC), le quali racchiudono le reali proprietà di casualità della sequenza.

Per ovviare a questo problema, si procede con l'operazione di centratura (o *mean removal*), sottraendo la media campionaria e ottenendo così un segnale a media nulla:

$$x_{zm}[n] = x[n] - \mu_x$$

In questo dominio traslato, l'oscillazione bilanciata tra valori positivi e negativi garantisce che, per ritardi $m \neq 0$, i prodotti incrociati si annullino mutuamente per interferenza distruttiva, permettendo di isolare e misurare con precisione il grado di scorrelazione del rumore.

4.6.2 Analisi del Polinomio Ottimale

L'autocorrelazione della sequenza generata con la maschera `0xB400` (Figura 4.1) conferma che il sistema produce un ottimo rumore pseudo-casuale.

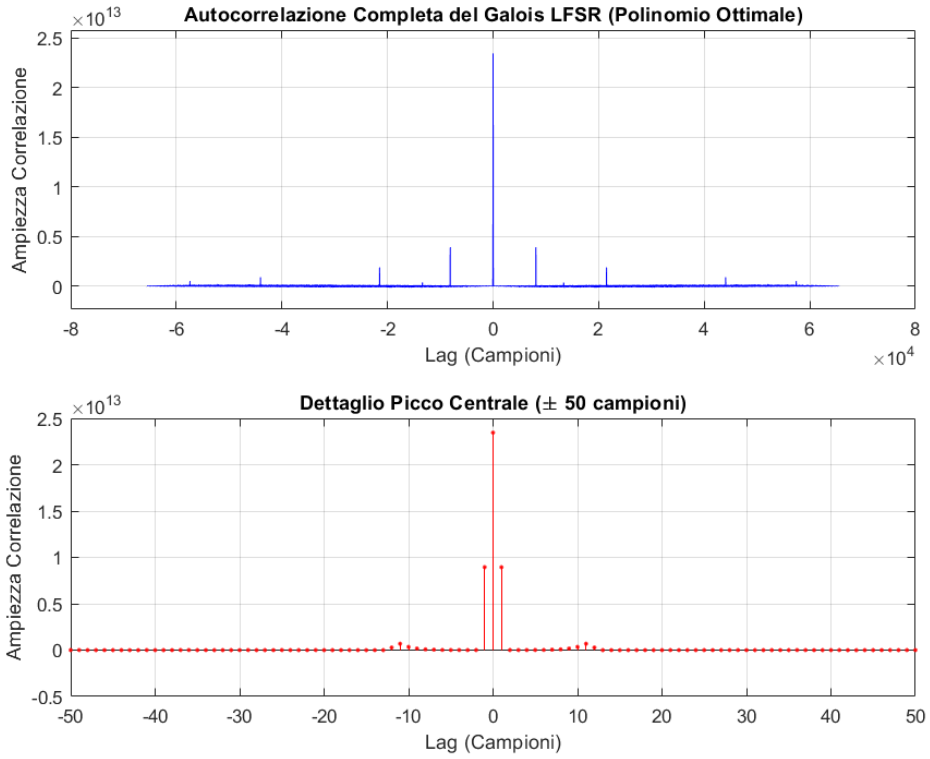


Figura 4.1: Autocorrelazione della sequenza ottimale. Il picco netto a zero indica che il rumore è imprevedibile.

Il grafico mostra un unico picco centrale dominante. I piccolissimi rilievi che si vedono sul fondo (chiamati anche "lobi secondari") sono una caratteristica normale delle sequenze generate da un registro a scorrimento (tecnicamente chiamate **PRBS** - *Pseudo-Random Binary Sequences*). Poiché l'LFSR non genera vero rumore infinito ma una sequenza che si ripete ogni 65.535 campioni, la matematica non potrà mai dare uno "zero" perfetto ovunque. Tuttavia, questi rilievi sono così piccoli da essere inaudibili all'orecchio umano e non compromettono in alcun modo la qualità del mascheramento audio.

4.6.3 Confronto Spettrale: Polinomio Ottimale vs Sub-ottimale

Per dimostrare l'impatto critico dell'architettura hardware sulla sicurezza del sistema, l'analisi statistica è stata estesa a un dataset generato alterando deliberatamente i *tap* del registro. Nello specifico, si è messo a confronto il polinomio primitivo di progetto (`0xB400`) con una configurazione matematica sub-ottimale (`0x0011`). I risultati di questa analisi spettrale comparata sono esposti nella Figura 4.2.

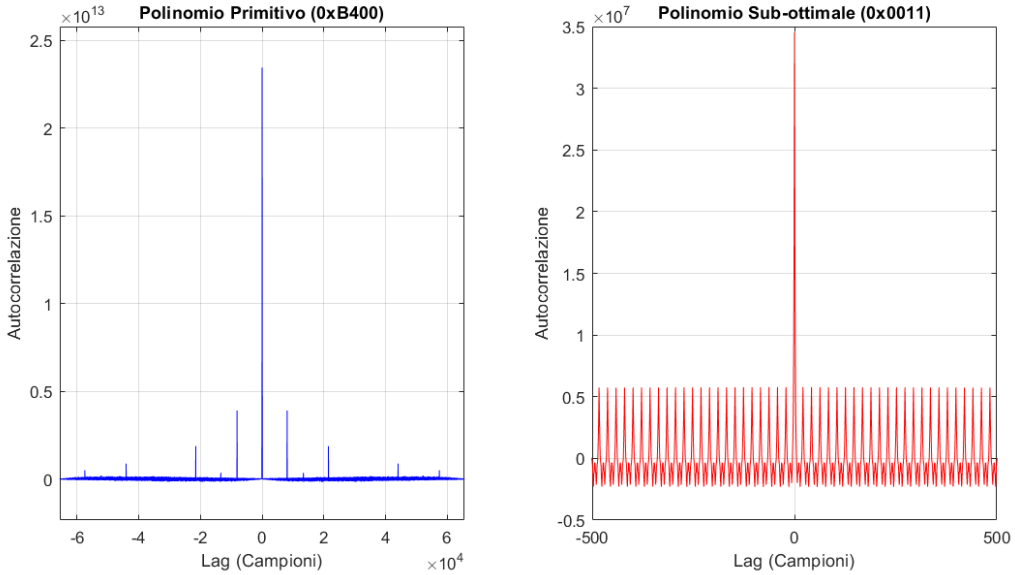


Figura 4.2: Confronto dell'autocorrelazione tra l'LFSR con polinomio primitivo (sinistra) e l'LFSR con configurazione sub-ottimale (destra). L'effetto pettine nel grafico di destra evidenzia una severa criticità nella distribuzione pseudo-casuale.

L'osservazione dei due spettri di autocorrelazione evidenzia una dicotomia netta nel comportamento matematico dei due generatori:

- **Generatore Ottimale (Sinistra):** L'autocorrelazione approssima in modo eccellente la risposta impulsiva ideale del rumore bianco (Delta di Dirac). Il netto isolamento del picco centrale dimostra una scorrelazione pressappoco totale per ogni ritardo $m \neq 0$. I trascurabili lobi secondari presenti sul *noise floor* rappresentano un artefatto fisiologico intrinseco alla natura pseudo-casuale (e dunque finita) delle sequenze PRBS, ma risultano di ampiezza tale da non compromettere in alcun modo l'entropia del mascheramento.
- **Generatore Sub-ottimale (Destra):** L'autocorrelazione subisce un vistoso degrado strutturale, palesando la comparsa di picchi multipli di ampiezza massimale distribuiti a intervalli regolari. Questo fenomeno, noto in teoria dei segnali come "effetto pettine", costituisce la firma spettrale inequivocabile di un segnale fortemente periodico. La configurazione non primitiva dei *tap* causa infatti un collasso prematuro della transizione di stato, costringendo l'LFSR a iterare in *loop* su un sottoinsieme ristretto di combinazioni, ben al di sotto dei 65.535 stati attesi.

In un contesto di telecomunicazione protetta, l'adozione del polinomio sub-ottimale vanificherebbe l'intero impianto di sicurezza. La spiccata ripetitività a breve termine del rumore esporrebbe la trasmissione al rischio di crittoanalisi statistica: un attaccante ostile potrebbe agilmente isolare il *pattern* periodico, inferire la configurazione della logica di retroazione e, infine, separare lo *scrambling* dalla portante vocale in chiaro. La presente validazione certifica pertanto il rigore matematico e la solidità architetturale dell'IP Core sintetizzato in Vitis HLS.

4.7 Sintesi di Alto Livello (High-Level Synthesis) e Analisi dei Report

Una volta verificata l'integrità logica e matematica dell'algoritmo tramite simulazione C e analisi MATLAB, si è proceduto alla sintesi di alto livello (*C Synthesis*). In questa fase, il tool Vitis HLS analizza il codice C e genera la descrizione hardware equivalente a livello RTL ottimizzata per il dispositivo target (*xc7z020-clg400-1*, ovvero il chip della Zybo Z7).

L'esito di questo processo produce il **Synthesis Summary Report**, un documento fondamentale che quantifica le prestazioni fisiche del blocco generato. L'analisi si divide in quattro metriche principali: vincoli temporali, prestazioni, interfacce e occupazione delle risorse.

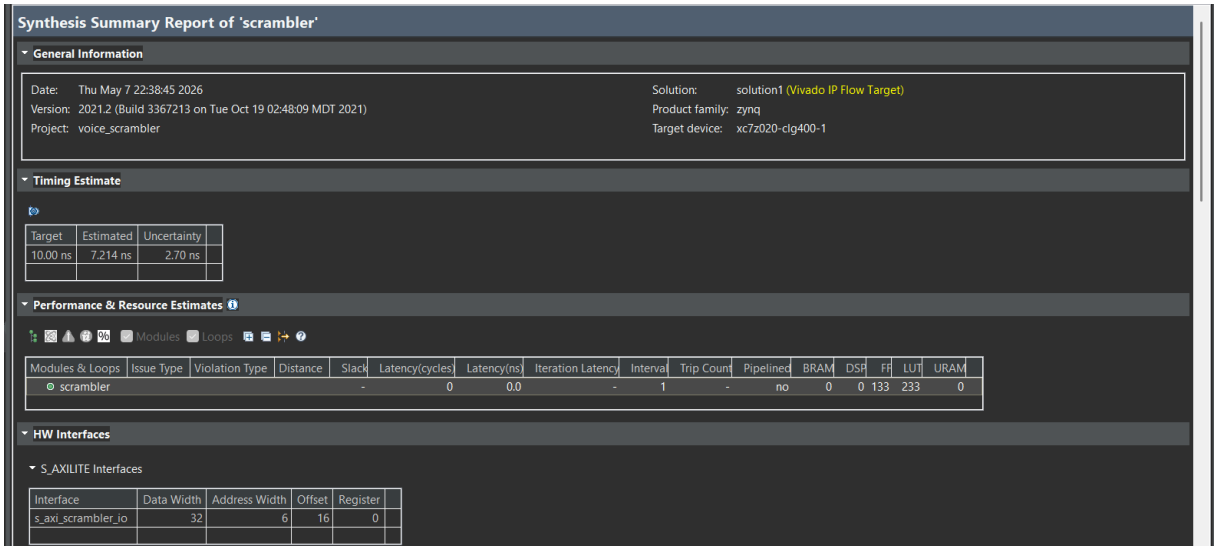


Figura 4.3: Synthesis Summary Report di Vitis HLS, con dettaglio sulle stime temporali (Timing Estimate), le prestazioni algoritmiche (Latenza e Initiation Interval) e l'occupazione delle risorse hardware.

4.7.1 Stima delle Tempistiche (Timing Estimate)

Il primo parametro da validare è il rispetto dei vincoli temporali (*Timing*). Il sistema SoC è progettato affinché il bus AXI4-Lite operi a una frequenza di 100 MHz, imponendo un periodo di clock target (*Target Period*) pari a 10.00 ns.

Come evidenziato nella Figura 4.3, il percorso critico (*Critical Path*) stimato per l'IP Scrambler richiede un tempo di propagazione di 7.214 ns. L'aggiunta del margine di incertezza tecnologica (*Uncertainty*) di 2.70 ns porta il totale vicino al limite, ma comunque al di sotto dei 10 ns. Questo indica un *slack* positivo: il circuito opererà correttamente alla frequenza richiesta senza incorrere in violazioni temporali (*timing violations*) o metastabilità.

4.7.2 Prestazioni: Latenza e Initiation Interval

La valutazione delle prestazioni in Vitis HLS si basa tipicamente su due parametri metrici fondamentali, visibili nella sezione *Performance & Resource Estimates* (Figura 4.3):

- **Latenza (Latency):** Indica il numero di cicli di clock necessari affinché l'IP produca il risultato in uscita a partire dal momento in cui ha acquisito tutti gli ingressi.
- **Intervallo di Iniziazione (Initiation Interval - II):** Indica il numero di cicli di clock che devono trascorrere prima che l'IP possa accettare un nuovo set di dati in ingresso.

L'analisi del report mostra valori eccezionali per entrambi i parametri: **Latenza pari a 0 cicli** e **Initiation Interval pari a 1 ciclo**.

Questi dati rivestono un'importanza architetturale fondamentale, in quanto convalidano e giustificano la scelta, discussa nei paragrafi precedenti, di **non adottare direttive di ottimizzazione** (quali `#pragma HLS PIPELINE` o `UNROLL`). La latenza di zero cicli denota che l'intero percorso dati (l'aggiornamento dello stato dell'LFSR e l'operazione di XOR) viene risolto in logica puramente combinatoria all'interno di un singolo colpo di clock. Parallelamente, un *Initiation Interval* unitario significa che l'hardware è già in grado di elaborare un nuovo campione audio ad ogni singolo ciclo di clock del bus (100 milioni di campioni al secondo). L'inserimento di una direttiva di *pipelining* in questo scenario sarebbe risultato non solo ridondante, ma tecnicamente errato, in quanto l'algoritmo non presenta cicli iterativi da srotolare o percorsi sequenziali lunghi da segmentare in stadi.

4.7.3 Mappatura delle Interfacce Software-Hardware (SW-to-HW Mapping)

Un'ulteriore sezione critica del report riguarda la trasposizione delle variabili C nei registri fisici dell'interfaccia AXI4-Lite (*SW I/O Information* e *HW Interfaces*).

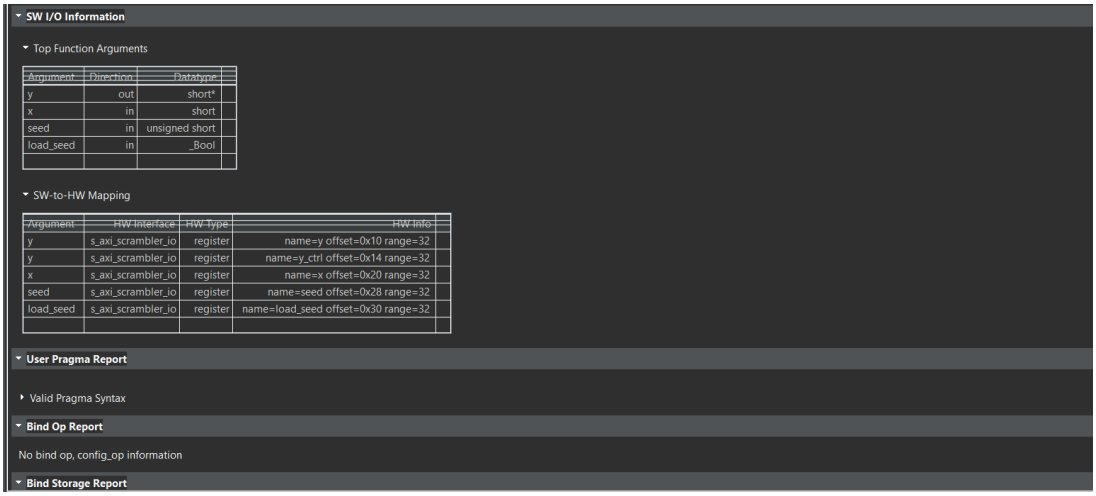
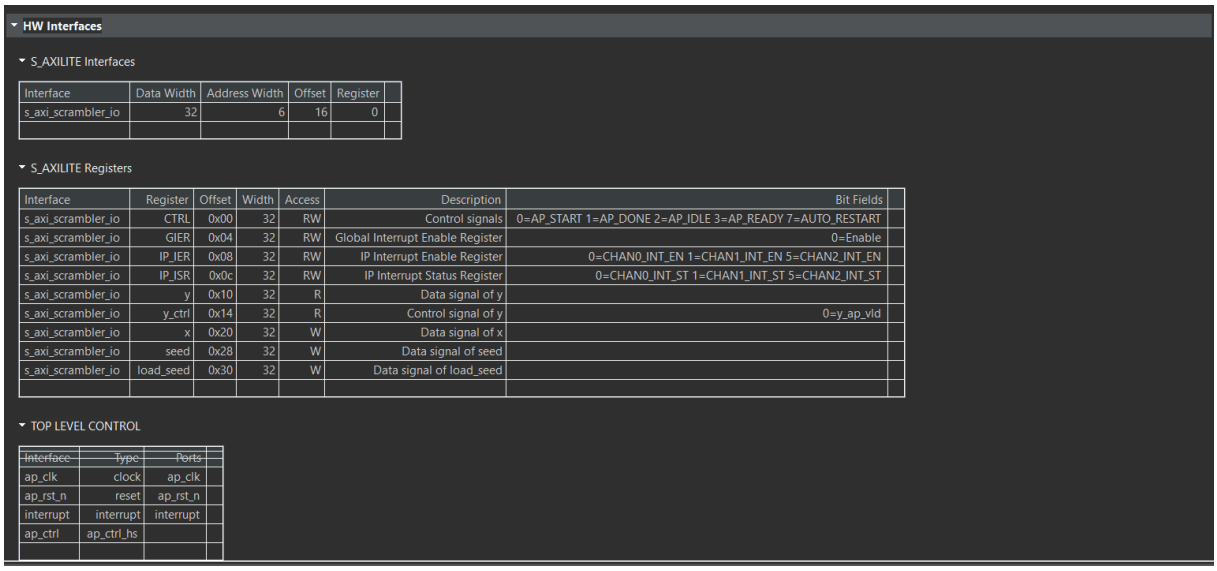


Figura 4.4: Report della mappatura SW-to-HW. Si nota come le singole porte C (x, y, seed) siano state consolidate sotto l'interfaccia `s_axi_scrambler_io` con specifici offset di memoria.

Come previsto dall'utilizzo del `bundle=scrambler_io`, tutte le porte sono state raggruppate sotto un'unica interfaccia *slave* (`s_axi_scrambler_io`), a cui il tool ha assegnato specifici offset esadecimali. La Figura 4.4 e il dettaglio dei registri hardware (Figura 4.5) confermano il seguente layout di memoria:

- `0x00`: Registro di controllo (**CTRL**) contenente i segnali *ap_start*, *ap_done*, *ap_idle* e *ap_ready* generati dalla direttiva sulla porta `return`.
- `0x10`: Registro di lettura per il campione criptato in uscita (`y`).
- `0x20`: Registro di scrittura per il campione audio in ingresso (`x`).
- `0x28`: Registro di scrittura per il parametro `seed`.
- `0x30`: Registro di scrittura per il segnale di controllo `load_seed`.



HW Interfaces

S_AXILITE Interfaces

Interface	Data Width	Address Width	Offset	Register
s_axi_scrambler_io	32	6	16	0

S_AXILITE Registers

Interface	Register	Offset	Width	Access	Description	Bit Fields
s_axi_scrambler_io	CTRL	0x00	32	RW	Control signals	0=AP_START 1=AP_DONE 2=AP_IDLE 3=AP_READY 7=AUTO_RESTART
s_axi_scrambler_io	GIER	0x04	32	RW	Global Interrupt Enable Register	0=Enable
s_axi_scrambler_io	IP_IER	0x08	32	RW	IP Interrupt Enable Register	0=CHAN0_INT_EN 1=CHAN1_INT_EN 5=CHAN2_INT_EN
s_axi_scrambler_io	IP_ISR	0x0c	32	RW	IP Interrupt Status Register	0=CHAN0_INT_ST 1=CHAN1_INT_ST 5=CHAN2_INT_ST
s_axi_scrambler_io	y	0x10	32	R	Data signal of y	
s_axi_scrambler_io	y_ctrl	0x14	32	R	Control signal of y	0=y_ap_vld
s_axi_scrambler_io	x	0x20	32	W	Data signal of x	
s_axi_scrambler_io	seed	0x28	32	W	Data signal of seed	
s_axi_scrambler_io	load_seed	0x30	32	W	Data signal of load_seed	

TOP LEVEL CONTROL

Interface	Type	Ports
ap_clk	clock	ap_clk
ap_rst_n	reset	ap_rst_n
interrupt	interrupt	interrupt
ap_ctrl	ap_ctrl_hs	

Figura 4.5: Dettaglio dei registri S_AXILITE generati. Il registro di controllo (offset `0x00`) espone ai bit 0, 1 e 2 i segnali di handshaking del blocco.

Questa mappa di indirizzamento costituisce il "contratto" definitivo tra l'hardware e il software: gli offset qui definiti verranno trascritti nei driver in linguaggio C (in ambiente Vitis) per permettere al processore ARM di accedere ai registri dell'acceleratore instanziato nella Programmable Logic.

4.7.4 Occupazione delle Risorse (Resource Utilization)

Infine, il report quantifica il costo dell'IP in termini di area fisica occupata sulla FPGA (come visibile precedentemente in Figura 4.3). Il sintetizzatore ha allocato un totale di **133 Flip-Flop (FF)** e **233 Look-Up Tables (LUT)**. L'utilizzo di blocchi DSP (*Digital Signal Processors*) e memoria RAM interna (BRAM) è nullo.

Questo risultato riflette perfettamente la natura crittografica dell'algoritmo: lo *Stream Cipher* basato su LFSR di Galois richiede esclusivamente logica posizionale di base (shift register per la memoria di stato) e logica combinatoria elementare (porte XOR per i *tap* polinomiali e il mascheramento finale). L'assenza di moltiplicazioni matematiche complesse esclude l'uso di DSP, rendendo l'IP Core Scrambler estremamente compatto. Tale leggerezza architetturale ne garantisce un impatto trascurabile sull'area totale della scheda Zynq-7000, lasciando ampio margine per l'inserimento di eventuali funzionalità aggiuntive (es. filtri FIR) in ottica di *Hardware/Software Co-Design*.

4.8 Validazione Hardware (C/RTL Co-simulation)

Successivamente alla generazione dell'architettura hardware, è imperativo verificare che il codice RTL (nel nostro caso, Verilog) prodotto dal sintetizzatore si comporti in modo funzionalmente identico al modello software C originale. Questa fase prende il nome di **C/RTL Co-simulation**.

Durante questo processo, Vitis HLS compila il *testbench* originario e lo interfaccia automaticamente con il simulatore hardware (Vivado XSIM). Gli stimoli generati dal codice C vengono tradotti in transazioni fisiche sui pin dell'interfaccia AXI4-Lite, elaborati dal modulo Verilog, e infine i risultati vengono estratti e confrontati con l'output software atteso.

4.8.1 Verifica Funzionale

L'esecuzione della co-simulazione ha riportato un esito completamente positivo. Come si evince dal log di esecuzione, il simulatore hardware ha processato correttamente le 22 transazioni generate dal testbench (corrispondenti alle fasi di inizializzazione e al processamento dell'array di campioni).

```
--- SIMULAZIONE SCRAMBLER ---
Inizializzazione TX con Seed...
Inizializzazione RX con stesso Seed...
In:     0 | Criptato:   1165 | Decriptato:     0
In:   100 | Criptato: -18910 | Decriptato:   100
[... output omesso per brevita' ...]

--- VERIFICA PERIODO MASSIMO LFSR ---
L'LFSR si e' ripetuto dopo 65535 colpi di clock.
SUCCESSO! L'LFSR ha lunghezza massima (2^16 - 1).

INFO: [COSIM 212-1000] *** C/RTL co-simulation finished: PASS ***
```

Listing 4.3: Estratto finale del log di C/RTL Co-simulation.

Il raggiungimento dello stato `PASS` certifica formalmente che la logica combinatoria e i registri sequenziali generati per l'LFSR e l'operatore XOR riproducono un mascheramento audio bit-perfect rispetto all'algoritmo matematico.

4.8.2 Analisi delle Prestazioni Post-Sintesi

Oltre alla validazione funzionale, la co-simulazione fornisce una stima accurata delle tempistiche operative reali (*Performance Estimates*), misurate direttamente sulle forme d'onda del simulatore. I risultati sono riportati in Figura 4.6.

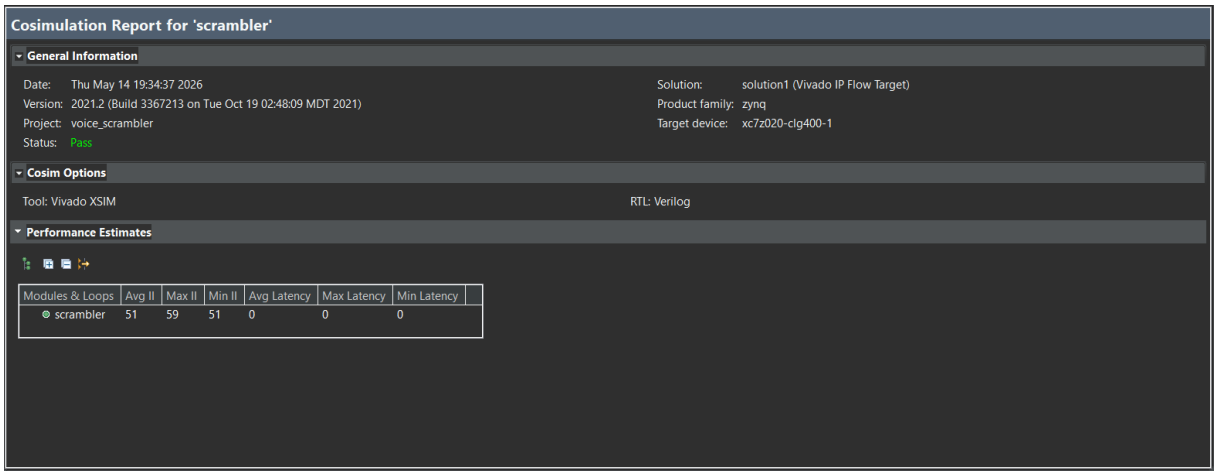


Figura 4.6: Report di Cosimulazione. Si noti la Latenza confermata a 0 cicli e l'Initiation Interval medio salito a 51 cicli a causa dell'overhead del protocollo AXI.

Il confronto di queste stime con quelle ottenute nella precedente fase di sintesi C rivela dettagli architetturali di fondamentale importanza:

- **Latenza (Latency):** Il simulatore XSIM conferma una latenza minima, massima e media pari a 0 cicli. Questo dimostra definitivamente che il percorso dati crittografico (calcolo del nuovo stato *seed* e cifratura del campione x) è stato implementato con successo in pura logica combinatoria, attraversabile in una frazione del periodo di clock (10 ns).

- **Intervallo di Iniziazione (Initiation Interval - II):** A differenza della sintesi C, che stimava un II pari a 1 ciclo, la co-simulazione riporta un intervallo reale compreso tra 51 e 59 cicli.

Questa discrepanza sull'*Initiation Interval* non rappresenta un'inefficienza o un'anomalia, ma riflette l'overhead reale introdotto dal protocollo di comunicazione **AXI4-Lite**. Durante la co-simulazione, per processare un singolo campione audio, il *testbench* deve eseguire molteplici transazioni sul bus:

1. Scrittura del parametro `load_seed`.
2. Scrittura del parametro `seed`.
3. Scrittura del campione in ingresso `x`.
4. Scrittura del bit *ap_start* nel registro di controllo (offset `0x00`).
5. Lettura (polling) del bit *ap_done* per verificare la fine del calcolo.
6. Lettura del campione criptato `y`.

Poiché il protocollo AXI4-Lite richiede diversi colpi di clock per completare l'*handshaking* di ogni singola operazione di lettura/scrittura (*Address Valid, Data Valid, Ready, Response*), l'accumulo di questi ritardi genera i circa 50 cicli osservati. Tale valore, corrispondente a circa 0.5 μs a 100 MHz, risulta comunque ordini di grandezza inferiore al vincolo di tempo reale dettato dalla frequenza di campionamento audio (48 kHz, pari a circa 20.8 μs per campione).

Il superamento della C/RTL Co-simulation conclude la fase di progettazione HLS: l'IP *Scrambler* è formalmente validato, pacchettizzato come IP Catalog e pronto per essere importato nel *Block Design* di Vivado.

4.9 Esportazione RTL e Generazione dell'IP Core

Il completamento con successo della co-simulazione C/RTL sancisce la fine della fase di progettazione e validazione in ambiente Vitis HLS. L'ultimo passaggio consiste nell'esportazione del design sotto forma di **IP Catalog**, rendendo l'acceleratore *Scrambler* un componente modulare pronto per l'integrazione nel sistema SoC tramite Vivado.

4.9.1 Configurazione dell'Esportazione

Seguendo le linee guida metodologiche del corso, la procedura di esportazione è stata configurata tramite il comando *Export RTL* selezionando il formato **Vivado IP (.zip)**. Questa scelta è fondamentale per garantire la massima compatibilità con il flusso di lavoro di *Hardware Development* che seguirà.

4.9.2 Finalizzazione del Flusso HLS

Al termine della procedura, il tool genera una cartella compressa contenente non solo il codice sorgente Verilog/VHDL, ma anche i file di vincolo, i driver software per il processore ARM (generati automaticamente in base alla mappa dei registri analizzata precedentemente) e la documentazione tecnica dell'interfaccia.

Con l'esportazione dell'IP, si conclude il ciclo di *High-Level Synthesis*. Il blocco risultante è ora caratterizzato da:

1. Una latenza deterministica e validata.
2. Un'interfaccia bus AXI4-Lite standard per il controllo software.

3. Un'occupazione di risorse ottimizzata per la piattaforma Zynq-7000.

Il capitolo successivo descriverà l'integrazione di questo componente all'interno dell'architettura hardware complessiva tramite Vivado.

CAPITOLO 5

Integrazione del Sistema Hardware (Vivado)

Conclusa la sintesi e la pacchettizzazione dell'IP Core crittografico in Vitis HLS, la fase successiva del flusso di *Hardware/Software Co-Design* prevede l'assemblaggio dell'intero sistema System-on-Chip (SoC). Per questa operazione è stato utilizzato l'ambiente **Xilinx Vivado**, il quale permette di integrare il processore ARM (PS) con le periferiche custom e standard all'interno della logica programmabile (PL), definendo le interconnessioni, le reti di clock e la mappatura in memoria.

5.1 Configurazione dell'Ambiente e Importazione degli IP

Il primo passo nella creazione del *Block Design* in Vivado è la configurazione dei *Repository* personalizzati. Oltre alla libreria standard fornita da Xilinx, sono state aggiunte due librerie fondamentali:

1. **Repository HLS:** La cartella contenente l'archivio `.zip` esportato da Vitis HLS al termine del Capitolo 4. Questo permette a Vivado di riconoscere l'IP *Scrambler* e renderlo disponibile nel catalogo dei componenti.
2. **Repository Zybo Audio Codec:** Un set di IP forniti dal produttore della scheda Zybo Z7, essenziale per importare il blocco `zybo_audio_ctrl`, ovvero il controller I2S necessario per interfacciarsi con il codec hardware SSM2603.

5.2 Configurazione del Processing System (Zynq-7000)

Il cuore del sistema è rappresentato dal blocco **ZYNQ7 Processing System**. Questo IP istanzia il processore ARM Cortex-A9 dual-core e gestisce le interfacce verso il mondo esterno. La sua configurazione ha richiesto un'attenta taratura per supportare i requisiti audio e crittografici del progetto:

- **Periferiche di I/O:** All'interno della scheda *Peripheral I/O Pins*, è stata abilitata l'interfaccia **UART 1** per la comunicazione seriale e il *debugging* software tramite terminale. Parallelamente, è stata attivata l'interfaccia **I2C 1**, che riveste un ruolo fondamentale per il funzionamento del sistema audio. Mentre il protocollo I2S si occupa del trasporto dei campioni audio digitali, il bus I2C agisce come interfaccia di controllo (*Control Interface*) necessaria per programmare i registri interni del codec SSM2603. Senza questa configurazione, eseguita dal processore ARM all'avvio, il codec rimarrebbe in uno stato di *standby* o con volumi non configurati, impedendo fisicamente la riproduzione o l'acquisizione del segnale.
- **Rete di Clock (Clock Configuration):** Questa è una delle configurazioni più critiche dell'intero sistema. Dalla sezione *PL Fabric Clocks* sono stati abilitati due segnali di temporizzazione distinti:

- `FCLK_CLK0 (100 MHz)`: Rappresenta il clock di sistema principale. Pilota il bus AXI4-Lite, l'interconnect e la logica interna di tutti gli IP instanziati (inclusi gli scrambler).
- `FCLK_CLK1 (12.288 MHz)`: Rappresenta il Master Clock (*MCLK*) dedicato esclusivamente al codec audio SSM2603. Questa frequenza specifica è matematicamente necessaria per derivare un campionamento esatto a 48 kHz.

- **Interrupts:** Per permettere agli IP custom di segnalare al processore il completamento delle operazioni (transizione da *Busy* a *Done*), è stata abilitata la porta `IRQ_F2P` (Fabric-to-Processing System Interrupts).

5.3 Integrazione degli IP e Architettura del Block Design

Una volta configurato il processore, si è proceduto all'instanziazione e al cablaggio dei moduli nella Programmable Logic. L'architettura finale del sistema è illustrata nella Figura 5.1.

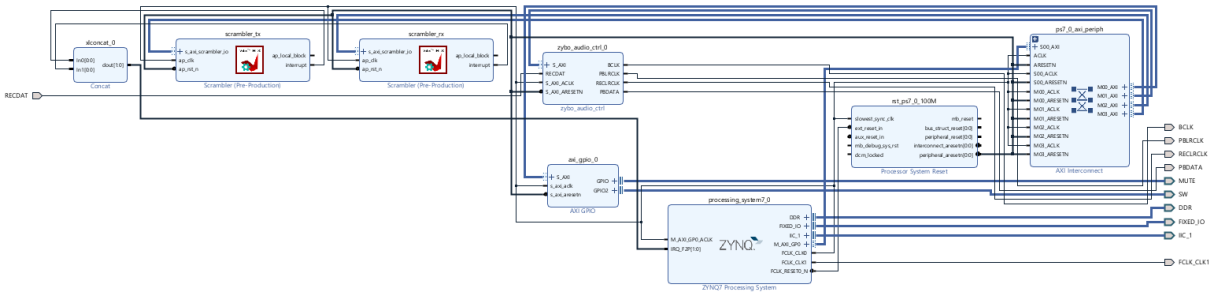


Figura 5.1: Schema architetturale completo (*Block Design*) generato in Vivado. Si noti la doppia istanza dello Scrambler (TX e RX), l'interconnessione AXI e la gestione centralizzata dei clock e degli interrupt.

5.3.1 Doppia Istanza Crittografica (TX e RX)

Come visibile nello schema a blocchi (Figura 5.1), una scelta architetturale peculiare di questo progetto è stata l'instanziazione di **due blocchi Scrambler identici**, rinominati rispettivamente `scrambler_tx` e `scrambler_rx`.

Questa ridondanza hardware è stata concepita per supportare una modalità di test "Loopback Crittografico" (o *Cascade*). In un'applicazione reale, un singolo SoC fungerebbe tipicamente solo da trasmettitore o solo da ricevitore. Avendo entrambi i blocchi a disposizione sulla medesima scheda, il processore può inviare il segnale audio pulito al blocco TX (ottenendo il rumore cifrato) e instradare immediatamente questo risultato al blocco RX per la decodifica locale. Ciò permette di testare e validare l'intera catena di elaborazione *end-to-end* senza necessitare di due piattaforme Zynq separate.

5.3.2 Interfacciamento Esterno: Audio Controller e AXI GPIO

Per consentire al sistema di interagire con i componenti hardware presenti sulla Zybo Z7, sono stati integrati i seguenti IP. Le porte di comunicazione di questi moduli sono state promosse a porte esterne (*Make External*), operazione necessaria per poterle successivamente vincolare ai pin fisici del package FPGA tramite il file dei vincoli XDC:

- **zybo_audio_ctrl:** Questo modulo funge da ponte tra il bus AXI e il protocollo seriale I2S. Le sue porte rese esterne — ovvero `BCLK` (Bit Clock), `PBLRCLK` e `RECLRCLK` (Word

Select per riproduzione e registrazione), `RECDAT` (input dati) e `PBDATA` (output dati) — permettono il trasporto fisico dei campioni audio digitali da e verso il codec SSM2603.

- **AXI GPIO:** Il blocco gestisce l'interazione con l'utente e il controllo dello stato del codec. È stato configurato con due interfacce esterne distinte:
  - `SW_tri_i`: Un bus di ingresso a 2 bit collegato agli switch a scorrimento (`SW0` e `SW1`), utilizzato dal software per selezionare la modalità operativa (es. bypass o scrambling).
  - `MUTE_tri_o`: Un segnale di uscita a 1 bit collegato al pin di *Mute* del codec audio per silenziarne l'uscita in caso di necessità.
- **Segnali di sistema esterni (Zynq PS):** Oltre agli IP dedicati, sono state rese esterne due interfacce critiche provenienti direttamente dal blocco Processing System:
  - `IIC_1`: I segnali `SCL` e `SDA` del bus I2C necessari per la configurazione dei registri interni del codec.
  - `FCLK_CLK1`: Il segnale di clock generato dal PS e promosso a porta esterna come *Master Clock* (`MCLK`), fondamentale per sincronizzare i convertitori del chip audio alla frequenza di 12.288 MHz.

5.3.3 Interconnessione, Memoria e Interrupts

La coesione tra il mondo software (PS) e i quattro blocchi hardware (`zybo_audio_ctrl`, GPIO, `scrambler_tx`, `scrambler_rx`) è garantita dall'**AXI Interconnect** (`ps7_0_axi_periph`). L'accesso a questo bus di interconnessione da parte del processore ARM avviene attraverso la porta master nativa **M_AXI_GP0** (General Purpose 0).

La scelta di instradare l'intero traffico dati attraverso l'interfaccia General Purpose, anziché sfruttare le porte High-Performance (HP) con accesso diretto in memoria (DMA), deriva da un'attenta analisi di banda. Le porte GP a 32 bit dello Zynq sono specificamente ottimizzate per trasferimenti a bassa latenza e bassa larghezza di banda tramite la tecnica del *Memory-Mapped I/O* (MMIO). Considerando che il sistema processa singoli campioni audio a 48 kHz (generando un *throughput* complessivo inferiore ai 2 Mbps, irrisorio rispetto ai limiti architetturali del SoC), la porta GP0 risulta perfettamente dimensionata. Essa gestisce agilmente sia la configurazione asincrona dei registri di controllo sia lo stream audio in tempo reale, evitando l'eccessiva complessità circuitale e lo spreco di risorse fisiche intrinsecamente legati all'impiego dei canali HP.

Tramite la funzione di *Connection Automation*, Vivado ha generato automaticamente la rete di decodifica degli indirizzi, visibile in dettaglio nel pannello **Address Editor** (Figura 5.2).

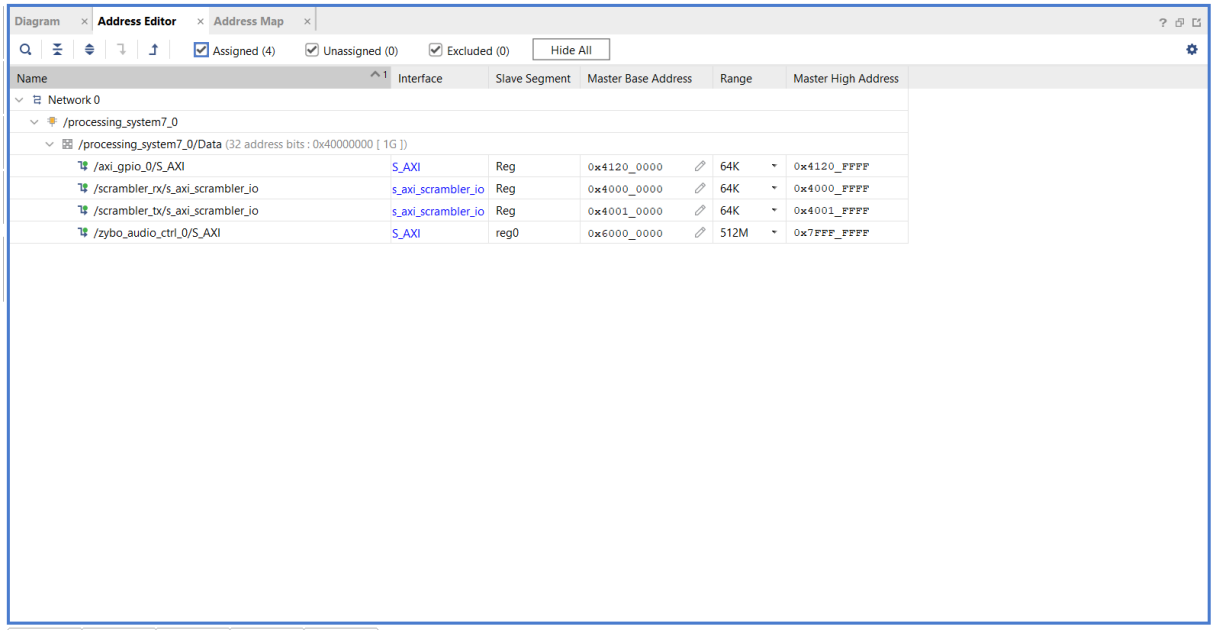


Figura 5.2: Pannello Address Editor di Vivado. Viene mostrata la mappa di memoria assegnata automaticamente alle periferiche AXI instanziate nella Programmable Logic.

Come si evince dall'analisi della mappa di memoria, a ciascun IP *slave* è stato assegnato uno spazio di indirizzamento dedicato (Base Address) all'interno dell'architettura a 32-bit dell'ARM Cortex-A9:

- **`scrambler_rx`:** allocato a partire dall'indirizzo `0x4000_0000` (con un range di 64K).
- **`scrambler_tx`:** allocato a partire dall'indirizzo `0x4001_0000` (range 64K).
- **`axi_gpio_0`:** allocato all'indirizzo `0x4120_0000` (range 64K).
- **`zybo_audio_ctrl_0`:** allocato all'indirizzo `0x6000_0000` (con un range più ampio di 512M).

L'analisi della mappa di memoria permette di evidenziare un concetto fondamentale dell'architettura SoC: il processore ARM "vede" ogni periferica AXI (come il GPIO o gli Scrambler) come se fosse una normale locazione di memoria RAM.

Prendendo come riferimento il blocco `axi_gpio_0`, si nota l'assegnazione di un **Base Address** pari a `0x4120_0000` con un **Range** di 64K. Matematicamente, un range di 64K corrisponde a 2^{16} byte; ciò implica che sono necessari esattamente 16 bit per indirizzare ogni singola locazione interna a quel modulo.

In termini di rappresentazione esadecimale, 16 bit corrispondono a 4 cifre. Pertanto, vale la regola dell'allineamento per la quale, affinché l'indirizzamento sia valido, il *Base Address* deve obbligatoriamente terminare con un numero di zeri pari alle cifre necessarie per il range (in questo caso, quattro zeri). Vivado gestisce questo vincolo in automatico, calcolando l'indirizzo massimo (**High Address**) a `0x4120_FFFF`.

Lo stesso principio si osserva per i moduli `scrambler_rx` e `scrambler_tx`, allineati rispettivamente a `0x4000_0000` e `0x4001_0000`. Questa rigida struttura di allineamento garantisce che gli *Offset* definiti durante la sintesi HLS (come lo `0x20` per l'input `x` o lo `0x10` per l'output `y`) cadano sempre correttamente all'interno della finestra di memoria assegnata, permettendo al software di raggiungere i registri hardware senza ambiguità.

Infine, particolare attenzione è stata dedicata alla rete degli interrupt. Il blocco Zynq espone una singola porta vettoriale per le interruzioni in ingresso (`IRQ_F2P`). Avendo due blocchi scrambler che generano interrupt indipendenti al completamento del cifrario (*ap_done*), è stato instanziato un blocco **Concat** (`xlconcat_0`). Questo IP concatena i due segnali scalari per

mapparli nei primi due bit di questo vettore indirizzato al *Generic Interrupt Controller* (GIC) dell'ARM, garantendo al sistema operativo o al codice *bare-metal* la capacità di distinguere quale dei due blocchi crittografici ha terminato la propria elaborazione.

5.4 Simulazione a Livello di Sistema (Zynq VIP)

Una volta ultimato il *Block Design* e interconnessi i vari IP, è fondamentale validare il funzionamento dell'intero sistema prima di procedere all'implementazione fisica. Tuttavia, simulare un intero System-on-Chip è un'operazione estremamente complessa: in un simulatore RTL tradizionale (come Vivado XSIM) non è possibile eseguire direttamente il codice software C/Assembly all'interno di un modello Verilog di un processore ARM Cortex-A9 dual-core per testare le periferiche.

Per risolvere questo limite strutturale, Xilinx mette a disposizione le **Verification IP (VIP)**. Le VIP sono routine e funzioni avanzate scritte in SystemVerilog che sostituiscono il processore fisico e ne emulano il comportamento sui bus di sistema. Esse permettono al progettista di generare transazioni AXI4 (letture e scritture in memoria) tramite semplici macro, verificando la corretta reazione delle periferiche custom (gli Scrambler) e del bus di interconnessione senza dover scrivere e compilare alcun software *bare-metal*.

5.4.1 Sviluppo del Testbench e Fasi Operative

Prima di poter impartire i comandi di lettura e scrittura tramite la VIP, è strettamente necessario conoscere la mappa dei registri (*Memory Map*) dell'IP Core instanziato. Tali informazioni vengono fornite esplicitamente dal file *header* `xscrambler_hw.h`, generato automaticamente da Vitis HLS durante la sintesi RTL (come discusso nella mappatura SW-to-HW del Capitolo 4) e incluso nel pacchetto IP esportato.

Di seguito è riportato un estratto di tale file, dal quale si evince la perfetta corrispondenza tra le variabili originarie del codice C e gli offset fisici sul bus AXI4-Lite:

```
1  // ==========================================================
2  // scrambler_io
3  // 0x00 : Control signals
4  //        bit 0  - ap_start (Read/Write/COH)
5  //        bit 1  - ap_done (Read/COR)
6  //        bit 2  - ap_idle (Read)
7  //        bit 3  - ap_ready (Read/COR)
8  //        bit 7  - auto_restart (Read/Write)
9  // 0x10 : Data signal of y
10 //        bit 15~0 - y[15:0] (Read)
11 // 0x20 : Data signal of x
12 //        bit 15~0 - x[15:0] (Read/Write)
13 // 0x28 : Data signal of seed
14 //        bit 15~0 - seed[15:0] (Read/Write)
15 // 0x30 : Data signal of load_seed
16 //        bit 0  - load_seed[0] (Read/Write)
17 // (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH =
       Clear on Handshake)
18
19 #define XSCRAMBLER_SCRAMBLER_IO_ADDR_AP_CTRL        0x00
20 #define XSCRAMBLER_SCRAMBLER_IO_ADDR_Y_DATA         0x10
21 #define XSCRAMBLER_SCRAMBLER_IO_ADDR_X_DATA         0x20
22 #define XSCRAMBLER_SCRAMBLER_IO_ADDR_SEED_DATA      0x28
23 #define XSCRAMBLER_SCRAMBLER_IO_ADDR_LOAD_SEED_DATA 0x30
```

Listing 5.1: Estratto del file `xscrambler_hw.h` generato da Vitis HLS, contenente la mappa dei registri AXI4-Lite dell'IP Scrambler.

Sommando questi offset (`0x28`, `0x30`, `0x20`, ecc.) all'indirizzo base assegnato all'IP `scrambler_tx` nell'Address Editor di Vivado (`0x4001_0000`), è stato possibile costruire gli indirizzi fisici assoluti per le transazioni del testbench.

Avendo mappato i registri, la simulazione è stata divisa in due fasi operative distinte che ricalcano le operazioni necessarie per il funzionamento dello *stream cipher*:

1. **Fase 1: Inizializzazione del Seed.** Viene testato il caricamento della chiave crittografica. Il testbench esegue scritture AXI per impostare il *Seed* (`0x1234` all'offset `0x28`) e asserire il segnale di controllo `load_seed` (`1` all'offset `0x30`). Successivamente, impartisce il primo `ap_start` (offset `0x00`). Tramite un ciclo `while`, la VIP esegue un *polling* continuo (lettura) sul registro di controllo finché l'IP non alza il bit `ap_done`, segnalando l'avvenuto aggiornamento dello stato interno. Prima di passare alla fase successiva, il `load_seed` viene de-asserito per evitare che il registro venga resettato al ciclo successivo.

2. **Fase 2: Elaborazione del Campione.** Viene testato il percorso dati (Data Path). Il testbench scrive un campione audio fittizio ($x = 500$, ovvero `0x01F4` all'offset `0x20`) e impartisce un secondo `ap_start`. Dopo un nuovo ciclo di *polling* per attendere la fine del calcolo, la VIP esegue un'operazione di lettura sul registro di uscita y (offset `0x10`) per prelevare il campione cifrato.

5.4.2 Analisi delle Transazioni AXI e del Risultato di Simulazione

Eseguendo il testbench, le transazioni generate dalla Zynq VIP attraversano l'intero sistema: nascono dalla porta master dell'ARM (`M_AXI_GP0`), attraversano l' *AXI Interconnect* e giungono infine all'interfaccia *slave* dell'IP (`s_axi_scrambler_io`). L'output prodotto dal simulatore nella TCL Console conferma il successo di ogni singolo trasferimento:

```
1  ================================================
2     INIZIO SIMULAZIONE SYSTEM-LEVEL (ZYNQ VIP)
3  ================================================
4  [TIME: 0 ns] Reset del Processing System...
5  [0] : *ZYNQ_VIP_INFO : FPGA Soft Reset called for 0x1
6  [200] : *ZYNQ_VIP_INFO : FPGA Soft Reset called for 0x0
7  [TIME: 2200 ns] FASE 1: Caricamento Seed (0x1234)...
8  [2200] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010028) ->
      AXI Write -> 4 bytes
9  [2435] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Write for Starting
      Address(0x40010028) with Response 'OKAY'
10 [2435] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010030) ->
      AXI Write -> 4 bytes
11 [2695] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Write for Starting
      Address(0x40010030) with Response 'OKAY'
12 [2695] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010000) ->
      AXI Write -> 4 bytes
13 [2955] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Write for Starting
      Address(0x40010000) with Response 'OKAY'
14 [2955] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010000) ->
      AXI Read -> 4 bytes
15 [3055] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Read for Starting Address
      (0x40010000) with Response 'OKAY'
16 [3065] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010030) ->
      AXI Write -> 4 bytes
17 [3195] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Write for Starting
      Address(0x40010030) with Response 'OKAY'
18 [TIME: 3195 ns] FASE 2: Invio campione audio (x = 500)...
19 [3195] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010020) ->
      AXI Write -> 4 bytes
```

```
[3315] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Write for Starting
   Address(0x40010020) with Response 'OKAY'
[3315] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010000) ->
   AXI Write -> 4 bytes
[3475] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Write for Starting
   Address(0x40010000) with Response 'OKAY'
[3475] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010000) ->
   AXI Read -> 4 bytes
[3575] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Read for Starting Address
   (0x40010000) with Response 'OKAY'
[3585] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Starting Address(0x40010010) ->
   AXI Read -> 4 bytes
[3675] : M_AXI_GP0 : *ZYNQ_VIP_INFO : Done AXI Read for Starting Address
   (0x40010010) with Response 'OKAY'
-----------------------------------------------------
   RISULTATO LETTO DAL BUS AXI: 0x00000579
-----------------------------------------------------
```

Listing 5.2: Output della TCL Console generato dalla Zynq VIP durante la simulazione.

L'analisi visiva delle forme d'onda generate dal simulatore conferma il rigoroso rispetto del protocollo AXI4-Lite sull'intera architettura.



Figura 5.3: Panoramica globale della simulazione System-Level. Si notano i due cicli di avvio dell'IP e le relative transazioni scambiate sul bus AXI.

Durante le operazioni di scrittura (come l'inizializzazione del *seed* o l'invio del campione audio), i trasferimenti avvengono unicamente quando i segnali di *handshaking* `VALID` (generato dal master) e `READY` (generato dallo slave) si incrociano a livello logico alto nello stesso colpo di clock sui rispettivi canali (`AW` per l'indirizzo e `W` per il dato), seguiti dalla conferma di ricezione (`BRESP = OKAY`) sul canale di risposta.

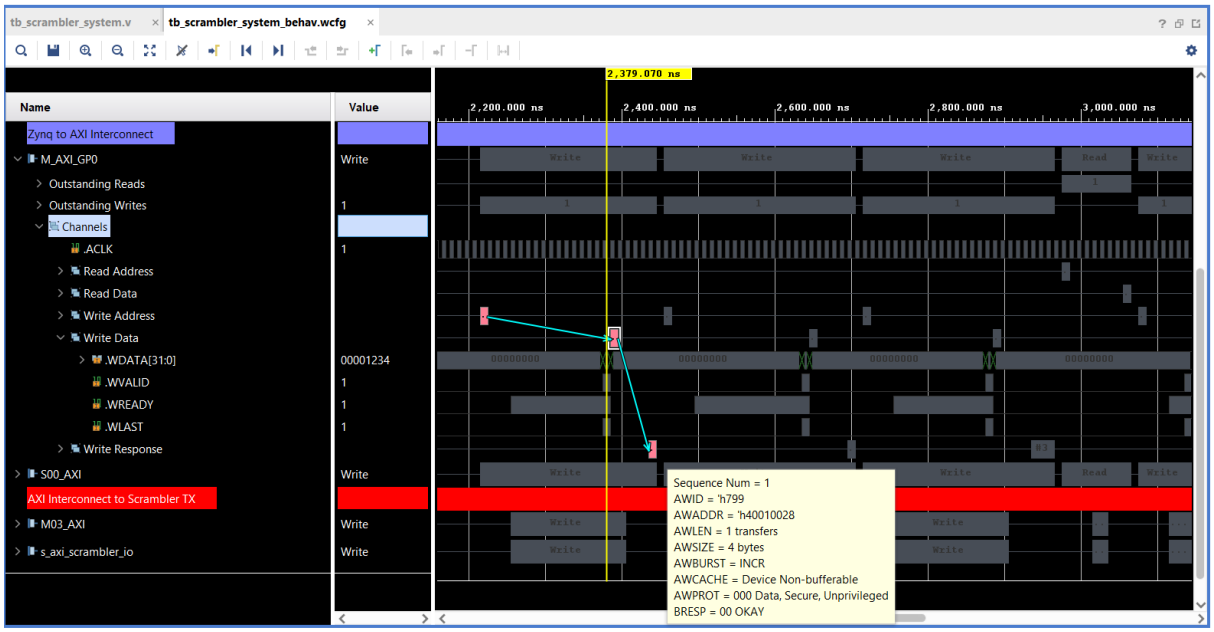


Figura 5.4: Scrittura del Seed (`0x1234` all'offset `0x28`).

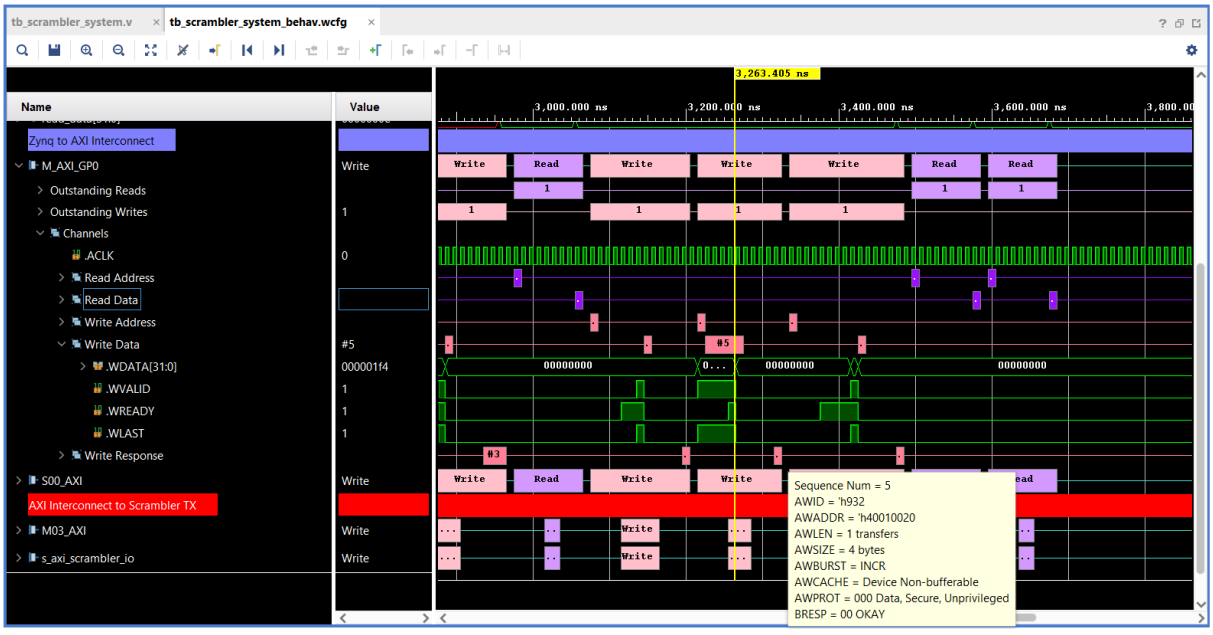


Figura 5.5: Scrittura del Campione (`0x01F4` all'offset `0x20`).

Un aspetto fondamentale della simulazione è l'analisi delle operazioni di *polling*. Come si evince dalla forma d'onda in Figura 5.6, durante il ciclo di lettura sul registro di controllo (offset `0x00`), il simulatore restituisce sul canale dati `R` il valore esadecimale `0x0000000E`.



Figura 5.6: Operazione di Polling: lettura dello stato `0x0E` dal registro di controllo. Si noti il completamento dell'handshake con `RVALID` e `RREADY` alti.

Convertendo tale valore in binario (considerando il *nibble* meno significativo: `1110`) e sovrapponendolo alla mappa dei registri HLS definita in precedenza, si ottiene la fotografia esatta dello stato della Macchina a Stati Finiti (FSM) dell'acceleratore a fine calcolo:

- **Bit 0 (`ap_start`) = 0:** Il comando di avvio è stato consumato dall'hardware al momento dello start (*Clear on Handshake*).
- **Bit 1 (`ap_done`) = 1:** L'elaborazione è terminata con successo e il dato è stabile nel registro di uscita.
- **Bit 2 (`ap_idle`) = 1:** Il blocco hardware è attualmente a riposo.
- **Bit 3 (`ap_ready`) = 1:** L'IP è pronto ad accettare un nuovo input per il ciclo successivo.

L'operazione *bitwise* (`read_data & 32'h00000002`) eseguita dal testbench serve proprio a mascherare tutti gli altri bit e isolare unicamente l'`ap_done`, sbloccando il ciclo `while` non appena questo si asserisce. Inoltre, è stata verificata l'efficacia del meccanismo **COR (Clear on Read)**: non appena la VIP esegue la lettura sul canale `R` e individua il bit `ap_done` a 1, l'hardware HLS azzera automaticamente tale bit al colpo di clock successivo, riarmando l'IP per una nuova elaborazione senza richiedere ulteriori scritture da parte del software.

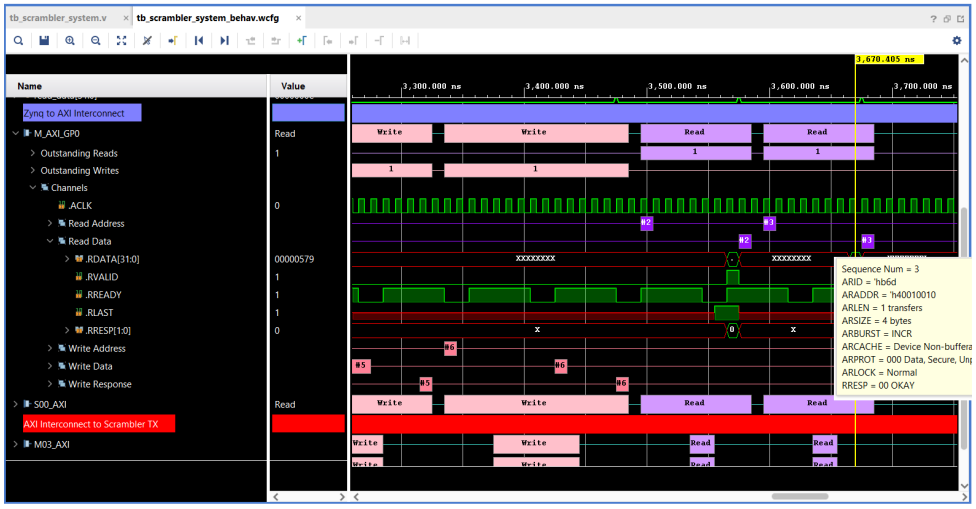


Figura 5.7: Lettura del risultato finale (`0x0579`) dal registro `y` all'offset `0x10`.

5.4.3 Validazione Matematica dell'Evoluzione di Stato

Il risultato stampato a video dalla simulazione, e letto fisicamente sul bus AXI come visibile in Figura 5.7 (`0x00000579`), nasconde una dinamica fondamentale dell'architettura HLS e fornisce la prova definitiva del corretto funzionamento della memoria di stato interna (*static variable*).

Come definito nel codice C, la funzione *scrambler* viene eseguita integralmente ad ogni asserzione di `ap_start`. Pertanto, i due `ap_start` inviati dal testbench hanno fatto avanzare l'LFSR di ben due passi temporali:

1. **Azione del Primo Start (Fase 1):** L'IP legge `load_seed = 1` e acquisisce il seed `0x1234`. Proseguendo nell'esecuzione, calcola l'LSB (0), esegue lo *shift* logico a destra (`0x1234 >> 1 = 0x091A`) e, non essendovi bit di *tap* da sommare, aggiorna il proprio stato interno definitivamente a `0x091A`.

2. **Azione del Secondo Start (Fase 2):** L'IP legge `load_seed = 0` (non acquisisce nulla) e preleva il campione audio `x = 500` (`0x01F4` in esadecimale). L'LFSR avanza partendo dal suo stato memorizzato (`0x091A`): calcola l'LSB (0), esegue lo *shift* a destra (`0x091A >> 1 = 0x048D`) e aggiorna lo stato interno a `0x048D`.

L'operazione finale di *masking* viene calcolata applicando l'operatore logico XOR tra il campione x e il nuovo stato dell'LFSR:

$$Y = 0\text{x}01F4 \oplus 0\text{x}048D$$

Eseguendo lo XOR bit-a-bit tra `0000 0001 1111 0100` e `0000 0100 1000 1101`, il risultato matematico è esattamente `0000 0101 0111 1001`, che corrisponde all'esadecimale `0x0579`.

Il dato letto dalla Zynq VIP coincide alla perfezione con il modello teorico. Questo convalida al 100% la robustezza dell'interfaccia hardware/software: l'IP risponde correttamente ai comandi AXI, conserva la cronologia del *seed* tra elaborazioni separate e maschera il dato in modo crittograficamente ineccepibile.

5.5 Vincoli Fisici (XDC) e Generazione del Bitstream

Ultimata la progettazione logica, l'ultimo step prima della sintesi è consistito nell'assegnazione dei vincoli fisici tramite il file *Xilinx Design Constraints* (`ZyboZ720_audio_constraints.xdc`, riportato integralmente nell'Appendice C).

Il file XDC è fondamentale in quanto mappa le porte logiche definite nel *Block Design* sui pin fisici (pad) del chip FPGA saldato sulla Zybo Z7, definendone contestualmente lo standard logico-elettrico (impostato globalmente a `LVCMOS33`). Le mappature implementate includono:

- **Segnali I2S e Clock:** I pin per i dati audio seriali (es. `BCLK` su `R19`, `RECDAT` su `R16`) e il clock principale `MCLK` (`FCLK_CLK1`) instradato sul pin `R17`.
- **Interfaccia I2C 1:** I segnali di configurazione `SCL` e `SDA` assegnati ai pin `N18` e `N17`.
- **Controllo (GPIO):** L'uscita per il `MUTE` (attivo basso) assegnata al pin `P18` e i due selettori hardware (`SW0` e `SW1`) attestati rispettivamente sui pin `G15` e `P15`.

Con i vincoli fisici formalizzati, è stato generato l'*HDL Wrapper* del sistema, trasformando lo schema grafico in un modulo Verilog *top-level*. A seguire, è stata avviata l'intera *toolchain* di implementazione hardware:

1. **Sintesi (Synthesis):** Mappatura delle porte logiche in primitive FPGA reali (LUT e Flip-Flop).
2. **Implementazione (Implementation):** Ottimizzazione, piazzamento (*Placement*) e cablaggio fisico (*Routing*) dei componenti all'interno del silicio.

3. **Generazione (Write Bitstream):** Compilazione del file `.bit` finale, ovvero il flusso di programmazione binario caricato sulla FPGA durante la fase di avvio.

5.6 Analisi dei Report di Implementazione (Post-Routing)

L'ultima fase del flusso di lavoro in Vivado, richiede una validazione accurata dei report generati dal tool. Affinché il sistema hardware possa essere considerato affidabile e pronto per il *deployment* sulla scheda fisica, è necessario analizzare tre metriche fondamentali: *Timing*, *Utilization* e *Power*.

5.6.1 Analisi Temporale (Timing Summary)

Il parametro più critico per il successo di un progetto digitale sincrono è il rispetto dei vincoli temporali. Questa analisi si concentra principalmente sulla verifica del **Worst Negative Slack (WNS)**.

Lo *Slack* (o margine temporale) rappresenta la differenza matematica tra il tempo richiesto per l'arrivo di un dato (imposto dal periodo del clock, che a 100 MHz corrisponde a 10 ns) e il tempo di arrivo effettivo del segnale dopo aver attraversato le porte logiche del percorso fisico. Se lo Slack assume un valore negativo (*Negative Slack*), il segnale giunge in ritardo al Flip-Flop di destinazione, causando un errore di campionamento e il fallimento del circuito. Al contrario, uno Slack positivo indica che il segnale arriva in anticipo e si stabilizza ben prima del fronte di salita del clock successivo. Il parametro WNS indica proprio il margine del percorso logico peggiore (più lento) dell'intero chip.

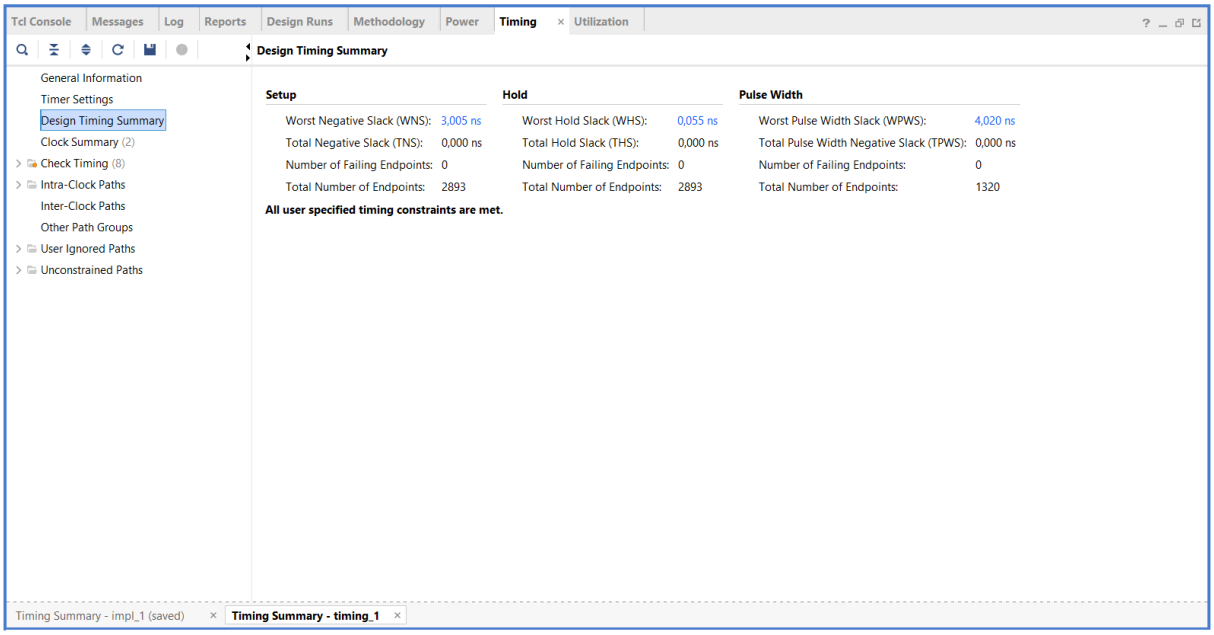


Figura 5.8: Design Timing Summary. Il parametro WNS positivo garantisce il perfetto soddisfacimento dei vincoli temporali (Setup e Hold).

Osservando il *Design Timing Summary* generato per il nostro sistema (Figura 5.8), si rileva un **WNS pari a 3.005 ns** (positivo) e un *Total Negative Slack (TNS)* pari a 0.000 ns. Vivado conferma l'esito con la dicitura esplicita: *"All user specified timing constraints are met."*.

Questo risultato certifica un'architettura estremamente robusta. Sapendo che il periodo di clock di sistema è di 10 ns, un margine di 3.005 ns sul percorso critico significa che l'onda dati impiega al massimo ≈ 6.995 ns per propagarsi attraverso le Slice della FPGA. Di conseguenza, il

design non solo regge perfettamente i 100 MHz previsti, ma teoricamente potrebbe essere spinto (tramite *overclocking*) fino a una frequenza massima di circa $\frac{1}{6.995\ \text{ns}} \approx 142.9$ MHz prima di manifestare metastabilità o errori logici.

5.6.2 Occupazione delle Risorse (Utilization)

Il report di *Utilization* quantifica il numero di primitive hardware fisiche consumate dal design all'interno della matrice programmabile della Zynq-7000.

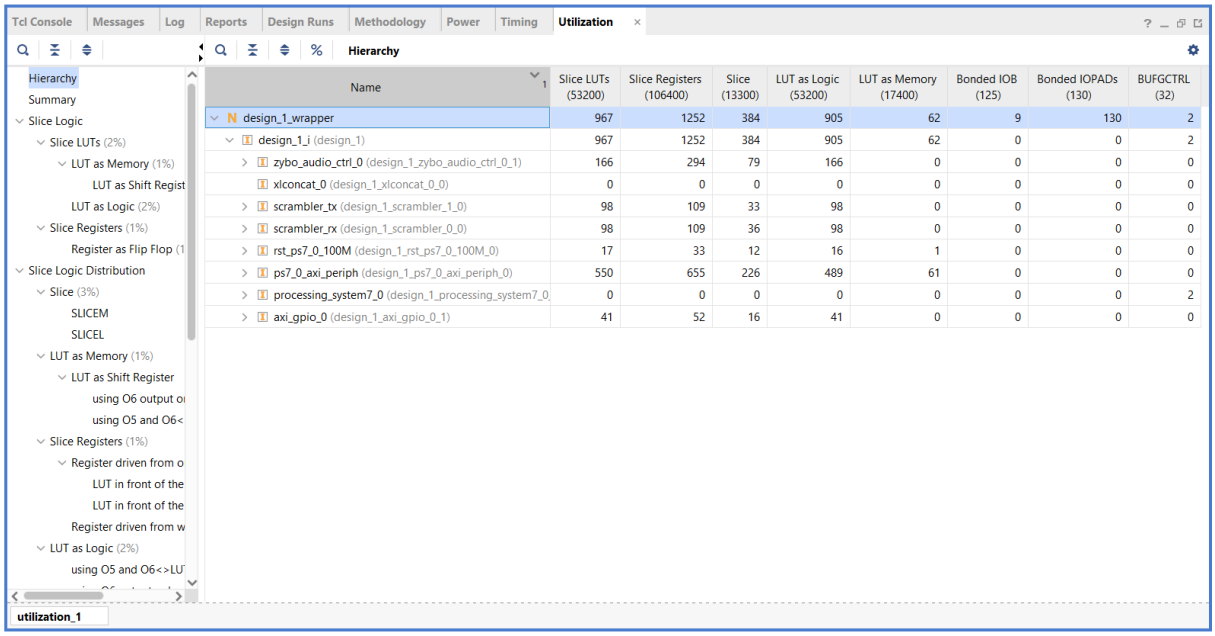


Figura 5.9: Report di Utilization. È visibile il contributo infinitesimale dei due IP crittografici rispetto all'intero *wrapper* di sistema.

Dalla gerarchia illustrata in Figura 5.9, si evince che l'intero *wrapper* di sistema occupa **967 Slice LUTs** e **1252 Slice Registers (Flip-Flop)**. A fronte delle risorse totali disponibili sulla Zybo Z7-20 (53.200 LUTs e 106.400 Registri), l'occupazione complessiva si attesta al di sotto del **2%**.

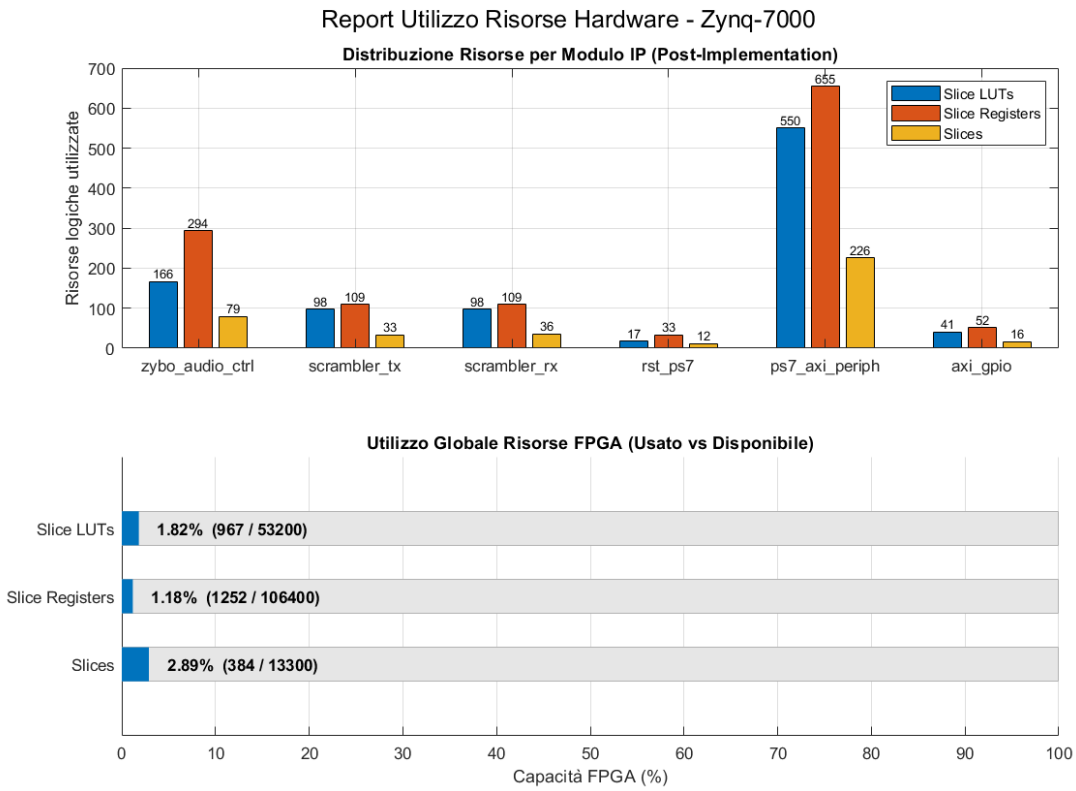


Figura 5.10: Rappresentazione grafica dell'occupazione delle risorse hardware post-implementazione. Il grafico superiore illustra la distribuzione del carico logico tra i singoli moduli IP istanziati. Il grafico inferiore evidenzia la percentuale di utilizzo globale rispetto alla capacità fisica totale della matrice FPGA Zynq-7000, confermando l'impatto trascurabile del design sull'area del chip.

Un'analisi dettagliata (Figura 5.11) rivela tuttavia un dato interessante: la maggior parte delle risorse non è consumata dai moduli crittografici, bensì dal componente `auto_pc_0` (Protocol Converter), che richiede da solo 398 LUT e 534 FF.

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (125)	Bonded IOPADs (130)	BUF
design_1_wrapper	968	1252	395	906	62	11	130	
design_1_i (design_1)	968	1252	395	906	62	0	0	
axi_gpio_0 (design_1_axi_gpio_0_1)	41	52	18	41	0	0	0	
processing_system7_0 (design_1_processing_system7_0_1)	2	0	2	2	0	0	0	
ps7_0_axi_periph (design_1_ps7_0_axi_periph_0)	549	655	231	488	61	0	0	
s00_couplers (s00_couplers_imp_UYSKKA)	398	534	175	337	61	0	0	
auto_pc (design_1_auto_pc_0)	398	534	175	337	61	0	0	
xbar (design_1_xbar_0)	151	121	63	151	0	0	0	
rst_ps7_0_100M (design_1_rst_ps7_0_100M_0)	17	33	10	16	1	0	0	
scrambler_rx (design_1_scrambler_0_0)	98	109	37	98	0	0	0	
scrambler_tx (design_1_scrambler_1_0)	98	109	38	98	0	0	0	
xlconcat_0 (design_1_xlconcat_0_0)	0	0	0	0	0	0	0	
zybo_audio_ctrl_0 (design_1_zybo_audio_ctrl_0_1)	166	294	81	166	0	0	0	

Figura 5.11: Dettaglio dell'utilizzo delle risorse. Il modulo `auto_pc_0` emerge come il componente a maggiore consumo logico nel sistema.

Il peso rilevante di questo blocco è giustificato dalla sua natura di "ponte" infrastrutturale: il Protocol Converter è inserito automaticamente da Vivado all'interno dell'*AXI Interconnect* per

garantire la compatibilità elettrica e logica tra le diverse periferiche. Nello specifico, il processore Zynq espone nativamente una porta master basata su protocollo AXI3, mentre il nostro IP custom (e il blocco AXI GPIO) comunicano tramite standard AXI4-Lite. Il Protocol Converter si sobbarca quindi un lavoro oneroso "dietro le quinte":

- **Traduzione di protocollo:** Gestisce la conversione delle specifiche tra AXI3 e AXI4-Lite.
- **Adattamento del bus:** Implementa le macchine a stati necessarie per garantire che gli handshake (*VALID/READY*) rispettino rigorosamente lo standard AMBA, convertendo le diverse larghezze di banda e i domini di clock ove necessario.

Analizzando invece i nostri moduli, si nota come le due istanze degli *Scrambler* (`scrambler_tx` e `scrambler_rx`) occupino appena 98 LUTs e 109 Registri ciascuno. Questa estrema leggerezza conferma le stime ottimistiche della sintesi HLS: la vera complessità del design è delegata al bus di sistema, mentre il core crittografico rimane un'implementazione snella ed efficiente, lasciando oltre il 98% del silicio libero per espansioni.

5.6.3 Stima dei Consumi (Power Estimation)

L'ultimo aspetto analizzato è la stima del consumo energetico del SoC, visibile nel pannello *Power* (Figura 5.12). Vivado stima una *Total On-Chip Power* di **1.542 W**, dominata per la quasi totalità dal processore ARM (*PS7*, con 1.394 W). Il contributo dinamico della logica programmabile (clocks, segnali e LUTs) incide per soli ~ 0.008 W.

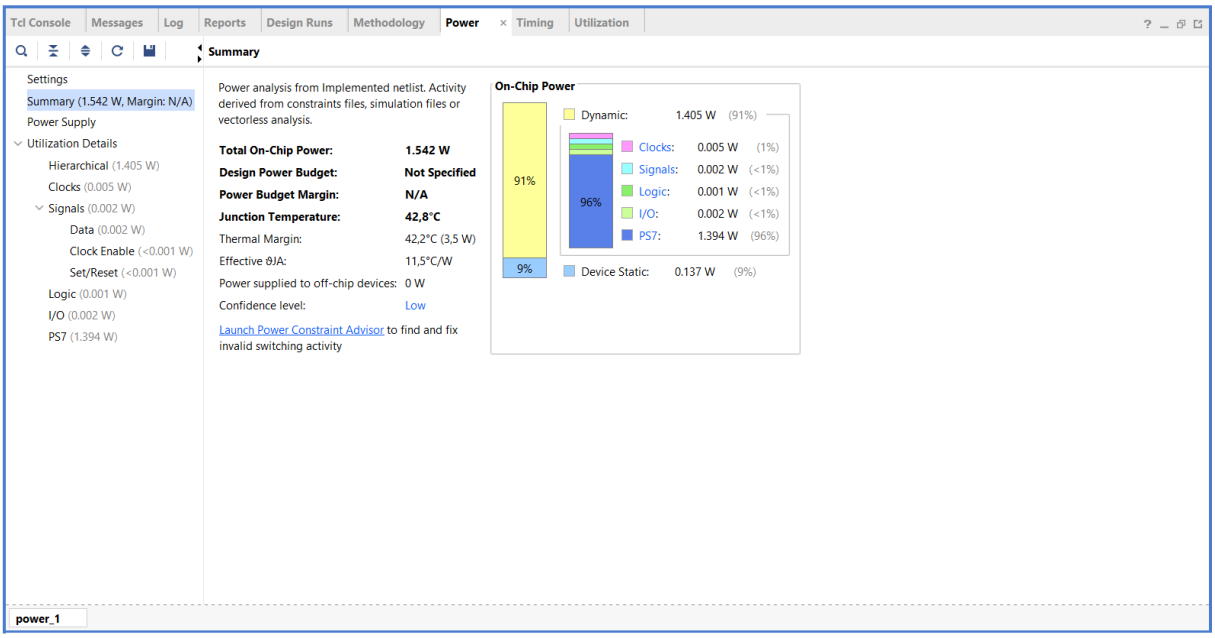


Figura 5.12: Report di Power Estimation. Si noti il *Confidence level* impostato su "Low", indice di una stima puramente statica e non basata su simulazioni vettoriali reali.

In sede accademica, è tuttavia d'obbligo analizzare queste metriche con profondo senso critico, invitando a diffidare enormemente delle stime di potenza mostrate in assenza di una reale simulazione post-routing. Il tool stesso segnala la problematica evidenziando un **Confidence level: Low** (livello di confidenza basso). Questo accade perché Vivado, non conoscendo il comportamento reale dell'audio in ingresso, sta utilizzando stime puramente statiche con una *Default toggle rate* (frequenza con cui i pin cambiano stato) ipotizzata al 12.5% e una probabilità statica dello 0.5 (ovvero assume che i segnali siano per il 50% del tempo a livello logico '0' e per il 50% a livello '1').

Senza fornire al sintetizzatore un file di annotazione della simulazione reale (formato `.saif` - *Switching Activity Interchange Format*) che registri l'effettiva *switching activity* dei segnali in base all'iniezione di un segnale audio realistico, queste metriche risultano puramente speculative. Ad ogni modo, il budget energetico calcolato conferma che la dissipazione termica stimata (Giunzione a 42.8°C) è ampiamente sostenibile dal package della scheda Zybo in condizioni operative normali senza richiedere sistemi di raffreddamento attivi.

CAPITOLO 6

Sviluppo Software e Validazione (Vitis)

Una volta conclusa l'implementazione dell'hardware e generato il *bitstream* in Vivado, il flusso di *Hardware/Software Co-Design* si sposta nell'ambiente **Xilinx Vitis**. In questa fase si sviluppa l'applicativo software (*bare-metal*) che verrà eseguito sul processore ARM Cortex-A9 per orchestrare le periferiche fisiche e gli acceleratori crittografici precedentemente istanziati nella logica programmabile (PL).

6.1 Configurazione dell'Ambiente e Board Support Package (BSP)

Il punto di congiunzione tra l'hardware (Vivado) e il software (Vitis) è rappresentato dall'archivio Xilinx Support Archive (file `.xsa`). In ambiente Vitis, il primo step operativo consiste nella creazione esplicita di un **Platform Project**, fornendo in input come base hardware (*Hardware Specification*) proprio il file `.xsa` esportato. Analizzando la topologia del sistema definita in Vivado, Vitis compila le librerie necessarie e genera automaticamente il **Board Support Package (BSP)**.

Una volta configurata e compilata la piattaforma hardware, il flusso di lavoro richiede la creazione di un **Application Project**. Questo secondo progetto viene impostato per poggiare esplicitamente le sue fondamenta sul *Platform Project* appena creato, fungendo da contenitore per il codice sorgente in linguaggio C dell'applicativo *bare-metal* (riportato integralmente in Appendice D).

Come illustrato in Figura 6.1, il BSP funge da strato di astrazione hardware (HAL) vitale per l'Application Project. Analizzando il file `.xsa`, il tool deduce quali componenti sono presenti nel *Block Design* e popola il BSP con i relativi driver.

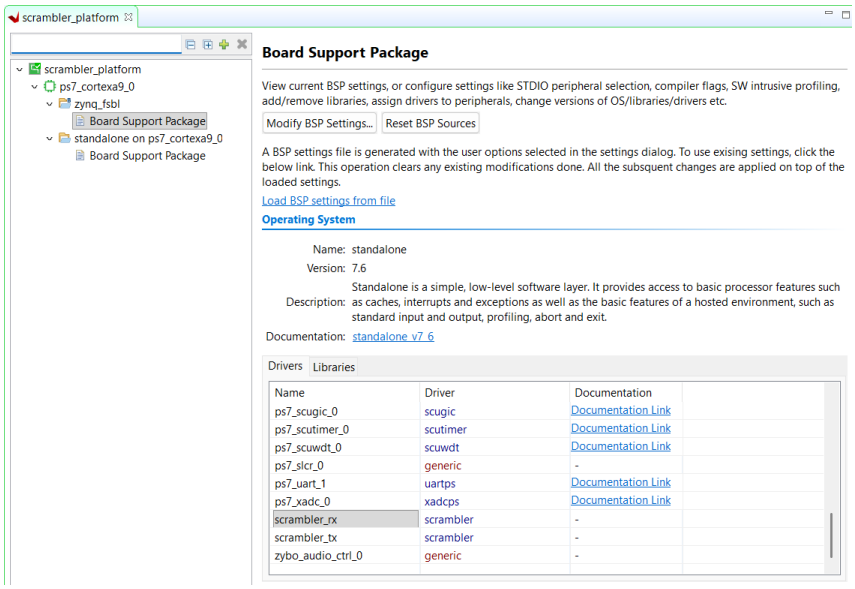


Figura 6.1: Board Support Package in Vitis. Si noti come il tool abbia generato automaticamente i driver `scrambler` per i due IP custom `scrambler_rx` e `scrambler_tx` istanziati nel Platform Project.

Oltre ai driver standard generati per le periferiche del *Processing System* (elencati nel BSP come `uartps` per la comunicazione seriale, `iicps` per il bus I2C e `scugic` per il controller degli interrupt), il BSP compie due operazioni fondamentali per il nostro progetto:

1. **Generazione di `xparameters.h`:** Questo header file mappa i *Base Address* decisi nell'Address Editor di Vivado in macro C. Ad esempio, l'indirizzo del nostro TX (`0x40010000`) diventa `XPAR_XSCRAMBLER_1_DEVICE_ID`, permettendo al programmatore di slegarsi dagli indirizzi numerici *hardcoded*.

2. **Driver Custom HLS:** Avendo esportato gli Scrambler da Vitis HLS, il BSP genera automaticamente la libreria `xscrambler.h`. Questa libreria ci fornisce funzioni C di altissimo livello (come `XScrambler_Start()` o `XScrambler_Set_seed()`) che mascherano la complessità delle transazioni AXI4-Lite sottostanti.

In particolare, l'ambiente Vitis è in grado di generare funzioni software specifiche come `XScrambler_Set_seed()`, seguendo un flusso deterministico di generazione e passaggio di metadati lungo l'intera *toolchain*:

1. **Fase HLS (Definizione del Layout Hardware):** Tutto ha inizio nel codice sorgente hardware (Capitolo 4) tramite la direttiva `#pragma HLS INTERFACE s_axilite port=seed bundle=scrambler_io`. Questo comando istruisce il sintetizzatore sul fatto che la variabile `seed` deve essere associata a un registro fisico mappato in memoria sul bus AXI4-Lite.

2. **Generazione del Driver Template:** Durante la sintesi, Vitis HLS esegue uno script interno che analizza l'albero sintattico del codice. Per ogni porta mappata in formato *slave AXI*, lo script genera automaticamente i file sorgente C (`xscrambler.c`) e gli header di mappatura degli offset (`xscrambler_hw.h`). I nomi delle funzioni vengono generati per concatenazione semantica: il prefisso dell'IP Core (`XScrambler_`) viene unito all'azione di controllo (`Set_` o `Get_`) e al nome esatto della variabile C hardware (`seed`), producendo la firma della funzione `XScrambler_Set_seed()`.

3. **Incapsulamento in IP-XACT e Transito in Vivado:** Questi file driver e le informazioni sui registri vengono salvati all'interno dell'IP Core nel formato standardizzato **IP-XACT** (tramite il file descrittore `component.xml`). Quando l'hardware viene assemblato in Vivado e successivamente esportato tramite l'archivio `.xsa`, questo file di metadati viaggia all'interno del pacchetto, portando con sé la conoscenza dell'architettura interna dei registri.

4. **Estrazione e Legame nel BSP (Vitis):** All'atto della creazione del *Platform Project* in Vitis, l'IDE scansiona l'archivio `.xsa`. Rilevata la presenza dell'IP Scrambler, il tool ne estrae i driver C pre-generati da HLS e li copia fisicamente nelle directory del Board Support Package. Contemporaneamente, Vitis legge il *Base Address* assegnato da Vivado (es. `0x4001_0000`) e lo inietta all'interno del file `xparameters.h`.

Questo meccanismo di cooperazione hardware/software garantisce che le API messe a disposizione dello sviluppatore nell'Application Project siano sempre perfettamente allineate con l'architettura dei registri fisici implementati sul silicio della FPGA, azzerando il rischio di disallineamento degli indirizzi durante l'accesso MMIO. Basandosi su questa piattaforma, è stato infine creato l'**Application Project** contenente il codice sorgente dell'applicativo (riportato integralmente in Appendice D).

6.2 Architettura del Software e Inizializzazione

L'esecuzione del programma ha inizio nella funzione `main()`, la quale è strutturata in diverse fasi sequenziali di inizializzazione prima di cedere il controllo al loop audio continuo.

6.2.1 Setup delle Periferiche e del Codec Audio

La prima fase prevede l'accensione e la configurazione dell'infrastruttura di base. Attraverso il bus I2C, il processore ARM invia una serie di comandi esadecimali ai registri interni del codec audio ADAU1761/SSM2603 tramite la funzione `LineinLineoutConfig()`. Questa sequenza accende i convertitori ADC e DAC, imposta il volume e configura l'interfaccia digitale in modalità I2S a 24-bit. Successivamente, si interviene sul blocco AXI GPIO per disabilitare il segnale di MUTE hardware:

```
// Disabilita il MUTE tramite accesso crudo in memoria
Xil_Out32(XPAR_AXI_GPIO_0_BASEADDR, 0b1);
```

Questa istruzione rappresenta un classico esempio di approccio *bare-metal* con *Memory-Mapped I/O*: si scrive direttamente un '1' logico all'indirizzo base del GPIO per alzare il pin fisico.

6.2.2 Inizializzazione HLS e Interrupts

Subito dopo, le due istanze degli acceleratori crittografici (`scrambler_tx` e `descrambler_rx`) vengono associate ai rispettivi driver tramite `hls_scrambler_init()` e collegate al Generic Interrupt Controller (GIC) dell'ARM. Le *Interrupt Service Routine* (ISR) associate aggiorneranno dei *flag* globali software (`ResultAvailScrambler_tx` e `rx`) non appena l'hardware HLS alzerà a livello logico alto il bit di stato `ap_done`. Questo specifico segnale, generato a livello RTL non appena il campione è stato cifrato, viaggia attraverso la PL e innesca fisicamente l'interrupt che risveglia il processore per la lettura del risultato.

6.3 Setup Interattivo (Interfaccia Utente via UART)

Per garantire la flessibilità del sistema, l'applicativo non utilizza chiavi crittografiche pre-impostate, ma richiede l'intervento dell'utente tramite terminale seriale (UART).

L'utente, alla connessione tramite un emulatore di terminale (es. PuTTY o TeraTerm), visualizzerà un *prompt* che richiede l'inserimento dei due *Seed* iniziali per i generatori LFSR. Questa logica è gestita dalla funzione custom `read_hex_from_uart()`, progettata per essere robusta contro errori di digitazione:

```
current_tx_seed = read_hex_from_uart("Inserisci il Seed Trasmettitore: ");
current_rx_seed = read_hex_from_uart("Inserisci il Seed Ricevitore: ");
```

Un aspetto progettuale di estremo rilievo riguarda la gestione del **Seed nullo (0000)**. Come noto, un LFSR inizializzato a zero rimane bloccato permanentemente in tale stato. Nel software sviluppato, in ottica di *Hardware/Software Co-Design*, il processore ARM non blocca l'inserimento del seed nullo e non consuma cicli di clock per correggerlo matematicamente, ma si limita a informare l'utente:

```
if (result == 0) {
    // Il controllo reale e' delegato all'hardware generato in HLS!
    xil_printf("\n\r[AVVISO] Seed nullo. L'HW usera' il seed di default.
        ");
}
```

Il seed invalido verrà poi iniettato nel blocco HLS, il quale (essendo stato sintetizzato con un `if` di controllo interno) forzerà autonomamente lo stato logico al valore di *default*. Questo dimostra un partizionamento ottimale delle responsabilità tra mondo software (interfaccia utente) e mondo hardware (sicurezza e controlli algoritmici).

6.4 Gestione del Flusso Audio e Routing Dinamico

Acquisite le chiavi e caricate fisicamente nell'hardware tramite la macro `XScrambler_Set_seed`, il software entra nel loop infinito all'interno della funzione `filter_or_bypass_input()`.

Questa *routine* agisce in polling stretto sui registri I2S: non appena un nuovo campione stereofonico (destro e sinistro) è disponibile in ingresso dall'ADC, il processore lo legge, ne controlla il routing e lo spara in uscita verso il DAC. Prima di elaborare il dato, viene letto lo stato logico degli switch fisici della scheda tramite la porta GPIO di ingresso:

```
1 sw_check = Xil_In32(XPAR_AXI_GPIO_0_BASEADDR + 8);
```

6.4.1 Sincronizzazione e Ripristino

Poiché i due scrambler generano una sequenza pseudo-casuale in costante evoluzione, qualsiasi interruzione del flusso disallinerebbe la chiave tra TX e RX, compromettendo irrimediabilmente la decrittazione. Per ovviare a questo problema, il software implementa un algoritmo di *edge-detection* software (rilevamento del fronte) in polling: memorizzando lo stato precedente degli switch (`last_sw_check`), il sistema si accorge dell'istante esatto in cui l'utente cambia modalità. In quel preciso momento, il software ri-forza (*load_ seed=1*) le chiavi iniziali in entrambi gli acceleratori fisici, assicurando che le due macchine a stati ripartano in perfetta e rigorosa sincronia, e stampando a video la modalità attualmente in uso.

6.4.2 Analisi delle Modalità di Routing

In base alla configurazione a 2 bit dei due selettori fisici (`SW0` e `SW1`), il software esegue un instradamento condizionale tramite un costrutto `if-else if` all'interno del loop principale. Per le modalità che prevedono l'utilizzo dell'acceleratore hardware, l'elaborazione di un singolo campione richiede tre passaggi fondamentali invocati tramite i driver HLS:

1. **Scrittura (Set):** Il software adatta il campione a 16 bit tramite uno *shift* (`u32DataL >> 8`) e lo invia al registro AXI dell'IP tramite l'API `XScrambler_Set_x()`.
2. **Avvio e Attesa (Start & Wait):** Il processore azzera la variabile di sincronizzazione, comanda fisicamente l'esecuzione del circuito hardware tramite `XScrambler_Start()` e si mette in attesa passiva con un ciclo `while(!ResultAvailScrambler_tx);`. Il ciclo si sblocca solo quando l'IP hardware alza il pin `ap_done`, scatenando l'interrupt che imposta il flag a 1.
3. **Lettura (Get):** Il software preleva il campione crittografato tramite l'istruzione `XScrambler_Get_y()`.

Le quattro modalità operative, illustrate singolarmente nei relativi schemi di flusso architetturali, sono le seguenti:

- **Stato 0 (Bypass Software - SW: 00):** Il segnale letto dall'ADC viene prelevato dal processore tramite il bus AXI e riscritto inalterato verso i registri del DAC. L'utente ascolterà la propria voce limpida. Pur trattandosi di un bypass, l'instradamento avviene interamente via software (*Memory-Mapped I/O*), lasciando gli scrambler HLS disalimentati logicamente (non ricevono il comando *ap_ start*).

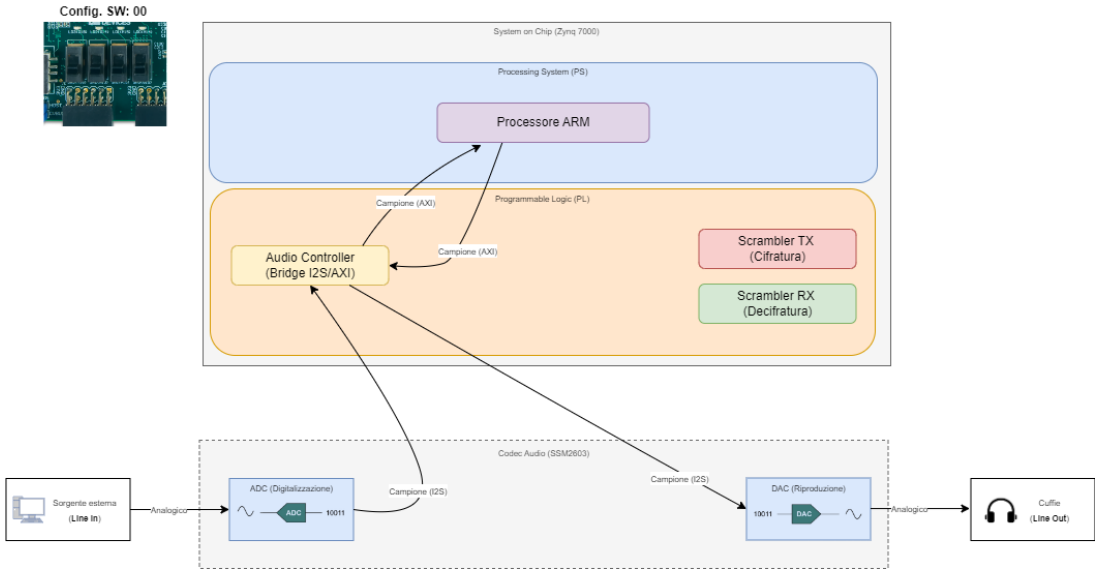


Figura 6.2: Routing Stato 0 (Bypass Software). Il segnale audio transita attraverso il processore ARM senza coinvolgere gli IP crittografici.

- **Stato 1 (Solo TX - SW: 01):** Il processore instrada il campione verso l'IP `scrambler_tx`, attende l'interrupt di completamento, e preleva il campione mascherato per inviarlo alle casse. L'utente ascolterà solo **rumore bianco**. Questa modalità simula la trasmissione su canale cifrato.

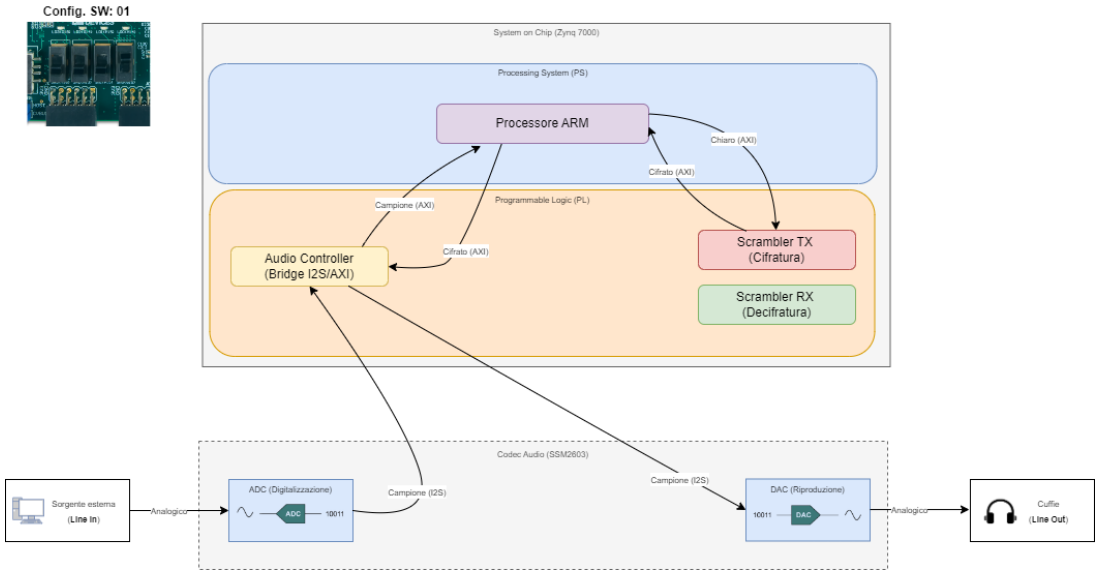


Figura 6.3: Routing Stato 1 (Solo TX). Il flusso dati attraversa esclusivamente l'IP Scrambler TX, generando in uscita il rumore cifrato.

- **Stato 2 (Solo RX - SW: 10):** Identica alla modalità 1, ma utilizza l'IP `descrambler_rx`. Serve a dimostrare che l'operazione di XOR e il generatore polinomiale sono perfettamente simmetrici.

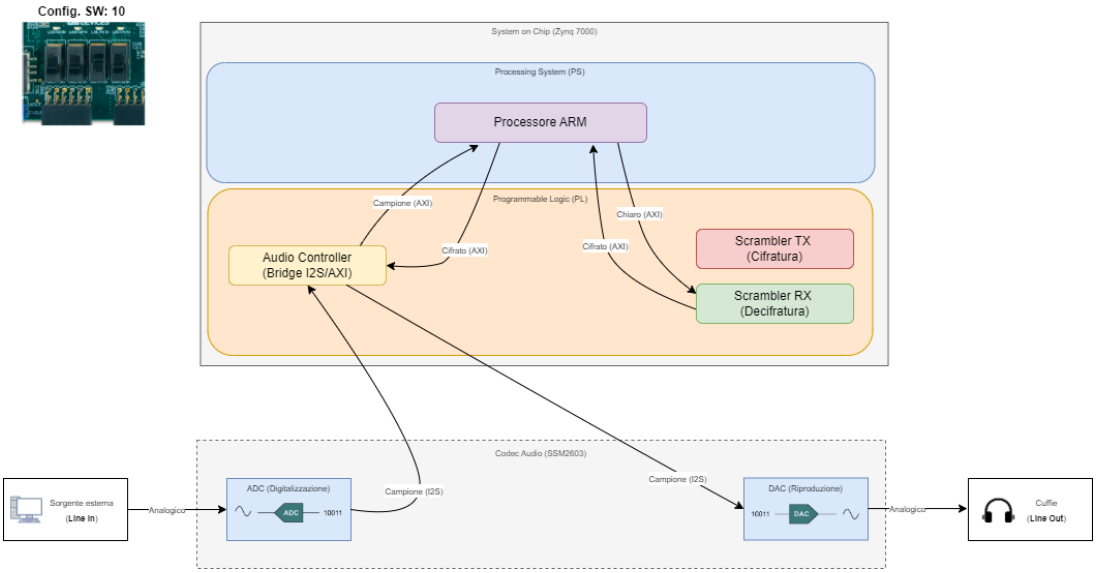


Figura 6.4: Routing Stato 2 (Solo RX). Configurazione simmetrica allo Stato 1, il segnale viene processato unicamente dall'IP Descrambler RX.

- **Stato 3 (Loopback a Cascata - SW: 11):** Questa è la modalità di validazione *End-to-End*. Il campione vocale entra nel blocco TX venendo distrutto (rumore), ma il risultato cifrato viene immediatamente instradato nell'input del blocco RX. Se le chiavi inserite via UART coincidono, l'operazione logica XOR $\oplus$ XOR ristabilisce i bit originari. L'utente ascolterà la propria voce intatta, sebbene processata in sequenza da due acceleratori fisici separati.

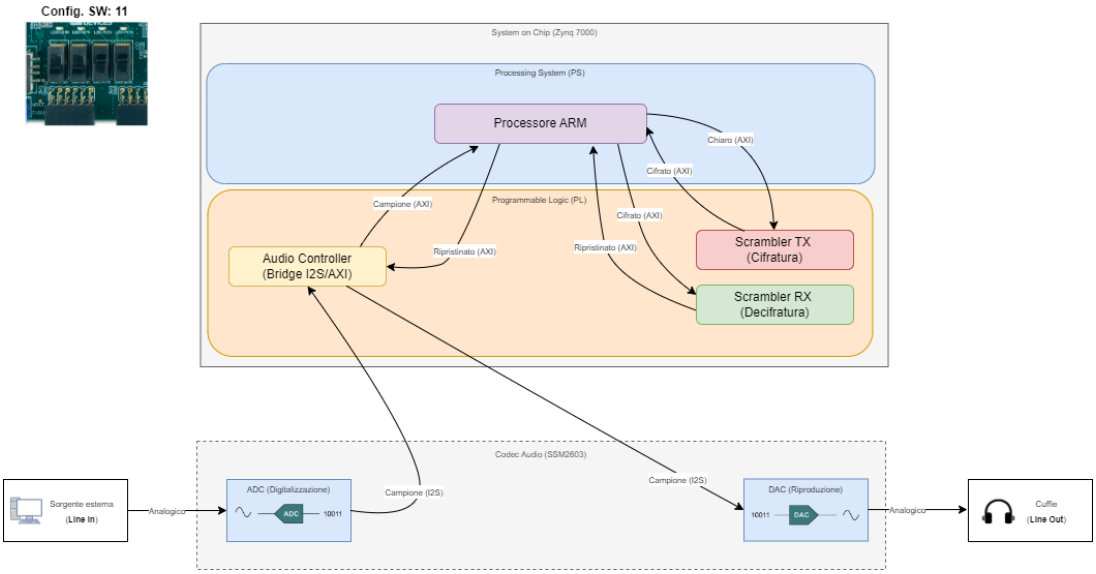


Figura 6.5: Routing Stato 3 (Cascade Loopback). Il campione attraversa in sequenza TX e RX. La doppia operazione logica annulla la cifratura, restituendo l'audio originale in output.

L'aver incapsulato l'intera complessità crittografica in moduli hardware ha permesso di mantenere il codice C estremamente snello, compatto e focalizzato esclusivamente sull'orchestrazione del sistema, centrando in pieno l'obiettivo del progetto.

6.4.3 Re-Keying Dinamico in Tempo Reale

L'architettura software prevede inoltre la possibilità di modificare le chiavi crittografiche (Seed) a runtime, senza la necessità di riavviare il sistema SoC. Questo meccanismo, noto in ambito crittografico come *re-keying*, è fondamentale per mantenere elevata la sicurezza (COMSEC) durante sessioni di trasmissione prolungate.

L'implementazione è gestita attraverso un controllo non bloccante (*polling*) sulla periferica UART, posizionato all'inizio di ogni iterazione del loop audio principale. Se il processore rileva la pressione del tasto dedicato (`'c'` o `'C'`), il flusso audio I2S viene temporaneamente sospeso. Il software provvede innanzitutto a svuotare il buffer della coda seriale (*flush*) da eventuali caratteri di invio residui per prevenire falsi input. Successivamente, richiama la routine `read_hex_from_uart()` per l'acquisizione delle nuove chiavi.

Una volta confermati i nuovi Seed, il processore aggiorna immediatamente i registri hardware degli IP Scrambler tramite transazioni AXI4-Lite, forza la reinizializzazione degli stati logici (`load_seed = 1`) e ripristina istantaneamente lo streaming audio con le nuove configurazioni di sicurezza applicate.

6.5 Risultati Sperimentali

La validazione pratica del sistema è stata condotta interfacciando la piattaforma hardware Zybo Z7 con sorgenti e destinazioni audio reali, al fine di valutarne il comportamento in condizioni operative e in tempo reale.

Il setup sperimentale ha previsto il collegamento di una sorgente audio esterna all'ingresso analogico *Line In* (Jack Azzurro) del codec SSM2603, l'inserimento di un paio di cuffie di monitoraggio nell'uscita *Headphone Out* (Jack Nero) e la connessione della porta UART a un PC host tramite emulatore di terminale (TeraTerm) impostato a 115200 baud.

Alla prima accensione, il terminale ha confermato l'avvenuta configurazione I2C del codec e ha richiesto l'inserimento dei parametri crittografici. Le prove pratiche si sono articolate in due scenari principali:

1. **Test di Sincronia e Ripristino (Chiavi Identiche):** Inserendo lo stesso *Seed* (es. `0x1234`) sia per il modulo TX che per il modulo RX, e commutando i selettori hardware nello Stato 3 (Cascade Loopback), il segnale audio in cuffia è risultato **perfettamente limpido e intellegibile**, privo di latenze percettibili o artefatti acustici. La transizione "al volo" (*on-the-fly*) tra lo Stato 0 (Bypass) e lo Stato 3 non ha introdotto interruzioni o *glitch* sonori, dimostrando l'efficacia della routine software di auto-sincronizzazione che ripristina istantaneamente le macchine a stati degli LFSR.

2. **Test di Sicurezza (Mismatch delle Chiavi):** Al fine di emulare un attacco di intercettazione o un errore di configurazione, il sistema è stato riavviato inserendo un *Seed* di trasmissione (`0x1234`) differente da quello di ricezione (`0x9999`). Impostando nuovamente gli switch nello Stato 3, l'audio in cuffia si è presentato come **puro rumore bianco**, rendendo impossibile il riconoscimento della sorgente musicale originale. Questo risultato pratico convalida empiricamente la sensibilità della cifratura a flusso (Stream Cipher) alle condizioni iniziali: la minima deviazione dello stato dell'LFSR porta a una sequenza pseudo-casuale divergente, rendendo la decodifica impossibile.

3. **Test di Robustezza (Inserimento Seed Nullo):** Per validare la tolleranza agli errori dell'interfaccia Hardware/Software, è stato deliberatamente digitato il valore `0000` come chiave di inizializzazione nel terminale UART. Come previsto in fase di progetto, il software si è limitato a notificare l'anomalia all'utente senza generare eccezioni bloccanti, iniettando il dato nullo nell'acceleratore. L'ascolto in cuffia ha confermato l'immediato e corretto innesco del mascheramento crittografico: la logica HLS ha autonomamente rilevato lo zero,

ripiegando all'istante sulla chiave di *default* (`0xACE1`) e salvando l'LFSR dal blocco critico permanente.

4. **Test di Re-Keying Dinamico (Cambio Chiave in Corsa):** Durante la riproduzione audio continua in modalità crittografata (Stato 3), è stata inviata la lettera `'c'` tramite terminale. Il sistema ha immediatamente sospeso il routing audio (producendo silenzio totale in cuffia) e ha presentato il prompt per l'inserimento delle nuove chiavi. Digitando i nuovi Seed esadecimali e confermando con invio, lo streaming è ripreso all'istante con i nuovi parametri crittografici. Questa prova ha confermato l'efficacia del polling non bloccante sull'interfaccia seriale e la reattività dei driver HLS nell'aggiornare le macchine a stati della PL *a runtime*, garantendo una transizione fluida e controllata.

Le prove hanno infine dimostrato l'assoluta robustezza dell'infrastruttura: il sistema ha operato stabilmente per diverse ore senza disallineamenti di fase tra campioni I2S, transazioni AXI e interrupt del processore, confermando il corretto dimensionamento dei vincoli temporali (Timing Constraints) analizzati nel Capitolo 5.

6.6 Considerazioni sulle Metodologie di Debugging Hardware

Durante lo sviluppo di sistemi System-on-Chip complessi, il raggiungimento del corretto funzionamento finale è solo una parte del processo; altrettanto fondamentale è la padronanza degli strumenti di indagine per identificare e risolvere i bug (*debugging*).

Nella didattica e nella pratica industriale, quando un IP custom non risponde come previsto sulla scheda fisica, si sconsiglia l'utilizzo esclusivo di funzioni software come la classica `printf`. Quest'ultima, pur essendo di facile implementazione, introduce severi ritardi temporali (delay) nell'esecuzione del processore ARM e altera le temporizzazioni di interazione con l'hardware, rischiando di mascherare o generare finti colli di bottiglia.

Per ovviare a ciò, l'ecosistema Xilinx offre due potenti strumenti per il collaudo hardware a basso livello, i quali tuttavia non sono stati impiegati nel presente progetto. Risulta opportuno motivare tecnicamente tale scelta architetturale.

6.6.1 XSCT (Xilinx Software Command-Line Tool)

L'XSCT è un'interfaccia a riga di comando che permette di pilotare le periferiche mappate in memoria aggirando completamente l'esecuzione del codice C o del sistema operativo. Effettuando un *power cycle* della scheda (per garantire che la FPGA sia vuota e priva di software pregresso), l'utente può inviare comandi TCL manuali per leggere e scrivere direttamente sui registri AXI, verificando l'esatta reazione logica del circuito.

Nel presente progetto, l'utilizzo di XSCT è stato reso **superfluo dall'impiego della Zynq Verification IP (VIP)**, documentata nel Capitolo 5. La VIP ha permesso di simulare l'esatta sequenza di letture e scritture AXI4-Lite (incluso il *polling* sui registri di controllo) direttamente nell'ambiente virtuale di Vivado XSIM. Avendo validato matematicamente e temporalmente le transazioni prima ancora di generare il *bitstream*, la probabilità di un errore di indirizzamento o di un blocco del bus sulla scheda fisica è stata abbattuta drasticamente, rendendo inutile il test manuale via riga di comando a valle dell'implementazione.

6.6.2 ILA (Integrated Logic Analyzer)

L'Integrated Logic Analyzer è un blocco IP inseribile nel *Block Design* che si comporta come un vero e proprio analizzatore di stati logici interno alla FPGA. A differenza di un oscilloscopio esterno, che monitora tensioni analogiche su pochi pin, l'ILA si aggancia a decine di reti interne (es. bus dati a 32 bit) e cattura l'evoluzione temporale degli 1 e degli 0 a ogni colpo di clock.

Spesso viene utilizzato in sinergia con l'XSCT: una scrittura manuale tramite XSCT funge da *trigger* (innesco) per l'ILA, che "fotografa" lo stato dei registri interni in quel preciso istante.

L'omissione dell'ILA in questo elaborato deriva da due considerazioni ingegneristiche:

1. **Natura Deterministica dell'IP:** L'algoritmo crittografico implementato (LFSR di Galois e operatore XOR) è un circuito puramente combinatorio e sequenziale sincrono estremamente lineare. Non presenta macchine a stati complessi, incroci di domini di clock asincroni (*Clock Domain Crossing*) o interazioni imprevedibili con segnali esterni che giustificherebbero un'indagine con analizzatore logico. L'unico comportamento logico richiesto era il mascheramento bit-perfect, ampiamente certificato dalla *C/RTL Co-simulation* di Vitis HLS (Capitolo 4).

2. **Efficienza delle Risorse:** L'ILA è uno strumento altamente invasivo dal punto di vista dell'area occupata sul silicio. Per immagazzinare i campioni catturati ad alta velocità, consuma una vasta quantità di memoria RAM interna (*Block RAM - BRAM*) e risorse logiche (LUT). Non avendone una reale necessità per il troubleshooting, si è evitato di instanziare tale IP, mantenendo l'occupazione delle risorse del SoC al di sotto del 2% e garantendo tempi di sintesi e di *routing* estremamente rapidi.

In conclusione, l'assenza di tecniche di *on-target hardware debugging* nel presente lavoro è il risultato diretto di una strategia di **Validazione Pre-Silicon** (*Shift-Left Testing*). L'aver investito tempo nella modellazione MATLAB, nella simulazione HLS e nel testbench di sistema con la Zynq VIP ha garantito che l'hardware giungesse sulla scheda Zybo Z7 già formalmente corretto, consentendo al processore ARM di orchestrarlo tramite codice C senza alcun intoppo fin dalla prima accensione.

CAPITOLO 7

Conclusioni e Sviluppi Futuri

Il presente elaborato ha documentato l'intero ciclo di progettazione, sviluppo e validazione di un sistema di crittografia audio *real-time* basato su architettura System-on-Chip (SoC) Xilinx Zynq-7000. L'obiettivo primario era la realizzazione di un *Proof of Concept* (PoC) capace di mascherare e ripristinare un segnale vocale, garantendo latenze deterministiche, elevata qualità di elaborazione e flessibilità operativa.

Alla luce dei risultati ottenuti nei test di validazione, è possibile tracciare un'analisi critica dell'architettura implementata, evidenziandone i punti di forza, i limiti strutturali e le potenziali traiettorie di evoluzione.

7.1 Efficienza Architetturale e Paradigma Co-Design

Il successo del progetto risiede fondamentalmente nell'applicazione rigorosa del paradigma di *Hardware/Software Co-Design*. Architetture ibride come lo Zynq permettono di superare i limiti fisiologici dei tradizionali microcontrollori, demandando i task critici al silicio programmabile.

Nel sistema realizzato, il partizionamento delle risorse è stato così distribuito:

- **Data Plane (Logica Programmabile):** Le operazioni intensive, come la generazione del rumore pseudo-casuale (LFSR di Galois a 100 MHz) e lo scrambling *bitwise* dei campioni a 48 kHz, sono state relegate all'hardware. Come confermato dai report di Vitis HLS, questo ha permesso di ottenere una latenza logica pari a zero cicli, traguardo inarrivabile via software.

- **Control Plane (Processing System):** Il processore ARM Cortex-A9 è stato sollevato dal calcolo crittografico iterativo, venendo impiegato esclusivamente per l'orchestrazione: inizializzazione del codec via I2C, gestione dell'interfaccia utente asincrona tramite UART e routing dinamico dei campioni audio sui registri AXI4-Lite.

Un'analisi architetturale impone di valutare la reale convenienza del partizionamento hardware dell'algoritmo crittografico rispetto a un'esecuzione puramente software. Dal punto di vista della mera potenza di calcolo bruta, il processore ARM Cortex-A9 (operante a 667 MHz) possiede risorse largamente sufficienti per calcolare l'algoritmo LFSR a 16 bit. Lavorando a 48 kHz, la finestra temporale tra due campioni è di circa $20,8\ \mu s$ (equivalenti a oltre 13.000 cicli di clock della CPU), entro la quale l'operazione di shift e XOR software verrebbe risolta in poche decine di cicli assembly, senza causare ritardi acustici o perdita di frame.

Tuttavia, la decisione di sintetizzare il core crittografico nella Programmable Logic (PL) è giustificata da quattro vantaggi ingegneristici determinanti che rendono il sistema robusto e scalabile:

1. **Determinismo Temporale (Jitter-free):** L'hardware dedicato garantisce un tempo di esecuzione rigorosamente costante e latenze fisse. L'esecuzione software, pur in configurazione *bare-metal*, è intrinsecamente soggetta a latenze variabili causate da svuotamenti della pipeline, *cache miss* o gestione asincrona degli interrupt di altre periferiche. Questa variabilità introduce un *jitter* temporale che, in sistemi audio multi-canale o in applicazioni real-time critiche, può causare distorsione di fase.

2. **Isolamento Crittografico (Security):** Le chiavi di cifratura (*Seed*) e lo stato interno dell'LFSR risiedono fisicamente nei registri isolati della FPGA. Essendo separati dallo spazio di allocazione dinamica della memoria principale (DDR), risultano immuni a una vasta classe di vulnerabilità tipiche del software (come *buffer overflow* o *memory dump*) mirate all'esfiltrazione delle chiavi crittografiche.

3. **Efficienza Energetica:** L'esecuzione di operazioni *bitwise* ad alta frequenza mappate su blocchi logici statici (LUT e Flip-Flop) riduce il consumo dinamico a frazioni di microwatt. Eseguire la medesima operazione in software implicherebbe il continuo risveglio, *fetch* e attivazione della ALU di un processore *general-purpose* per 48.000 volte al secondo, comportando un dispendio energetico nell'ordine dei milliwatt, fattore critico in apparati radio portatili alimentati a batteria.

4. **Offloading e Scalabilità:** Demandare il calcolo alla PL esonera la CPU da un carico di lavoro iterativo e continuo. Come prospettato successivamente per i futuri scenari di telecomunicazione di rete, il processore dovrà gestire lo stack TCP/IP (lwIP), il routing dei pacchetti UDP e gli interrupt MAC Ethernet. Mantenere la crittografia in software genererebbe un pesante *overhead*, rischiando la saturazione del processore. L'offloading hardware garantisce che la CPU resti libera per la gestione del protocollo di rete. Inoltre, qualora si volesse scalare il sistema verso cifrari industriali complessi (es. AES-256 o ChaCha20), le prestazioni software crollerebbero drasticamente, mentre l'hardware assorbirebbe l'upgrade richiedendo semplicemente una maggiore allocazione spaziale di LUT, mantenendo inalterata la frequenza di elaborazione.

7.1.1 Analisi dei Colli di Bottiglia e Velocizzazione

Un'analisi critica del sistema impone una riflessione sulle reali tempistiche di esecuzione. Sebbene l'IP Core HLS presenti una latenza interna nulla, la co-simulazione ha evidenziato un *Initiation Interval* reale di circa 50 cicli di clock. Questo overhead è interamente imputabile all'interfaccia **AXI4-Lite**, che richiede molteplici transazioni di *handshaking* per ogni singolo campione audio processato.

Tuttavia, rapportando questo dato al dominio dell'applicazione, il sistema **non necessita di ulteriori velocizzazioni**. Lavorando a una frequenza di campionamento audio di 48 kHz, un nuovo campione viene generato dal codec ogni $\sim 20,8\ \mu s$. Poiché il bus della FPGA viaggia a 100 MHz (periodo di 10 ns), l'overhead di 50 cicli si traduce in un tempo di elaborazione di appena $0,5\ \mu s$. Il processore ARM dispone quindi di un immenso margine temporale ($\sim 20\ \mu s$ di inattività) per eseguire le letture e le scritture prima dell'arrivo del campione successivo.

L'adozione di bus ad altissime prestazioni come **AXI4-Stream** associati a controller DMA (*Direct Memory Access*) risulterebbe in questo contesto ingegneristicamente sovradimensionata, comportando un inutile spreco di risorse logiche. Tale *upgrade* diverrebbe invece strettamente necessario solo qualora il sistema venisse scalato per processare decine di flussi audio paralleli contemporaneamente (es. centralini telefonici criptati).

7.2 Limiti di Sicurezza e Scalabilità (Upgrade a 32-bit)

Il principale limite architetturale dell'attuale PoC non risiede nelle prestazioni temporali, bensì nella sicurezza crittografica. Come ampiamente dimostrato in ambiente MATLAB (Capitolo 3), l'implementazione didattica di un LFSR a 16 bit genera un *pattern* di rumore che si ripete ogni 1,36 secondi. Questa periodicità a breve termine espone inevitabilmente la trasmissione all'attacco *Two-Time Pad* (Auto-Ritardo).

Per innalzare il sistema a standard di sicurezza operativi (militari o industriali) e rendere l'attacco matematicamente inapplicabile, è necessario il passaggio a un **LFSR a 32 bit**. Grazie all'approccio *High-Level Synthesis*, questo *upgrade* richiede uno sforzo progettuale minimo:

1. Modificare la precisione dei tipi nel sorgente C (`scrambler.h`) adottando interi a 32 bit (`unsigned int`).
2. Aggiornare la maschera hardware in `scrambler.c` con un polinomio primitivo a 32 bit (es. `0x80200003`).

Poiché l'interfaccia AXI4-Lite dello Zynq è nativamente a 32 bit, l'infrastruttura di comunicazione esistente non subirebbe alcuna modifica o ritardo. L'impatto sul silicio si limiterebbe a un incremento irrisorio di Flip-Flop, pur estendendo il periodo di non-ripetibilità della chiave crittografica a **oltre 24 ore continue** di conversazione.

7.3 Sviluppi Futuri: Architetture Multi-Board e VoIP

L'implementazione presentata opera in modalità locale (Loopback e Cascade), sfruttando i jack analogici della singola scheda Zybo Z7 per dimostrare la fattibilità dell'algoritmo di cifratura e decifratura in tempo reale.

Lo sviluppo naturale di questo progetto prevede la transizione verso un'**architettura Dual-Board distribuita**, trasformando il sistema in un vero e proprio apparato ricetrasmittente **VoIP (Voice over IP) crittografato**. In questo scenario futuro:

- **Trasmissione su Rete Ethernet:** Il campione audio acquisito e cifrato dall'IP HW verrebbe impacchettato dal processore ARM all'interno di pacchetti UDP sfruttando uno stack di rete leggero (come **lwIP** - *lightweight IP*) ed emesso fisicamente attraverso la porta Gigabit Ethernet della Zybo verso una rete LAN o WAN.
- **Ricezione Remota:** Una seconda scheda Zybo Z7, configurata con il medesimo *Seed*, resterebbe in ascolto sulla porta UDP designata. Alla ricezione del *payload*, lo stack lwIP passerebbe i dati cifrati all'acceleratore locale per la decrittazione hardware, prima dell'inoltro finale al DAC per l'ascolto.

Tale implementazione richiederebbe un'attenta gestione della negoziazione del layer fisico (PHY) e una bufferizzazione software atta a mitigare il *jitter* tipico delle reti IP, incapsulando il solido core crittografico hardware (qui realizzato) all'interno di un'infrastruttura di telecomunicazione reale e robusta. Un'ulteriore espansione in logica programmabile potrebbe prevedere l'instanziazione in cascata di **Filtri Digitali (FIR/IIR)**, al fine di equalizzare il segnale vocale prima della cifratura o di sopprimere eventuali disturbi fuori banda.

7.4 Conclusione

In ultima analisi, il progetto ha dimostrato in modo tangibile come la combinazione di flussi di progettazione moderni (Vivado e Vitis HLS) permetta di sviluppare sistemi crittografici dedicati in tempi rapidi e con prestazioni di ordine superiore. Il prototipo ibrido prodotto è pienamente funzionante e ingegneristicamente solido. Esso non solo fornisce una soluzione concreta ed efficiente al mascheramento dell'informazione, ma costituisce una *baseline* architetturale validata, pronta per accogliere evoluzioni verso standard crittografici industriali e protocolli di telecomunicazione telematici.

APPENDICE A

Modellazione e Simulazione di Sistema: Codice MATLAB

In questa appendice viene riportato lo script completo sviluppato in ambiente MATLAB per la validazione ad alto livello dell'algoritmo di scrambling e per l'esecuzione del collaudo di robustezza tramite attacco crittografico.

```
% ========================================================================
% Simulazione Pre-Progettazione (Model-Based Design)
% Scrambler Audio LFSR 16-bit e Dimostrazione Attacco Auto-Ritardo
% ========================================================================
clear; clc; close all;

%% 1. ACQUISIZIONE E CONDIZIONAMENTO DEL SEGNALE AUDIO
[voce, fs] = audioread('messaggio.wav');
if size(voce, 2) > 1
    voce = voce(:, 1); % Forzatura a canale singolo (monofonico)
end

% Quantizzazione a 16 bit per emulare l'architettura dei registri FPGA
voce_int16_raw = int16(voce * 32767); % Salva la versione originale
    senza noise gate
voce_int16 = voce_int16_raw;

% Implementazione Noise Gate Digitale: azzeramento del rumore termico di
    fondo
voce_int16(abs(voce_int16) < 300) = 0;
N = length(voce_int16);

%% 2. GENERAZIONE DELLA CHIAVE DI FLUSSO (LFSR in configurazione di
    Galois)
lfsr_noise = zeros(N, 1, 'int16');
stato = uint16(hex2dec('1234'));     % Inizializzazione del Seed
    crittografico
polinomio = uint16(hex2dec('B400')); % Assegnazione del polinomio
    primitivo

for i = 1:N
    lfsr_noise(i) = typecast(stato, 'int16');
    lsb = bitand(stato, 1);
    stato = bitshift(stato, -1);
    if lsb == 1
        stato = bitxor(stato, polinomio);
    end
end

%% 3. SIMULAZIONE DEL CANALE DI TRASMISSIONE E RICEZIONE
disp('-> Computazione blocco di Cifratura (TX)...');
segnale_criptato = bitxor(voce_int16, lfsr_noise);
segnale_criptato_raw = bitxor(voce_int16_raw, lfsr_noise); % Cifratura
    del segnale senza noise gate
```

```
39
40 disp('-> Computazione blocco di Decifratura (RX)...');
41 % Decifratura ideale assumendo perfetta sincronizzazione del ricevitore
42 segnale_decriptato = bitxor(segnale_criptato, lfsr_noise);
43
44 % Verifica del tasso di errore (BER) post-ricostruzione
45 errori = sum(voce_int16 ~= segnale_decriptato);
46 errori_raw = sum(voce_int16_raw ~= bitxor(segnale_criptato_raw,
       lfsr_noise));
47 fprintf('Errori di ricostruzione (Noise Gate): %d campioni\n', errori);
48 fprintf('Errori di ricostruzione (Raw): %d campioni\n', errori_raw);
49
50 %% 4. SIMULAZIONE ATTACCO CRITTOANALITICO (Auto-Ritardo causale)
51 disp('-> Esecuzione Attacco Two-Time Pad...');
52 % Il ritardo T corrisponde al periodo massimo teorico dell'LFSR a 16 bit
53 ritardo = 65535;
54 segnale_recuperato = zeros(N, 1, 'int16');
55 segnale_recuperato_raw = zeros(N, 1, 'int16'); % Buffer per attacco
       senza noise gate
56
57 % Operazione di XOR tra il campione corrente e il campione nel buffer
       passato
58 for i = (ritardo + 1):N
59     segnale_recuperato(i) = bitxor(segnale_criptato(i), segnale_criptato
           (i - ritardo));
60     segnale_recuperato_raw(i) = bitxor(segnale_criptato_raw(i),
           segnale_criptato_raw(i - ritardo)); % Attacco sul raw
61 end
62
63 %% 5. ESPORTAZIONE DEI FILE AUDIO RISULTANTI
64 audiowrite('1_criptato.wav', double(segnale_criptato)/32768, fs);
65 audiowrite('2_decriptato.wav', double(segnale_decriptato)/32768, fs);
66 audiowrite('3_attacco_noise.wav', double(segnale_recuperato_raw)/32768,
       fs); % Salvataggio senza noise gate
67 audiowrite('4_attacco.wav', double(segnale_recuperato)/32768, fs); %
       Salvataggio con noise gate
68 disp('Esportazione completata. Riferimento per test di ascolto: "4
       _attacco.wav".');
69
70 %% 6. TRACCIAMENTO GRAFICO - PANORAMICA DEL SISTEMA
71 t = (0:N-1) / fs;
72 figure('Name', 'Model-Based Design: Scrambler & Attacco', 'Position',
       [100, 100, 900, 700]);
73
74 subplot(4,1,1);
75 plot(t, voce, 'b'); title('1. Voce Originale M(t) (In Chiaro)');
76 ylabel('Ampiezza'); grid on; ylim([-1 1]);
77
78 subplot(4,1,2);
79 plot(t, double(segnale_criptato)/32768, 'r'); title('2. Segnale
       Trasmesso C(t) (Criptato, Rumore Bianco)');
80 ylabel('Ampiezza'); grid on; ylim([-1 1]);
81
82 subplot(4,1,3);
83 plot(t, double(segnale_decriptato)/32768, 'g'); title('3. Ricevitore
       Legittimo (Decriptato)');
84 ylabel('Ampiezza'); grid on; ylim([-1 1]);
85
```

```
subplot(4,1,4);
plot(t, double(segnale_recuperato)/32768, 'k'); title('4. Attaccante: Segnale Recuperato tramite Auto-Ritardo');
xlabel('Tempo (s)'); ylabel('Ampiezza'); grid on; ylim([-1 1]);

%% 7. TRACCIAMENTO GRAFICO - SCOMPOSIZIONE MATEMATICA DELLO XOR
% Traslazione dell'array per consentire il corretto allineamento visivo
segnale_criptato_traslato = zeros(N, 1, 'int16');
segnale_criptato_traslato((ritardo+1):N) = segnale_criptato(1:(N-ritardo));

figure('Name', 'Dettaglio dell''Attacco: I Segnali in XOR', 'Position', [150, 150, 900, 700]);

subplot(3,1,1);
plot(t, double(segnale_criptato)/32768, 'r');
title('1. Operando A: Segnale Criptato Intercettato C(t)');
ylabel('Ampiezza'); grid on; ylim([-1 1]);

subplot(3,1,2);
plot(t, double(segnale_criptato_traslato)/32768, 'm');
title('2. Operando B: Segnale Criptato Ritardato C(t - T)');
ylabel('Ampiezza'); grid on; ylim([-1 1]);

subplot(3,1,3);
plot(t, voce, 'b', 'LineWidth', 3);
hold on;
plot(t, double(segnale_recuperato)/32768, 'k--', 'LineWidth', 1.5);
title('3. Risultato: C(t) \oplus C(t - T) (Blu solido: Originale | Nero tratteggiato: Recuperato)');
legend('Voce Originale M(t)', 'Segnale Recuperato', 'Location', 'best');
xlabel('Tempo (s)'); ylabel('Ampiezza'); grid on; ylim([-1 1]);
hold off;

%% 8. TRACCIAMENTO GRAFICO - ZOOM SULLA COLLISIONE ACUSTICA
figure('Name', 'Zoom Collisione Acustica', 'Position', [200, 200, 900, 400]);

plot(t, voce, 'b', 'LineWidth', 3);
hold on;
plot(t, double(segnale_recuperato)/32768, 'k--', 'LineWidth', 1.5);
title('Dettaglio Sovrapposizione: Collisione tra M(t) e M(t-T)');
legend('Voce Originale M(t) ("tre")', 'Segnale Recuperato (Collisione con "uno, due")', 'Location', 'best');
xlabel('Tempo (s)'); ylabel('Ampiezza'); grid on;
ylim([-1 1]);
xlim([4.2 5.5]); % Isola la finestra temporale della collisione finale
hold off;
```

APPENDICE B

Progettazione HLS: Codici Sorgente e Validazione

In questa appendice sono riportati integralmente i file sviluppati nell'ambiente Vitis HLS per la sintesi dell'acceleratore crittografico, comprensivi del *testbench* utilizzato per la validazione algoritmica e l'esportazione dei dati per analisi in MATLAB.

B.1 File: scrambler.h

```
#ifndef SCRAMBLER_H
#define SCRAMBLER_H

#include <stdbool.h>

// Definizione del tipo dato per i campioni audio I2S (16-bit)
typedef short data_t;

// Definizione del tipo dato per lo stato/chiave LFSR (16-bit)
typedef unsigned short seed_t;

// Prototipo della funzione top-level
void scrambler(data_t *y, data_t x, seed_t seed, bool load_seed);

#endif
```

B.2 File: scrambler.c

```
#include "scrambler.h"

void scrambler(data_t *y, data_t x, seed_t seed, bool load_seed) {
    // Direttive HLS per creare i registri AXI-Lite
    #pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=return
    #pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=x
    #pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=y
    #pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=seed
    #pragma HLS INTERFACE mode=s_axilite bundle=scrambler_io port=
        load_seed

    // Lo stato interno deve essere 'static' per conservare il valore
        tra le chiamate
    // Inizializzato a un valore non zero (l'LFSR si blocca se = 0)
    static seed_t lfsr_state = 0xACE1;

    // Se il processore ARM ci dice di caricare un nuovo seed,
        resettiamo l'LFSR
    if (load_seed) {
        if (seed == 0) {
```

```
            lfsr_state = 0xACE1; // Protezione: seed non nullo
        } else {
            lfsr_state = seed;
        }
    }

    // 1. Calcolo del prossimo numero pseudo-casuale (Galois LFSR a 16 bit)
    unsigned short lsb = lfsr_state & 1; // Prende il bit meno significativo
    lfsr_state >>= 1;                    // Shift a destra
    if (lsb) {
        lfsr_state ^= 0xB400;            // Operazione XOR con il polinomio (Tap 16, 14, 13, 11)
    }

    // 2. Crittografia/Decrittografia: XOR tra il campione audio e il numero casuale
    *y = x ^ (data_t)lfsr_state;
}
```

B.3 File: scrambler_test.c

```
#include <stdio.h>
#include "scrambler.h"

// Funzione di validazione matematica del periodo dell'LFSR
int test_lfsr_period() {
    unsigned short start_seed = 0x1234;
    unsigned short current = start_seed;
    unsigned long count = 0;

    printf("\n--- VERIFICA PERIODO MASSIMO LFSR ---\n");
    do {
        // Emulazione software della topologia hardware
        unsigned short lsb = current & 1;
        current >>= 1;
        if (lsb) {
            current ^= 0xB400;
        }
        count++;
    } while (current != start_seed && count < 100000);

    printf("Periodo ciclico calcolato: %lu clock.\n", count);

    if (count == 65535) {
        printf("SUCCESSO: Raggiunta la lunghezza massima teorica.\n");
        return 0;
    } else {
        printf("FALLIMENTO: Polinomio non primitivo.\n");
        return 1;
    }
}

// Funzione per esportare i dati del polinomio corretto verso MATLAB
void genera_dataset_matlab() {
```

```
34      FILE *fp = fopen("lfsr_samples.txt", "w");
35      if (fp == NULL) {
36          printf("Errore nell'apertura del file!\n");
37          return;
38      }
39
40      unsigned short current = 0x1234;
41
42      // Generazione dell'intero periodo massimo
43      for(long i = 0; i < 65535; i++) {
44          fprintf(fp, "%d\n", current);
45
46          unsigned short lsb = current & 1;
47          current >>= 1;
48          if (lsb) {
49              current ^= 0xB400;
50          }
51      }
52
53      fclose(fp);
54      printf("File 'lfsr_samples.txt' generato con successo.\n");
55  }
56
57  // Funzione per esportare i dati di un polinomio sub-ottimale per
        confronto in MATLAB
58  void genera_dataset_matlab_subopt() {
59      FILE *fp = fopen("lfsr_samples_subottimale.txt", "w");
60      if (fp == NULL) {
61          printf("Errore nell'apertura del file!\n");
62          return;
63      }
64
65      unsigned short current = 0x1234;
66      unsigned short bad_poly = 0x0011; // Polinomio sub-ottimale
            arbitrario
67
68      for(long i = 0; i < 65535; i++) {
69          fprintf(fp, "%d\n", current);
70
71          unsigned short lsb = current & 1;
72          current >>= 1;
73          if (lsb) {
74              current ^= bad_poly;
75          }
76      }
77
78      fclose(fp);
79      printf("File 'lfsr_samples_subottimale.txt' generato con successo.\n
            ");
80  }
81
82  int main() {
83      // Definizione di un segnale di test (burst impulsivo)
84      data_t input_signal[10] = {0, 100, -200, 300, -400, 500, -600, 700,
            -800, 900};
85      data_t encrypted_signal[10];
86      data_t decrypted_signal[10];
87
```

```
    data_t out;
    int i;

    printf("--- SIMULAZIONE SCRAMBLER ---\n");

    // FASE 1: Tx (Crittografia)
    scrambler(&out, 0, 0x1234, true); // Load Seed
    for(i=0; i<10; i++) {
        scrambler(&out, input_signal[i], 0, false);
        encrypted_signal[i] = out;
    }

    // FASE 2: Rx (Decrittografia con stesso Seed)
    scrambler(&out, 0, 0x1234, true);
    for(i=0; i<10; i++) {
        scrambler(&out, encrypted_signal[i], 0, false);
        decrypted_signal[i] = out;
    }

    // Verifica integrita' post-trasmissione
    int errors = 0;
    for(i=0; i<10; i++) {
        if(input_signal[i] != decrypted_signal[i]) {
            errors++;
        }
    }

    // Esecuzione dei test matematici e generazione dataset
    int period_errors = test_lfsr_period();
    genera_dataset_matlab();
    genera_dataset_matlab_subopt();

    if (errors == 0 && period_errors == 0) {
        printf("\nTEST COMPLETO SUPERATO CON SUCCESSO!\n");
        return 0; // Il return 0 indica a Vitis HLS che la simulazione e
            ' passata
    } else {
        printf("\nERRORE DURANTE I TEST!\n");
        return 1;
    }
}
```

B.4 Script MATLAB: Analisi Statistica LFSR

Di seguito sono riportati gli script utilizzati per il calcolo dell'autocorrelazione e la verifica delle proprietà di rumore pseudo-casuale dei dataset generati da Vitis HLS.

File: analisi_lfsr_opt.m

```
% ========================================================================
% Analisi Statistica LFSR a 16-bit (Galois)
% Validazione PRBS tramite Autocorrelazione Discreta
% ========================================================================
clear; clc; close all;

%% 1. Caricamento dei dati esportati da Vitis HLS
```

```
filename = 'lfsr_samples.txt';
if ~isfile(filename)
    error('File "%s" non trovato. Verificare il path.', filename);
end
lfsr_data = load(filename);

%% 2. Condizionamento del segnale (Rimozione componente continua)
% La sequenza [1, 65535] possiede un offset DC che maschera la correlazione.
% Si rende il segnale a media nulla per isolare la dinamica AC.
lfsr_data_zero_mean = lfsr_data - mean(lfsr_data);

%% 3. Calcolo dell'Autocorrelazione
[R, lags] = xcorr(lfsr_data_zero_mean);

%% 4. Rendering Grafico
fig = figure('Name', 'Analisi PRBS LFSR', 'Position', [100, 100, 800, 600]);

% Grafico Globale
subplot(2,1,1);
plot(lags, R, 'b');
title('Autocorrelazione Completa del Galois LFSR (Polinomio Ottimale)');
xlabel('Lag (Campioni)');
ylabel('Ampiezza Correlazione');
grid on;
ylim([-max(R)*0.1 max(R)*1.1]);

% Grafico di Dettaglio (Delta di Dirac)
subplot(2,1,2);
stem(lags, R, 'r', 'Marker', '.', 'LineStyle', '-');
title('Dettaglio Picco Centrale (\pm 50 campioni)');
xlabel('Lag (Campioni)');
ylabel('Ampiezza Correlazione');
xlim([-50 50]);
grid on;
```

File: confronto_lfsr.m

```
% ========================================================================
% Confronto Spettrale: Polinomio Primitivo vs Sub-ottimale
% Dimostrazione di vulnerabilita' statistica (periodicita' parassita)
% ========================================================================
clear; clc; close all;

%% 1. Acquisizione Dati
% Assicurati che i nomi dei file corrispondano a quelli generati dal testbench C
file_opt = 'lfsr_samples.txt';
file_sub = 'lfsr_samples_subottimale.txt';

if ~isfile(file_opt) || ~isfile(file_sub)
    error('File di testo non trovati nella directory corrente.');
end

data_opt = load(file_opt);
data_sub = load(file_sub);
```

```
18
19 %% 2. Condizionamento (Rimozione media)
20 data_opt_zm = data_opt - mean(data_opt);
21 data_sub_zm = data_sub - mean(data_sub);
22
23 %% 3. Calcolo Autocorrelazioni
24 disp('Calcolo autocorrelazione polinomio ottimale...');
25 [R_opt, lags_opt] = xcorr(data_opt_zm);
26
27 disp('Calcolo autocorrelazione polinomio sub-ottimale...');
28 [R_sub, lags_sub] = xcorr(data_sub_zm);
29
30 %% 4. Rendering Grafico Comparativo
31 figure('Name', 'Analisi Comparativa Polinomi LFSR', 'NumberTitle', 'off'
       , 'Position', [100, 100, 1000, 500]);
32
33 % Polinomio Primitivo (Ideale)
34 subplot(1,2,1);
35 plot(lags_opt, R_opt, 'b');
36 title('Polinomio Primitivo (0xB400)');
37 xlabel('Lag (Campioni)');
38 ylabel('Autocorrelazione');
39 grid on;
40 ylim([-max(R_opt)*0.1 max(R_opt)*1.1]);
41
42 % Polinomio Sub-ottimale (Dinamica Periodica)
43 subplot(1,2,2);
44 plot(lags_sub, R_sub, 'r');
45 title('Polinomio Sub-ottimale (0x0011)');
46 xlabel('Lag (Campioni)');
47 ylabel('Autocorrelazione');
48 grid on;
49 xlim([-500 500]); % Zoom sulle componenti periodiche di breve durata
50
51 disp('Grafici generati con successo!');
```

APPENDICE C

Integrazione Hardware: Vincoli e Configurazioni (Vivado)

In questa sezione vengono riportati il testbench, i vincoli fisici e le configurazioni utilizzate per l'implementazione del sistema sulla scheda Zybo Z7-20.

C.1 Testbench System-Level (Zynq VIP)

Di seguito è riportato il testbench Verilog sviluppato per eseguire la simulazione del sistema completo (*C/RTL Co-simulation a livello SoC*) tramite la Xilinx Zynq Verification IP, senza l'ausilio di software su processore ARM.

```
1  `timescale 1ns / 1ps
2
3  module tb_scrambler_system();
4
5  // 1. Dichiarazione dei soli segnali utili per il testbench
6      wire BCLK;
7      wire FCLK_CLK1;
8      wire PBDATA;
9      wire PBLRCLK;
10     wire RECLRCLK;
11     wire [0:0] MUTE_tri_o;
12
13     reg RECDAT;
14     reg [1:0] SW_tri_i;
15
16     wire IIC_1_scl_io;
17     wire IIC_1_sda_io;
18
19     // 2. Instanziazione del DUT con i pin flottanti lasciati vuoti
20     design_1_wrapper DUT (
21         .BCLK(BCLK),
22         .DDR_addr(),
23         .DDR_ba(),
24         .DDR_cas_n(),
25         .DDR_ck_n(),
26         .DDR_ck_p(),
27         .DDR_cke(),
28         .DDR_cs_n(),
29         .DDR_dm(),
30         .DDR_dq(),
31         .DDR_dqs_n(),
32         .DDR_dqs_p(),
33         .DDR_odt(),
34         .DDR_ras_n(),
35         .DDR_reset_n(),
36         .DDR_we_n(),
37         .FCLK_CLK1(FCLK_CLK1),
38         .FIXED_IO_ddr_vrn(),
```

```
        .FIXED_IO_ddr_vrp(),
        .FIXED_IO_mio(),
        .FIXED_IO_ps_clk(),
        .FIXED_IO_ps_porb(),
        .FIXED_IO_ps_srstb(),
        .IIC_1_scl_io(IIC_1_scl_io),
        .IIC_1_sda_io(IIC_1_sda_io),
        .MUTE_tri_o(MUTE_tri_o),
        .PBDATA(PBDATA),
        .PBLRCLK(PBLRCLK),
        .RECDAT(RECDAT),
        .RECLRCLK(RECLRCLK),
        .SW_tri_i(SW_tri_i)
    );

    // ------------------------------------------------------
    // 3. Macro per accedere alle API della Zynq VIP
    // ------------------------------------------------------
    `define ZYNQ_VIP DUT.design_1_i.processing_system7_0.inst

    // Variabili per raccogliere le risposte del bus AXI
    reg [1:0] resp;
    reg [31:0] read_data;

    // ------------------------------------------------------
    // 4. Blocco Main di Simulazione
    // ------------------------------------------------------
    initial begin
        // Inizializzazione degli input fisici
        RECDAT = 1'b0;
        SW_tri_i = 2'b11; // Modalita' loopback (Stato 3)

        $display("=====================================================");
        $display("   INIZIO SIMULAZIONE SYSTEM-LEVEL (ZYNQ VIP)      ");
        $display("=====================================================");

        // A. Reset e Inizializzazione della VIP
        $display("[TIME: %0t] Reset del Processing System...", $time);
        `ZYNQ_VIP.fpga_soft_reset(32'h1);
        #200;
        `ZYNQ_VIP.fpga_soft_reset(32'h0);
        #2000;

        // ============================================================
        // FASE 1: INIZIALIZZAZIONE DEL SEED
        // ============================================================
        $display("[TIME: %0t] FASE 1: Caricamento Seed (0x1234)...",
            $time);

        // Scrittura Seed TX (all'indirizzo base assegnato da Vivado: 0
            x40010000 + offset 0x28)
        `ZYNQ_VIP.write_data(32'h40010028, 4, 32'h00001234, resp);

        // Asserzione load_seed = 1 (offset 0x30)
        `ZYNQ_VIP.write_data(32'h40010030, 4, 32'h00000001, resp);

        // PRIMO START: L'hardware si avvia, legge load_seed=1 e carica
            0x1234 nel registro interno
```

```
        `ZYNQ_VIP.write_data(32'h40010000, 4, 32'h00000001, resp);

        // Polling: Attendiamo che l'IP abbia finito di caricare il seed
        read_data = 0;
        while ((read_data & 32'h00000002) == 0) begin
            `ZYNQ_VIP.read_data(32'h40010000, 4, read_data, resp);
            #10;
        end

        // De-asserzione load_seed = 0 (Altrimenti al prossimo start si
           resetta)
        `ZYNQ_VIP.write_data(32'h40010030, 4, 32'h00000000, resp);

        // ============================================================
        // FASE 2: ELABORAZIONE CAMPIONE AUDIO
        // ============================================================
        $display("[TIME: %0t] FASE 2: Invio campione audio (x = 500)..."
            , $time);

        // Scrittura del Campione Audio (x = 500 dec = 0x01F4, all'
           offset 0x20)
        `ZYNQ_VIP.write_data(32'h40010020, 4, 32'h000001F4, resp);

        // SECONDO START: L'hardware si avvia, legge x=500, vede
           load_seed=0, ed esegue lo XOR
        `ZYNQ_VIP.write_data(32'h40010000, 4, 32'h00000001, resp);

        // Polling: Aspetto che il calcolo del campione finisca
        read_data = 0;
        while ((read_data & 32'h00000002) == 0) begin
            `ZYNQ_VIP.read_data(32'h40010000, 4, read_data, resp);
            #10;
        end

        // Lettura del Campione Criptato in uscita 'y' (offset 0x10)
        `ZYNQ_VIP.read_data(32'h40010010, 4, read_data, resp);

        $display("-------------------------------------------------");
        $display("   RISULTATO LETTO DAL BUS AXI: 0x%08X", read_data);
        $display("-------------------------------------------------");

        #1000;
        $finish;
    end

endmodule
```

C.2 File dei Vincoli Fisici (XDC)

Il seguente listato (`ZyboZ720_audio_constraints.xdc`) definisce il mapping tra le porte logiche del *Block Design* e i pin fisici della FPGA, stabilendo inoltre lo standard elettrico LVCMOS a 3.3V.

```
# ZyBo Z7-20 board - Audio Related constraints
set_property PACKAGE_PIN R19 [get_ports BCLK]
set_property IOSTANDARD LVCMOS33 [get_ports BCLK]

```

```
set_property PACKAGE_PIN T19 [get_ports PBLRCLK]
set_property IOSTANDARD LVCMOS33 [get_ports PBLRCLK]

set_property PACKAGE_PIN Y18 [get_ports RECLRCLK]
set_property IOSTANDARD LVCMOS33 [get_ports RECLRCLK]

set_property PACKAGE_PIN R16 [get_ports RECDAT]
set_property IOSTANDARD LVCMOS33 [get_ports RECDAT]

set_property PACKAGE_PIN R18 [get_ports PBDATA]
set_property IOSTANDARD LVCMOS33 [get_ports PBDATA]

# GPIO[0] output - For MUTE signal (Attivo Basso)
set_property PACKAGE_PIN P18 [get_ports {MUTE_tri_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {MUTE_tri_o[0]}]

# MCLK for the audio codec. it is at 12.288 MHz
set_property PACKAGE_PIN R17 [get_ports FCLK_CLK1]
set_property IOSTANDARD LVCMOS33 [get_ports FCLK_CLK1]

# I2C 1 interface
set_property PACKAGE_PIN N18 [get_ports IIC_1_scl_io]
set_property IOSTANDARD LVCMOS33 [get_ports IIC_1_scl_io]

set_property PACKAGE_PIN N17 [get_ports IIC_1_sda_io]
set_property IOSTANDARD LVCMOS33 [get_ports IIC_1_sda_io]

# GPIO2[1:0] for routing the audio (SW0 and SW1)
set_property PACKAGE_PIN G15 [get_ports {SW_tri_i[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW_tri_i[0]}]

set_property PACKAGE_PIN P15 [get_ports {SW_tri_i[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SW_tri_i[1]}]
```

APPENDICE D

Sviluppo Software: Applicazione Bare-Metal (Vitis)

Di seguito viene riportato il codice sorgente in linguaggio C sviluppato all'interno dell'ambiente Xilinx Vitis per la gestione dell'architettura SoC su scheda Zybo Z7-20. L'applicativo sfrutta i driver auto-generati dal BSP per pilotare le periferiche standard e gli acceleratori HLS custom.

D.1 Header File: zybo_audio.h

L'header file definisce le macro fondamentali per l'accesso ai registri *Memory-Mapped* del controller I2S e per la configurazione del codec audio ADAU1761.

```
#ifndef __AUDIO_H_
#define __AUDIO_H_

#include "xparameters.h"

// Base addresses mappato da Vivado
#define AUDIO_BASE              XPAR_ZYBO_AUDIO_CTRL_0_BASEADDR

// Slave address for the ADAU audio controller
#define IIC_SLAVE_ADDR          0x1A

// I2C Serial Clock frequency in Hertz
#define IIC_SCLK_RATE           100000

// ADAU internal registers
enum audio_regs {
    R0_LEFT_ADC_INPUT           = 0x00,
    R1_RIGHT_ADC_INPUT          = 0x01,
    R2_LEFT_DAC_VOLUME          = 0x02,
    R3_RIGHT_DAC_VOLUME         = 0x03,
    R4_ANALOG_AUDIO_PATH        = 0x04,
    R5_DIGITAL_AUDIO_PATH       = 0x05,
    R6_POWER_MANAGEMENT         = 0x06,
    R7_DIGITAL_AUDIO_INTERFACE  = 0x07,
    R8_SAMPLING_RATE            = 0x08,
    R9_ACTIVE                   = 0x09,
    R15_SOFTWARE_RESET          = 0x0F,
    R16_ALC_CONTROL_1           = 0x10,
    R17_ALC_CONTROL_2           = 0x11,
    R18_NOISE_GATE              = 0x12
};

// Audio controller I2S registers
enum i2s_regs {
    I2S_DATA_RX_L_REG           = 0x00 + AUDIO_BASE,
    I2S_DATA_RX_R_REG           = 0x04 + AUDIO_BASE,
    I2S_DATA_TX_L_REG           = 0x08 + AUDIO_BASE,
    I2S_DATA_TX_R_REG           = 0x0c + AUDIO_BASE,
```

```
    I2S_STATUS_REG              = 0x10 + AUDIO_BASE,
};

#endif
```

D.2 Main Program: zybo_testapp.c

Il listato principale che orchestra l'inizializzazione del sistema, la lettura UART, la gestione degli interrupt hardware, il cambio dinamico delle chiavi (re-keying) e il loop di multiplexing del segnale audio.

```
#include "zybo_audio.h"
#include <stdio.h>
#include <xil_io.h>
#include <sleep.h>
#include "xiicps.h"
#include <xil_printf.h>
#include <xparameters.h>

#include "xscrambler.h"
#include "xuartps.h"
#include "xscutimer.h"
#include "xscugic.h"

typedef short       Xint16;
typedef long        Xint32;
#define TIMER_FREQ_HZ (XPAR_CPU_CORTEXA9_0_CPU_CLK_FREQ_HZ / 2)

unsigned char IicConfig(unsigned int DeviceIdPS);
void AudioWriteToReg(unsigned char u8RegAddr, short u16Data);
void LineinLineoutConfig();

XIicPs Iic;
XScuTimer TimerInstance;

// Istanze HW degli Scrambler
XScrambler scrambler_tx;
XScrambler descrambler_rx;

// Interrupt Controller Instance
XScuGic ScuGic;

// Variabili globali usate dalle ISR per segnalare che il calcolo e'
   pronto
int ResultAvailScrambler_tx, ResultAvailScrambler_rx;

// Variabili globali per i Seed scelti dall'utente (Default: 0x1234)
unsigned short current_tx_seed = 0x1234;
unsigned short current_rx_seed = 0x1234;

// --- FUNZIONE PER LEGGERE IL SEED DA TASTIERA ---
unsigned short read_hex_from_uart(char* prompt_msg) {
    unsigned short result = 0;
    char c;
    int char_count = 0; // Contatore per limitare a 4 cifre

    xil_printf("%s", prompt_msg);
```

```
46
47      while (1) {
48          // Aspetta che arrivi un carattere
49          while (!XUartPs_IsReceiveData(XPAR_PS7_UART_1_BASEADDR));
50          c = XUartPs_ReadReg(XPAR_PS7_UART_1_BASEADDR,
                XUARTPS_FIFO_OFFSET);
51
52          // 1. GESTIONE INVIO (Termina l'inserimento)
53          if (c == '\r' || c == '\n') {
54              if (result == 0) {
55                  // Il controllo reale e' delegato all'hardware generato
                        in HLS!
56                  xil_printf("\n\r[AVVISO] Seed nullo inserito. L'hardware
                        applichera' automaticamente il seed di default (0
                        xACE1).");
57              }
58              xil_printf("\n\r");
59              break;
60          }
61
62          // 2. GESTIONE BACKSPACE (Cancella l'ultimo carattere)
63          else if (c == '\b' || c == 0x7F) {
64              if (char_count > 0) {
65                  xil_printf("\b \b");   // Cancella il carattere
                        visivamente dal terminale
66                  result = result >> 4;  // Annulla matematicamente l'
                        ultimo inserimento
67                  char_count--;
68              }
69          }
70
71          // 3. GESTIONE CARATTERI VALIDI (Max 4 cifre)
72          else if (char_count < 4) {
73              if (c >= '0' && c <= '9') {
74                  result = (result << 4) | (c - '0');
75                  XUartPs_WriteReg(XPAR_PS7_UART_1_BASEADDR,
                        XUARTPS_FIFO_OFFSET, c); // Echo
76                  char_count++;
77              } else if (c >= 'a' && c <= 'f') {
78                  result = (result << 4) | (c - 'a' + 10);
79                  XUartPs_WriteReg(XPAR_PS7_UART_1_BASEADDR,
                        XUARTPS_FIFO_OFFSET, c - 32); // Echo Maiuscolo
80                  char_count++;
81              } else if (c >= 'A' && c <= 'F') {
82                  result = (result << 4) | (c - 'A' + 10);
83                  XUartPs_WriteReg(XPAR_PS7_UART_1_BASEADDR,
                        XUARTPS_FIFO_OFFSET, c); // Echo
84                  char_count++;
85              }
86              // Qualsiasi altra lettera viene ignorata in silenzio
87          }
88      }
89      return result;
90  }
91
92  // -----------------------------------------------------------
93
94  // Inizializzazione degli IP HLS Scrambler
```

```
95  int hls_scrambler_init(XScrambler *scramblerPtr_tx, XScrambler *
        descramblerPtr_rx)
96  {
97      XScrambler_Config *cfgPtr_tx, *cfgPtr_rx;
98      int status;
99
100     // Configura il TX (Scrambler 0 nello schema a blocchi)
101     cfgPtr_tx = XScrambler_LookupConfig(XPAR_XSCRAMBLER_1_DEVICE_ID);
102     if (!cfgPtr_tx) {
103         print("ERROR: Lookup of TX Scrambler configuration failed.\n\r")
                ;
104         return XST_FAILURE;
105     }
106     status = XScrambler_CfgInitialize(scramblerPtr_tx, cfgPtr_tx);
107     if (status != XST_SUCCESS) {
108         print("ERROR: Could not initialize TX Scrambler.\n\r");
109         return XST_FAILURE;
110     }
111
112     // Configura l'RX (Scrambler 1 nello schema a blocchi)
113     cfgPtr_rx = XScrambler_LookupConfig(XPAR_XSCRAMBLER_0_DEVICE_ID);
114     if (!cfgPtr_rx) {
115         print("ERROR: Lookup of RX Descrambler configuration failed.\n\r
                ");
116         return XST_FAILURE;
117     }
118     status = XScrambler_CfgInitialize(descramblerPtr_rx, cfgPtr_rx);
119     if (status != XST_SUCCESS) {
120         print("ERROR: Could not initialize RX Descrambler.\n\r");
121         return XST_FAILURE;
122     }
123
124     return status;
125 }
126
127 int TimerInitialize(void)
128 {
129     int Status;
130     XScuTimer *TimerInstancePtr = &TimerInstance;
131     XScuTimer_Config *ConfigTmrPtr;
132
133     ConfigTmrPtr = XScuTimer_LookupConfig(XPAR_PS7_SCUTIMER_0_DEVICE_ID)
            ;
134     Status = XScuTimer_CfgInitialize(TimerInstancePtr, ConfigTmrPtr,
            ConfigTmrPtr->BaseAddr);
135     if (Status != XST_SUCCESS) return XST_FAILURE;
136
137     XScuTimer_SetPrescaler(TimerInstancePtr, 0);
138     return Status;
139 }
140
141 void TimerDelay(u32 uSDelay)
142 {
143     u32 timerCnt;
144     timerCnt = (TIMER_FREQ_HZ / 1000000) * uSDelay;
145     XScuTimer_Stop(&TimerInstance);
146     XScuTimer_DisableAutoReload(&TimerInstance);
147     XScuTimer_LoadTimer(&TimerInstance, timerCnt);
```

```
148     XScuTimer_Start(&TimerInstance);
149     while (XScuTimer_GetCounterValue(&TimerInstance)) {}
150     return;
151 }
152
153 // ISR per il TX
154 void hls_scrambler_tx_isr(void *InstancePtr)
155 {
156     XScrambler *pAccelerator = (XScrambler *)InstancePtr;
157     XScrambler_InterruptClear(pAccelerator, 1);
158     ResultAvailScrambler_tx = 1;
159 }
160
161 // ISR per l'RX
162 void hls_descrambler_rx_isr(void *InstancePtr)
163 {
164     XScrambler *pAccelerator = (XScrambler *)InstancePtr;
165     XScrambler_InterruptClear(pAccelerator, 1);
166     ResultAvailScrambler_rx = 1;
167 }
168
169 int setup_interrupt()
170 {
171     int result;
172     XScuGic_Config *pCfg = XScuGic_LookupConfig(XPAR_SCUGIC_0_DEVICE_ID)
        ;
173     if (pCfg == NULL) return XST_FAILURE;
174
175     result = XScuGic_CfgInitialize(&ScuGic, pCfg, pCfg->CpuBaseAddress);
176     if(result != XST_SUCCESS) return result;
177
178     result = XScuGic_SelfTest(&ScuGic);
179     if(result != XST_SUCCESS) return result;
180
181     Xil_ExceptionInit();
182     Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT, (
        Xil_ExceptionHandler)XScuGic_InterruptHandler, &ScuGic);
183     Xil_ExceptionEnable();
184
185     // Connette ISR del TX (VEC_ID 1 mappato su
        SCRAMBLER_TX_INTERRUPT_INTR)
186     result = XScuGic_Connect(&ScuGic, XPAR_FABRIC_SCRAMBLER_1_VEC_ID, (
        Xil_InterruptHandler)hls_scrambler_tx_isr, &scrambler_tx);
187     if(result != XST_SUCCESS) return result;
188
189     // Connette ISR dell'RX (VEC_ID 0 mappato su
        SCRAMBLER_RX_INTERRUPT_INTR)
190     result = XScuGic_Connect(&ScuGic, XPAR_FABRIC_SCRAMBLER_0_VEC_ID, (
        Xil_InterruptHandler)hls_descrambler_rx_isr, &descrambler_rx);
191     if(result != XST_SUCCESS) return result;
192
193     XScuGic_Enable(&ScuGic, XPAR_FABRIC_SCRAMBLER_0_VEC_ID);
194     XScuGic_Enable(&ScuGic, XPAR_FABRIC_SCRAMBLER_1_VEC_ID);
195     return XST_SUCCESS;
196 }
197
198 void filter_or_bypass_input(void)
199 {
```

```
200     unsigned long u32DataL, u32DataR;
201     unsigned long u32Temp;
202     int sw_check;
203     int last_sw_check = -1; // Memoria per capire se e' cambiata la
            configurazione degli switch
204     char uart_char;         // Variabile per la lettura del terminale in
            corsa
205
206     while (1)
207     {
208         // --- 0. GESTIONE CAMBIO SEED IN CORSA (Polling non bloccante
                UART) ---
209         if (XUartPs_IsReceiveData(XPAR_PS7_UART_1_BASEADDR)) {
210             uart_char = XUartPs_ReadReg(XPAR_PS7_UART_1_BASEADDR,
                    XUARTPS_FIFO_OFFSET);
211
212             // Se l'utente preme 'c' o 'C' sulla tastiera
213             if (uart_char == 'c' || uart_char == 'C') {
214                 xil_printf("\n\r\n\r***␣PAUSA␣FLUSSO␣AUDIO␣***\n\r");
215                 xil_printf("Procedura␣di␣re-keying␣in␣corso...\n\r");
216
217                 // --- SVUOTAMENTO BUFFER UART (FLUSH) ---
218                 // Consuma eventuali caratteri di 'Invio' (\r o \n)
                        rimasti in coda
219                 while (XUartPs_IsReceiveData(XPAR_PS7_UART_1_BASEADDR))
                        {
220                     XUartPs_ReadReg(XPAR_PS7_UART_1_BASEADDR,
                            XUARTPS_FIFO_OFFSET);
221                 }
222
223                 // Sfrutta la funzione esistente per acquisire i nuovi
                        seed
224                 current_tx_seed = read_hex_from_uart("Inserisci␣Nuovo␣
                        Seed␣TX:␣");
225                 current_rx_seed = read_hex_from_uart("Inserisci␣Nuovo␣
                        Seed␣RX:␣");
226
227                 xil_printf("\n\r[INFO]␣Nuove␣chiavi:␣TX=0x%04X,␣RX=0x%04
                        X\n\r",
228                         (current_tx_seed == 0) ? 0xACE1 :
                            current_tx_seed,
229                         (current_rx_seed == 0) ? 0xACE1 :
                            current_rx_seed);
230
231                 // Aggiornamento hardware TX
232                 XScrambler_Set_seed(&scrambler_tx, current_tx_seed);
233                 XScrambler_Set_load_seed(&scrambler_tx, 1);
234                 XScrambler_Start(&scrambler_tx);
235                 while(!XScrambler_IsDone(&scrambler_tx));
236                 XScrambler_Set_load_seed(&scrambler_tx, 0);
237
238                 // Aggiornamento hardware RX
239                 XScrambler_Set_seed(&descrambler_rx, current_rx_seed);
240                 XScrambler_Set_load_seed(&descrambler_rx, 1);
241                 XScrambler_Start(&descrambler_rx);
242                 while(!XScrambler_IsDone(&descrambler_rx));
243                 XScrambler_Set_load_seed(&descrambler_rx, 0);
244
```

```
245                 // Forza il riallineamento visivo al rientro nel ciclo
246                 last_sw_check = -1;
247                 xil_printf("Hardware aggiornato. Ripresa streaming audio
                        ...\n\r\n\r");
248             }
249         }
250 
251         // --- 1. LETTURA AUDIO E SWITCH ---
252         do { u32Temp = Xil_In32(I2S_STATUS_REG); } while (u32Temp == 0);
253         Xil_Out32(I2S_STATUS_REG, 0x00000001); // Clear data rdy bit
254 
255         u32DataL = Xil_In32(I2S_DATA_RX_L_REG);
256         u32DataR = Xil_In32(I2S_DATA_RX_R_REG);
257 
258         sw_check = Xil_In32(XPAR_AXI_GPIO_0_BASEADDR + 8);
259 
260         // --- 2. AUTO-SINCRONIZZAZIONE AL CAMBIO SWITCH CON I SEED
                INIZIALI ---
261         if (sw_check != last_sw_check) {
262             xil_printf("\n\rCambio configurazione hardware (SW) rilevato
                        . Riallineo con TX: %04X, RX: %04X...\n\r",
263                        (current_tx_seed == 0) ? 0xACE1 : current_tx_seed,
264                        (current_rx_seed == 0) ? 0xACE1 : current_rx_seed);
265 
266             // Ricarica il TX con il seed inserito dall'utente all'avvio
267             XScrambler_Set_seed(&scrambler_tx, current_tx_seed);
268             XScrambler_Set_load_seed(&scrambler_tx, 1);
269             XScrambler_Start(&scrambler_tx);
270             while(!XScrambler_IsDone(&scrambler_tx));
271             XScrambler_Set_load_seed(&scrambler_tx, 0);
272 
273             // Ricarica l'RX con il seed inserito dall'utente all'avvio
274             XScrambler_Set_seed(&descrambler_rx, current_rx_seed);
275             XScrambler_Set_load_seed(&descrambler_rx, 1);
276             XScrambler_Start(&descrambler_rx);
277             while(!XScrambler_IsDone(&descrambler_rx));
278             XScrambler_Set_load_seed(&descrambler_rx, 0);
279 
280             // Stampa la modalita' operativa corrente
281             if (sw_check == 0) {
282                 xil_printf(">>> Modalita' ATTIVA: BYPASS (Pass-through
                            audio pulito)\n\r");
283             } else if (sw_check == 1) {
284                 xil_printf(">>> Modalita' ATTIVA: SOLO TX (Trasmissione
                            Scrambled)\n\r");
285             } else if (sw_check == 2) {
286                 xil_printf(">>> Modalita' ATTIVA: SOLO RX (Ricezione
                            Scrambled)\n\r");
287             } else if (sw_check == 3) {
288                 xil_printf(">>> Modalita' ATTIVA: CASCATA END-TO-END (
                            Cifratura e Decifratura)\n\r");
289             }
290 
291             last_sw_check = sw_check; // Aggiorna lo stato
292         }
293 
294         // --- 3. ROUTING AUDIO ---
295         if (sw_check == 0) {
```

```
296             // Caso 0: BYPASS (Voce pulita)
297         }
298         else if (sw_check == 1) {
299             // Caso 1: SOLO TX -> RUMORE
300             XScrambler_Set_x(&scrambler_tx, u32DataL >> 8);
301             ResultAvailScrambler_tx = 0;
302             XScrambler_Start(&scrambler_tx);
303             while(!ResultAvailScrambler_tx){};
304 
305             u32DataL = XScrambler_Get_y(&scrambler_tx);
306             u32DataR = u32DataL;
307 
308             u32DataL = u32DataL << 8;
309             u32DataR = u32DataR << 8;
310         }
311         else if (sw_check == 2) {
312             // Caso 2: SOLO RX -> RUMORE (Dimostra la simmetria)
313             XScrambler_Set_x(&descrambler_rx, u32DataL >> 8);
314             ResultAvailScrambler_rx = 0;
315             XScrambler_Start(&descrambler_rx);
316             while(!ResultAvailScrambler_rx){};
317 
318             u32DataL = XScrambler_Get_y(&descrambler_rx);
319             u32DataR = u32DataL;
320 
321             u32DataL = u32DataL << 8;
322             u32DataR = u32DataR << 8;
323         }
324         else if (sw_check == 3) {
325             // Caso 3: CASCATA (Cripta e Decripta) -> VOCE PULITA
326             XScrambler_Set_x(&scrambler_tx, u32DataL >> 8);
327             ResultAvailScrambler_tx = 0;
328             XScrambler_Start(&scrambler_tx);
329             while(!ResultAvailScrambler_tx){};
330             short encrypted_sample = XScrambler_Get_y(&scrambler_tx);
331 
332             XScrambler_Set_x(&descrambler_rx, encrypted_sample);
333             ResultAvailScrambler_rx = 0;
334             XScrambler_Start(&descrambler_rx);
335             while(!ResultAvailScrambler_rx){};
336             u32DataL = XScrambler_Get_y(&descrambler_rx);
337             u32DataR = u32DataL;
338 
339             u32DataL = u32DataL << 8;
340             u32DataR = u32DataR << 8;
341         }
342 
343         Xil_Out32(I2S_DATA_TX_L_REG, u32DataL);
344         Xil_Out32(I2S_DATA_TX_R_REG, u32DataR);
345     }
346 }
347 
348 int main(void)
349 {
350     int status;
351 
352     IicConfig(XPAR_XIICPS_0_DEVICE_ID);
353     TimerInitialize();
```

```
354     LineinLineoutConfig();
355     xil_printf("\n\r--- SYSTEM STARTUP ---\n\r");
356     xil_printf("ADAU1761 configured\n\r");
357
358     // Disabilita il MUTE tramite accesso crudo in memoria
359     Xil_Out32(XPAR_AXI_GPIO_0_BASEADDR, 0b1);
360
361     // Setup degli Scrambler
362     status = hls_scrambler_init(&scrambler_tx, &descrambler_rx);
363     if(status != XST_SUCCESS) return(-1);
364
365     // Setup degli interrupt
366     status = setup_interrupt();
367     if(status != XST_SUCCESS) return(-1);
368
369     // Abilita gli interrupt globali per gli IP
370     XScrambler_InterruptEnable(&scrambler_tx, 1);
371     XScrambler_InterruptGlobalEnable(&scrambler_tx);
372     XScrambler_InterruptEnable(&descrambler_rx, 1);
373     XScrambler_InterruptGlobalEnable(&descrambler_rx);
374
375     ResultAvailScrambler_tx = 0;
376     ResultAvailScrambler_rx = 0;
377
378     // --- SETUP INTERATTIVO DELLA CHIAVE CRITTOGRAFICA ---
379     xil_printf("\n\r------------------------------------------------\n
        \r");
380     xil_printf(" INIZIALIZZAZIONE SCRAMBLER AUDIO \n\r");
381     xil_printf("------------------------------------------------\n\r")
        ;
382
383     current_tx_seed = read_hex_from_uart("Inserisci il Seed per il
        Trasmettitore (es. 1234): ");
384     current_rx_seed = read_hex_from_uart("Inserisci il Seed per il
        Ricevitore     (es. 1234): ");
385
386     xil_printf("\n\r[INFO] CHIAVI EFFETTIVE HW: TX=0x%04X, RX=0x%04X\n\r
        ",
387             (current_tx_seed == 0) ? 0xACE1 : current_tx_seed,
388             (current_rx_seed == 0) ? 0xACE1 : current_rx_seed);
389
390     if(current_tx_seed != current_rx_seed) {
391         xil_printf("[ATTENZIONE] Le chiavi sono DIVERSE. La
            decrittazione fallira'!\n\r");
392     } else {
393         xil_printf("[OK] Le chiavi coincidono. La comunicazione sara'
            sicura.\n\r");
394     }
395     xil_printf("------------------------------------------------\n\r")
        ;
396     // ------------------------------------------------
397
398     // Inizializzazione Hardware iniziale con le chiavi scelte
399     XScrambler_Set_seed(&scrambler_tx, current_tx_seed);
400     XScrambler_Set_load_seed(&scrambler_tx, 1);
401     XScrambler_Start(&scrambler_tx);
402     while(!XScrambler_IsDone(&scrambler_tx));
403     XScrambler_Set_load_seed(&scrambler_tx, 0);
```

```
404
405     XScrambler_Set_seed(&descrambler_rx, current_rx_seed);
406     XScrambler_Set_load_seed(&descrambler_rx, 1);
407     XScrambler_Start(&descrambler_rx);
408     while(!XScrambler_IsDone(&descrambler_rx));
409     XScrambler_Set_load_seed(&descrambler_rx, 0);
410
411     xil_printf("TUTTO␣PRONTO.␣Avvio␣flusso␣audio...\n\r");
412     filter_or_bypass_input();
413
414     return 0;
415 }
416
417 unsigned char IicConfig(unsigned int DeviceIdPS)
418 {
419     XIicPs_Config *Config;
420     int Status;
421
422     //Initialize the IIC driver so that it's ready to use
423     //Look up the configuration in the config table, then initialize it.
424     Config = XIicPs_LookupConfig(DeviceIdPS);
425     if(NULL == Config) {
426         return XST_FAILURE;
427     }
428
429     Status = XIicPs_CfgInitialize(&Iic, Config, Config->BaseAddress);
430     if(Status != XST_SUCCESS) {
431         return XST_FAILURE;
432     }
433
434     //Set the IIC serial clock rate.
435     XIicPs_SetSClk(&Iic, IIC_SCLK_RATE);
436
437     return XST_SUCCESS;
438 }
439
440
441 /*************************************************************
442  * Function to write 9-bits to one of the registers from the audio
443  * controller.
444  * @param   u8RegAddr is the register address.
445  * @param   u16Data is the data word to write ( only least significant 9
        bits).
446  * @return  none.
447  *************************************************************/
448 void AudioWriteToReg(unsigned char u8RegAddr, short u16Data) {
449
450     unsigned char u8TxData[2];
451
452     u8TxData[0] = (u8RegAddr << 1 ) | ((u16Data >> 8) & 0x01); // append
        msb of 9-bit data to the reg addr after shifting left
453     u8TxData[1] = u16Data & 0xFF;
454
455     XIicPs_MasterSendPolled(&Iic, u8TxData, 2, IIC_SLAVE_ADDR);
456     while(XIicPs_BusIsBusy(&Iic));
457 }
458
459 /*************************************************************
```

```
460  * Configures Line-In input, ADC's, DAC's, Line-Out and HP-Out.
461  * @param   none.
462  * @return  none.
463  ************************************************************/
464 void LineinLineoutConfig() {
465 
466     // software reset
467     AudioWriteToReg(R15_SOFTWARE_RESET, 0x000);
468     TimerDelay(75000);
469     // power mgmt: 0_00110010=>0,Power up, power up, OSC dn, out off,
        DAC up, ADC up, MIC off, LineIn up
470     //  AudioWriteToReg(R6_POWER_MANAGEMENT, 0x030);
471     AudioWriteToReg(R6_POWER_MANAGEMENT, 0x032);
472     // left ADC Input: 0_01010111=>0,mute disable, Line volume 0 dB
473     AudioWriteToReg(R0_LEFT_ADC_INPUT,0x017);
474     // right ADC Input: 0_00010111=>0,mute disable, Line volume 0 dB
475     AudioWriteToReg(R1_RIGHT_ADC_INPUT,0x017);
476     AudioWriteToReg(R2_LEFT_DAC_VOLUME,0x079);
477     AudioWriteToReg(R3_RIGHT_DAC_VOLUME,0x079);
478     // analog audio path: 0_00010010=>0,-6 dB side attenuation, sidetone
         off, DAC selected, bypass disabled, line input, mic mute
        disabled, 0 dB mic
479     AudioWriteToReg(R4_ANALOG_AUDIO_PATH, 0x012);
480     // digital audio path: 0_00000000=>0_000, clear offset, no mute, no
        de-emphasize, adc high-pass filter enabled
481     AudioWriteToReg(R5_DIGITAL_AUDIO_PATH, 0x000);
482     // digital audio interface: 0_00001110=>0, BCLK not inverted, slave
        mode, no l-r swap, normal LRC and PBRC, 24-bit, I2S mode
483     AudioWriteToReg(R7_DIGITAL_AUDIO_INTERFACE, 0x00A);
484     TimerDelay(75000);
485     // Digital core:0_00000001=>0_0000000, activate core
486     AudioWriteToReg(R9_ACTIVE, 0x001);
487     // power mgmt: 0_00100010 0_Power up, power up, OSC dn, out ON, DAC
        up, ADC up, MIC off, LineIn up
488     AudioWriteToReg(R6_POWER_MANAGEMENT, 0x022); // power mgmt:
        001100010 turn on OUT
489 
490 }
```

Elenco delle figure

1.1 I terminali di trasmissione e ricezione del sistema SIGSALY (1943). Sviluppato dai Bell Telephone Laboratories, è stato utilizzato durante la Seconda Guerra Mondiale per le comunicazioni confidenziali al massimo livello statale tra Winston Churchill e Franklin D. Roosevelt. L'enorme ingombro fisico dell'apparato evidenzia il salto tecnologico rispetto all'implementazione su un singolo chip (SoC) moderno (Fonte: Crypto Museum). 4
1.2 Rappresentazione schematica del flusso di progettazione *Hardware/Software Co-Design*: dalla modellazione ad alto livello fino all'implementazione *bare-metal*. . . 6

2.1 Diagramma a blocchi concettuale ad alto livello del sistema. Si evidenzia il partizionamento Hardware/Software: il Processing System (ARM) funge da nodo di instradamento, mentre la Programmable Logic ospita gli acceleratori crittografici e l'Audio Controller per interfacciarsi con il codec esterno. 7
2.2 Schema a blocchi di un LFSR in configurazione di Galois. Il bit uscente (Output) funge da linea di retroazione (Feedback): viene inserito come nuovo bit iniziale e contemporaneamente sommato ai bit traslati in corrispondenza dei tap. (Fonte: Wikipedia) 8
2.3 Diagramma a blocchi funzionale del codec SSM2603, che illustra il percorso del segnale dai convertitori ADC al processore digitale e ai DAC di uscita. (Fonte: Analog Devices SSM2603 Datasheet) 11
2.4 Panoramica della scheda Zybo Z7. Le porte analogiche collegate al codec audio (Line In, Mic In, Headphone Out) sono evidenziate dal callout numero 16. (Fonte: Digilent Zybo Z7 Reference Manual) 12

3.1 Tracciati d'onda della simulazione Model-Based Design (MATLAB). Il grafico inferiore (4) mostra il segnale recuperato tramite Auto-Ritardo. 17
3.2 Dettaglio analitico delle componenti in XOR. La sovrapposizione tra l'output dell'attacco (nero) e la traccia nativa (blu trasparente) evidenzia visivamente le zone di perfetta convergenza e le aree di collisione distruttiva. 18
3.3 Dettaglio ingrandito della collisione acustica nell'intervallo temporale finale. L'asse delle ascisse è limitato alla finestra $t \in [4.2s, 5.5s]$. È visibile come il segnale recuperato (tratteggio nero, corrispondente alle parole "uno, due" ritardate) sovrasti e distrugga l'ampiezza della traccia nativa (linea blu continua, corrispondente alla parola "tre"), rendendone impossibile l'estrazione in chiaro. 19

4.1 Autocorrelazione della sequenza ottimale. Il picco netto a zero indica che il rumore è imprevedibile. 26
4.2 Confronto dell'autocorrelazione tra l'LFSR con polinomio primitivo (sinistra) e l'LFSR con configurazione sub-ottimale (destra). L'effetto pettine nel grafico di destra evidenzia una severa criticità nella distribuzione pseudo-casuale. 27
4.3 Synthesis Summary Report di Vitis HLS, con dettaglio sulle stime temporali (Timing Estimate), le prestazioni algoritmiche (Latenza e Initiation Interval) e l'occupazione delle risorse hardware. 28
4.4 Report della mappatura SW-to-HW. Si nota come le singole porte C (x, y, seed) siano state consolidate sotto l'interfaccia `s_axi_scrambler_io` con specifici offset di memoria. 29
4.5 Dettaglio dei registri S_AXILITE generati. Il registro di controllo (offset `0x00`) espone ai bit 0, 1 e 2 i segnali di handshaking del blocco. 30

4.6 Report di Cosimulazione. Si noti la Latenza confermata a 0 cicli e l'Initiation Interval medio salito a 51 cicli a causa dell'overhead del protocollo AXI. 31

5.1 Schema architetturale completo (*Block Design*) generato in Vivado. Si noti la doppia istanza dello Scrambler (TX e RX), l'interconnessione AXI e la gestione centralizzata dei clock e degli interrupt. 35
5.2 Pannello Address Editor di Vivado. Viene mostrata la mappa di memoria assegnata automaticamente alle periferiche AXI instanziate nella Programmable Logic. 37
5.3 Panoramica globale della simulazione System-Level. Si notano i due cicli di avvio dell'IP e le relative transazioni scambiate sul bus AXI. 40
5.4 Scrittura del Seed (`0x1234` all'offset `0x28`). 41
5.5 Scrittura del Campione (`0x01F4` all'offset `0x20`). 41
5.6 Operazione di Polling: lettura dello stato `0x0E` dal registro di controllo. Si noti il completamento dell'handshake con `RVALID` e `RREADY` alti. 42
5.7 Lettura del risultato finale (`0x0579`) dal registro `y` all'offset `0x10`. 42
5.8 Design Timing Summary. Il parametro WNS positivo garantisce il perfetto soddisfacimento dei vincoli temporali (Setup e Hold). 44
5.9 Report di Utilization. È visibile il contributo infinitesimale dei due IP crittografici rispetto all'intero *wrapper* di sistema. 45
5.10 Rappresentazione grafica dell'occupazione delle risorse hardware post-implementazione. Il grafico superiore illustra la distribuzione del carico logico tra i singoli moduli IP istanziati. Il grafico inferiore evidenzia la percentuale di utilizzo globale rispetto alla capacità fisica totale della matrice FPGA Zynq-7000, confermando l'impatto trascurabile del design sull'area del chip. 46
5.11 Dettaglio dell'utilizzo delle risorse. Il modulo `auto_pc_0` emerge come il componente a maggiore consumo logico nel sistema. 46
5.12 Report di Power Estimation. Si noti il *Confidence level* impostato su "Low", indice di una stima puramente statica e non basata su simulazioni vettoriali reali. 47

6.1 Board Support Package in Vitis. Si noti come il tool abbia generato automaticamente i driver `scrambler` per i due IP custom `scrambler_rx` e `scrambler_tx` istanziati nel Platform Project. 49
6.2 Routing Stato 0 (Bypass Software). Il segnale audio transita attraverso il processore ARM senza coinvolgere gli IP crittografici. 53
6.3 Routing Stato 1 (Solo TX). Il flusso dati attraversa esclusivamente l'IP Scrambler TX, generando in uscita il rumore cifrato. 53
6.4 Routing Stato 2 (Solo RX). Configurazione simmetrica allo Stato 1, il segnale viene processato unicamente dall'IP Descrambler RX. 54
6.5 Routing Stato 3 (Cascade Loopback). Il campione attraversa in sequenza TX e RX. La doppia operazione logica annulla la cifratura, restituendo l'audio originale in output. 54